SIMATS ENGINEERING

PROGRAMMING IN JAVA

ASSIGNMENT 4 – CO4

EVENT-DRIVEN GUI DESIGN

Course: Programming in Java (CSA09)
Course Outcome: CO4 – Applets, the event delegation model, listeners and adapter classes, inner and anonymous classes, AWT components and controls, layout managers, menus and dialogs, Swing components and GUI–JDBC integration (Sessions 49–56) **Nature of Assignment:** Individual
Duration: One Week
Total Marks: 50
Submission Date: _____

Bloom's Levels Assessed: K4 – Analyse and K5 – Evaluate (the level of each task isstated under the task heading)

Mandatory Execution and GitHub Submission

- **Every student must individually execute every task in this assignment**, including all "Complete the Code" programs.
- Create a GitHub repository named **CSA09-<RegisterNumber>-Assignment4**and share it with the course faculty (public, or private with the faculty added as acollaborator).
- Keep one folder for each task (for example: Task01, Task02 …or Q1, Q2 …), containing the Java source code, input files and output screenshots.
- Include a README.md with your name, register number, Java version, the stepsto compile and run each program, and a short note of the result of each task. • Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted.
- Submit the GitHub link below and on the cover page of your report. **An assignment submitted without a working GitHub link will not be evaluated.**

GitHub Repository Link:

_____**How to Answer the**

Additional Questions

Complete the Code: Copy the given code into your repository, replace every TODOwithworking code without changing the rest of the program, execute it and compare your output with the expected output.

Alternative Concept and Complexity Analysis: Describe how the solution wouldchange if the alternative concept were used, with code or pseudocode, and analyse andcompare the time complexity and space complexity of both solutions using Big-Onotation, supported by your own measurements wherever possible.

PURPOSE OF THE ASSIGNMENT

This assignment evaluates your ability to **design an event-driven user interface beforebuilding it**, and to keep the interface separate from the application logic.
You are expected to demonstrate the following progression:

> **Interpret the Requirement → Identify Components and Events →Design the Layout → Design the Handlers → Implement →Trace →Challenge → Refine**

Where pseudocode is required, it must follow the SIMATS Engineering – Standardfor Algorithmic Pseudocode. Implementation must use AWT and/or Swing.

TASK 1 – GUI DESIGN REVIEW **Bloom's Level:** K4 – Analyse

Diagnose and Repair a Poor Event-Handling Design

6 Marks

A junior developer has built a student fee-payment window. Every button is handledbythe same listener, and the complete program logic is written inside it.

```
public class FeeWindow extends Frame implements ActionListener { TextField regNo = new
TextField(); TextField amount = new TextField(); Button pay = new Button("Pay"), clear =
new Button("Clear"), receipt = newButton("Receipt");
  public void actionPerformed(ActionEvent e) {
    if (e.getActionCommand() == "Pay") {
      double a = Double.parseDouble(amount.getText());
      Connection c = DriverManager.getConnection(URL, USER, PWD);
c.createStatement().executeUpdate("INSERT INTO fees VALUES('" + regNo.getText()"'," +
a + ")");
      Thread.sleep(3000); // wait for bank confirmation
      System.out.println("Paid");
    } else { regNo.setText(""); amount.setText(""); }
  }
}
```

Task 1.1 – Diagnose the Design | 2 Marks

Identify at least **five design deficiencies** in this code. Do not focus merely on syntax. Examine aspects such as:

- Separation of interface and logic
- String comparison of the action command
- Handling of invalid or empty input
- Database work performed on the event-dispatch thread and the effect of sleep() on the window
- Feedback to the user
- The Receipt button
- Security of the SQL statement
- Closing of the window and of the database connection
- Checked exceptions that are not handled

For each deficiency, explain briefly what the user would experience or what couldgowrong.

Task 1.2 – Repair the Design | 3 Marks

Redesign the program so that the window only collects input and shows results, andaseparate FeeService class performs validation and payment. Present the class structureand rewrite the event handling.

Task 1.3 – Design Defence | 1 Mark

In not more than 100 words, explain why your redesign is easier to test and maintain. **Alternative**

Concept and Complexity Analysis

If, instead of one listener with an if–else chain on the action command, each button hadits own listener object (or a Swing Action), what would the solution be? Analyse the timeto dispatch an event for k buttons (up to k comparisons versus O(1)) and the space usedby k listener objects.

Evidence Expected

A reasoned diagnosis, a corrected design and a concise justification.

TASK 2 – FROM REQUIREMENT TO EVENT-DRIVEN DESIGN**Bloom's Level:** K4 – Analyse

Hospital Outpatient Appointment Desk

8 Marks

The outpatient desk needs a window in which the receptionist enters a patient ID, selectsa department from a list, chooses a doctor (the doctors shown depend on thedepartment), picks a time slot and books the appointment. The window must showaconfirmation or an error, and must ask for confirmation before closing if a booking is inprogress.

Task 2.1 – Model the Events | 2 Marks

Prepare a table listing each **event source**, the **event object** generated, the **listener interface** and the **action** taken.

Task 2.2 – Design the Component Hierarchy | 2 Marks

Draw the containment tree of frames, panels and components, and name the AWT/Swingclass used for each.

Task 2.3 – Design the Handler Logic | 3 Marks

Write pseudocode for (a) refreshing the doctor list when the department changes, and(b) validating and booking the appointment.

Task 2.4 – Trace the Event Sequence | 1 Mark

Trace the order of method calls from the moment the window is opened to the moment it is closed, including the window-event methods. Compare this briefly with the life-cyclemethods of an applet (init, start, paint, stop, destroy).

Alternative Concept and Complexity Analysis

If the doctor list were fetched from the database every time the department changes, instead of being loaded once into a Map<String, List<Doctor>>, what would the solutionbe? Analyse the time per department change and the memory used for d departmentsand m doctors in each approach.

Evidence Expected

An event table, a component hierarchy, handler pseudocode and a correct event trace.

TASK 3 – FROM SKETCH TO LAYOUT

Bloom's Level: K4 – Analyse

Campus Library Search Screen

9 Marks

The library search screen has: a title bar at the top; a search row with a text field, acategory Choice and a Search button; a results area in the centre; a row of Prev, Next and Reserve buttons at the bottom; and a status message. When the windowis resized, the results area must grow while the search row and buttons keep their size.

Task 3.1 – Choose the Layout Managers | 3 Marks

Select layout managers (FlowLayout, BorderLayout, GridLayout, CardLayout) for eachcontainer and justify each choice with respect to the resizing requirement.

Task 3.2 – Design the Nested Containers | 2 Marks

Draw the nested panel structure showing which layout manager each panel uses. **Task 3.3 –**

Implement the Screen | 3 Marks

Implement the screen in AWT or Swing.

Task 3.4 – Test the Resizing Behaviour | 1 Mark

Provide screenshots at three window sizes and explain whether each region behavedasdesigned.

Alternative Concept and Complexity Analysis

If GridBagLayout or absolute positioning (setLayout(null) with setBounds) were usedinstead of nested BorderLayout and FlowLayout panels, what would the solution be?Compare the resizing behaviour and analyse the layout computation time and thenumber of container objects (space) in each design.

Evidence Expected

Justified layout choices, a nested container diagram, working code and resizing evidence.

TASK 4 – EVENT-HANDLING FORENSICS **Bloom's Level:** K4 – Analyse

7 Marks

A drawing pad is supposed to draw a dot wherever the user clicks and showthe clickcount in a label. Users report that nothing is drawn and the count stays at 0.

```
public class DrawPad extends Frame {
  int count = 0; Label status = new Label("Clicks: 0"); public
  DrawPad() {
    add(status, BorderLayout.SOUTH);
    addMouseListener(new MouseAdapter() {
      public void mouseclicked(MouseEvent e) {
        count++;
        Graphics g = getGraphics(); g.fillOval(e.getX(), e.getY(), 6, 6); }
    });
    setSize(400, 300); setVisible(true);
  }
}
```

Task 4.1 – Locate the Defect | 2 Marks

Identify why the click is never handled and why the label never changes. **Task 4.2 –**

Construct a Failing Interaction | 2 Marks

Even after the first defect is fixed, the dots disappear when the window is minimised andrestored. Describe the interaction that reveals this and explain why it happens.

Task 4.3 – Repair the Program | 2 Marks

Correct the program so that clicks are counted, the label updates and the dots surviverepainting (store the points and draw them in paint()).

Task 4.4 – Prevent Recurrence | 1 Mark

Explain how the @Override annotation would have exposed the first defect at compiletime.

Complete the Code

Complete TODO 1 and TODO 2 below, execute the program and attach screenshotsbefore and after minimising the window.

```
import java.awt.BorderLayout;
import java.awt.Frame;
import java.awt.Graphics;
import java.awt.Label;
import java.awt.Point;
import java.awt.event.MouseAdapter;
import java.awt.event.MouseEvent;
import java.awt.event.WindowAdapter;
import java.awt.event.WindowEvent;
import java.util.ArrayList;
import java.util.List;

public class DrawPad extends Frame {
    private final List<Point> points = new ArrayList<>(); private final
    Label status = new Label("Clicks: 0");

    public DrawPad() {
        super("Draw Pad");
        add(status, BorderLayout.SOUTH);
        addMouseListener(new MouseAdapter() {
            // TODO 1: Override mouseClicked correctly: store the point, update the// label and
            request a repaint

        });
        addWindowListener(new WindowAdapter() {
            @Override public void windowClosing(WindowEvent e) { dispose(); }});
        setSize(400, 300);
    }

    @Override public void paint(Graphics g) {
        // TODO 2: Draw every stored point as a 6 × 6 filled circle centred on the point}

    public static void main(String[] args) { new DrawPad().setVisible(true); }}
```

Remove @Override from TODO 1, misspell the method as mouseclicked, and recordthecompiler message.

Alternative Concept and Complexity Analysis

Your repaired design stores the points and redraws them in paint(). If the dots wereinstead drawn once onto an off-screen BufferedImage that is copied to the screeninpaint(), what would the solution be? Analyse the time per repaint for n clicks (O(n) versuscopying one image) and the space used (O(n) points versus width × height pixels).

Evidence Expected

Correct defect identification, a reproducible failing interaction, a repaired programandaprevention measure.

TASK 5 – SAME SCREEN, DIFFERENT EVENT-HANDLINGSTYLES

Bloom's Level: K5 – Evaluate

7 Marks

A simple currency-converter window has an amount field, a From/To currency selection, a Convert button

and a result label.

Task 5.1 – Four Implementations | 3 Marks

Implement the Convert action in four ways: (a) the frame itself implements ActionListener, (b) a named inner class, (c) an anonymous inner class, (d) a lambda expression. Alsohandle window closing using a WindowAdapter.

Task 5.2 – Change the Requirement | 2 Marks

The converter must now also convert when the user presses Enter in the amount fieldand must show the result in a dialog box. Modify each version and record howmuchcode had to change in each.

Task 5.3 – Compare the Styles | 2 Marks

Compare the four styles on readability, access to the frame's fields, reuse of the handler and ease of change. State which style you would use for a large application and why.

Complete the Code

Complete TODO 1 and TODO 2 below. This is the lambda version (style d). Executeit and attach screenshots of a valid conversion and an invalid amount.

```
import java.awt.FlowLayout;
import java.awt.event.ActionListener;
import javax.swing.JButton;
import javax.swing.JComboBox;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JTextField;
import javax.swing.SwingUtilities;

public class Converter extends JFrame {
    private final JTextField amount = new JTextField(10); private final
    JComboBox<String> target =
            new JComboBox<>(new String[]{"USD", "EUR"});
    private final JLabel result = new JLabel(" ");

    private static double rate(String currency) {
        return currency.equals("USD") ? 0.012 : 0.011;
    }

    public Converter() {
        super("Currency Converter");
        setLayout(new FlowLayout());
        JButton convertButton = new JButton("Convert");
        add(new JLabel("INR")); add(amount); add(target);
        add(convertButton); add(result);
        // TODO 1: Register ONE lambda listener for both the Convert button and the// Enter key
        in the amount field


        setDefaultCloseOperation(EXIT_ON_CLOSE);
        pack();
    }

    private void convert() {
        // TODO 2: Parse the amount and show the converted value with 2 decimals, or// "Invalid
        amount" if parsing fails


    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new Converter().setVisible(true));
    }
}
```

Alternative Concept and Complexity Analysis

If the Convert action were written as a method reference (this::convert) or as one sharedAction object used by the button, a menu item and a keyboard shortcut, what wouldthesolution be? Compare the number of listener objects created (space) and the workneeded to add a new trigger, with the four styles you implemented.

Evidence Expected

Four working implementations, evidence of the change effort and a reasoned comparison.

TASK 6 – DESIGN CHALLENGE

Bloom's Level: K5 – Evaluate

Campus Event Registration Desktop Application

13 Marks

The cultural committee needs a desktop application to manage event registrations. Students register for events through a form; the committee views all registrations inatable, filters them by event and exports the list. The data is stored in a MySQL database. The application must remain responsive while data is loading, must reject invalid input clearly, and must not allow a student to register twice for the same event.

You are not given the design.

Stage 1 – Interpret the Problem | 2 Marks

Identify the users, the screens, the data and the rules. State the requirements as testablestatements.

Stage 2 – Propose the Screen and Event Flow | 2 Marks

Sketch each screen, including a menu bar (MenuBar/JMenuBar), and drawthe flowbetween screens and dialogs.

Stage 3 – Identify the Critical Design Decisions | 2 Marks

Identify where input validation, duplicate checking and database access occur, and howlong-running database work is kept off the event-dispatch thread.

Stage 4 – Design the Classes and Handlers | 4 Marks

Present a class diagram separating the user interface, service and data-access (JDBC) layers, and pseudocode for the Register and Filter actions.

Stage 5 – Implement and Demonstrate | 3 Marks

Implement the application using Swing (JFrame, JPanel, JTextField, JComboBox, JTable, JMenuBar) with JDBC, and demonstrate normal, invalid and duplicate registrations.

Alternative Concept and Complexity Analysis

If all registrations were loaded into the JTable at once, instead of loading one page at atime using SQL LIMIT and OFFSET in a SwingWorker, what would the solution be?Analyse the loading time and the memory used for 100,000 registrations, and the effect on the responsiveness of the window, in both approaches.

Evidence Expected

A complete, layered GUI design with screen flow, handler pseudocode, a working Swing–JDBC application and demonstrated test cases.

MARK DISTRIBUTION

Task	Competency Assessed	Bloom's Level	Marks
Task	1 Diagnose and repair a poor event-handling	design K4 –	Analyse 6
Task	2 Design an event-driven interface fromrequirement	K4 – a	Analyse 8
Task	3 Design and implement layouts		Analyse 9
Task	4 Debug event handling using failing interactions		Analyse 7
Task	5 Compare event-handling styles under changingrequirements		Evaluate 7
Task	6 Independently design and build a layeredJDBC application		GUI–K5 –Evaluate 13
	TOTAL		**50**

Submission Requirements

GitHub repository link (mandatory) containing the executed code for every task, including every completed "Complete the Code" program, with input files, output screenshots and a README.md.

- Report containing diagnoses, tables, diagrams, pseudocode and comparisons • Java source code for Tasks 1.2, 3, 4, 5 and 6
- Screenshots showing each tested interaction
- SQL script for Task 6

A screenshot of a window alone is not sufficient evidence; each screenshot must belinked to the requirement or test it demonstrates.

Assessment Verification

Students may be asked to explain any part of their design or code, trace an event through their handlers, or modify a handler during a brief verification discussion/viva.

<YOUR COLLEGE / DEPARTMENT NAME>

(write your college and department here)

CSA09 – PROGRAMMING IN JAVA

ASSIGNMENT 4 – CO4 REPORT

Event-Driven GUI Design (AWT, Swing, Layouts, Listeners, JDBC)

Item	Details
Student name	<YOUR NAME>
Register number	<REGISTER NUMBER>
Course	CSA09 – Programming in Java
Assignment	Assignment 4 – CO4 (Sessions 49–56), Individual, 50 marks
Java version used	OpenJDK 21 (java -version)
GitHub repository	https://github.com/<your-username>/CSA09-<RegisterNumber>-Assignment4
Submission date	<DATE>

The pages before this cover page are the assignment as issued. The answers follow in the same order as the tasks: Task 1 to Task 6, each with the diagnosis or design, the complete source code, the captured program output, the figures and the alternative-concept analysis. ER diagrams are given for the three tasks that use a database (Tasks 1, 2 and 6).

Contents

Part	Topic	Marks
Task 1	Diagnose and repair a poor event-handling design (fee-payment window)	6
Task 2	From requirement to event-driven design (outpatient appointment desk)	8
Task 3	From sketch to layout (campus library search screen)	9
Task 4	Event-handling forensics (drawing pad) – with Complete the Code	7
Task 5	Same screen, different event-handling styles (currency converter) – with Complete the Code	7
Task 6	Campus event registration desktop application (Swing + JDBC + MySQL)	13
Appendix A	Repository layout, how to compile and run, GitHub commit steps	–
Appendix B	Test harness and headless shims (how the screenshots in this report were produced)	–
Appendix C	Requirement-to-evidence traceability table and viva preparation	–

How the evidence in this report was produced (please read)

Every program in this report was compiled with javac and executed. However, the machine used to prepare this report is a server **without a screen** and without a MySQL server, so some evidence was produced in a different way from a normal desktop run. This is stated here and again under each figure, so that nothing is presented as something it is not.

Evidence	How it was produced	What it proves / what it does not prove
Console output of services, tests, benchmarks	Plain java commands; captured text is pasted unchanged.	Real executions. Timings depend on the machine; trends and operation counts do not.
Screenshots of windows ("off-screen render")	A test harness runs the real window classes in a headless JVM, sends real events (button clicks, Enter, mouse clicks, combo-box selections) and paints the real Swing component tree into a PNG with Swing's own painting code. A small test-only shim (Appendix B) stands in for the native window.	The layout, the components, the text and the event behaviour are the real ones. The operating-system window frame, fonts and look-and-feel may differ from a real desktop, and "minimise / restore" is imitated by asking the window to paint itself again.
AWT window events (windowOpened, windowActivated ...)	Documented sequence (Task 2.4). The program WindowTrace.java is provided to print the real sequence.	Not executed here because a real window is needed.
MySQL / JDBC (Tasks 1, 2, 6)	JDBC classes are written and compile-checked. The application runs here with an in-memory DAO. The SQL script was checked with SQLite as a stand-in (Task 6).	The SQL logic (constraints, joins, paging) is verified. The MySQL driver, server and error texts are not.

For the final submission: run every program on your own computer (the assignment requires individual execution), retake the screenshots from the real windows, run the MySQL script and the JDBC version of Task 6, and replace the placeholders. The programs, tests and analysis do not need to change.

Task 1 – GUI Design Review: Diagnose and Repair a Poor Event-Handling Design

Bloom's level K4 – Analyse. 6 marks. The junior developer's fee-payment window handles every button in one listener and keeps all program logic inside it. The original code (as given) is reproduced below.

Task01/FeeWindowBad.java.txt - original code from the assignment

```
// ORIGINAL code given in the assignment (kept only for reference - it does not even compile:
// the string concatenation in the INSERT statement is missing a + sign).
public class FeeWindow extends Frame implements ActionListener {
    TextField regNo = new TextField();
    TextField amount = new TextField();
    Button pay = new Button("Pay"), clear = new Button("Clear"), receipt = new Button("Receipt");

    public void actionPerformed(ActionEvent e) {
        if (e.getActionCommand() == "Pay") {
            double a = Double.parseDouble(amount.getText());
            Connection c = DriverManager.getConnection(URL, USER, PWD);
            c.createStatement().executeUpdate("INSERT INTO fees VALUES('" + regNo.getText() "'," +
a + ")");
            Thread.sleep(3000);              // wait for bank confirmation
            System.out.println("Paid");
        } else {
            regNo.setText("");
            amount.setText("");
        }
    }
}
```

Task 1.1 – Diagnose the design (ten deficiencies)

#	Deficiency (where)	What the user experiences / what can go wrong
D1	Interface and logic are mixed. actionPerformed() contains input parsing, JDBC, the bank wait and the console message.	Nothing can be tested without opening a window and a real database. Any change (a new rule, another database) means editing the same method, and one mistake breaks every button.
D2	The action command is compared with == (e.getActionCommand() == "Pay").	== compares references, not text. It works only while both strings are the same interned literal. If the label comes from a resource file, a translation or new String(...), the test is false and clicking Pay does nothing. (Shown in the test output below.)
D3	Everything that is not "Pay" goes to the else branch.	Clicking Receipt runs the Clear code: the fields are wiped and nothing is shown. The Receipt button is not implemented at all, and the user loses what was typed.
D4	No validation of the input. Double.parseDouble(amount.getText()) is called on raw text.	An empty or non-numeric amount throws NumberFormatException on the event thread: a stack trace on the console, and the window shows nothing. Negative, zero or absurd amounts and any register number are accepted and stored.
D5	Database work and Thread.sleep(3000) run on the event-dispatch thread.	The window freezes for the whole payment: it cannot repaint, the button stays half-pressed, the window cannot be moved or closed ("Not responding"). A slow database makes it worse.
D6	SQL built by string concatenation ("... VALUES('" + regNo.getText() + ...).	SQL injection: a register number such as x', 0); DROP TABLE fees; -- changes the statement. Even honest text containing an apostrophe breaks it. The column list is also missing, so any change of the table breaks the statement.
D7	The Connection (and Statement) are never closed.	Every payment leaks a connection. After enough payments the database refuses new connections and the whole college stops being able to pay.
D8	Checked exceptions (SQLException, InterruptedException) are not handled.	As written the code does not compile; when "fixed" with a blanket throws or an empty catch, errors disappear silently and the user does not know whether the money was taken.

#	Deficiency (where)	What the user experiences / what can go wrong
D9	Feedback is System.out.println("Paid") only.	The receptionist looking at the window sees no confirmation and no error. "Paid" is printed even though the "bank confirmation" is only a sleep.
D10	Closing is not handled (no WindowListener); a syntax error (missing +) is present.	An AWT Frame ignores the close button, so the window cannot be closed normally and a connection that is still open is never released.

Table 1.1 – Diagnosis (D1–D10). The design deficiencies are D1–D9; D10 includes the syntax error that stops the original from compiling.

Task 1.2 – Repair the design

Class structure. The window now only collects input and shows results. A separate FeeService validates the input and performs the payment. The service does not know Swing, and it does not know JDBC: it talks to two interfaces (PaymentRepository and BankGateway) so that the database and the bank can be replaced.

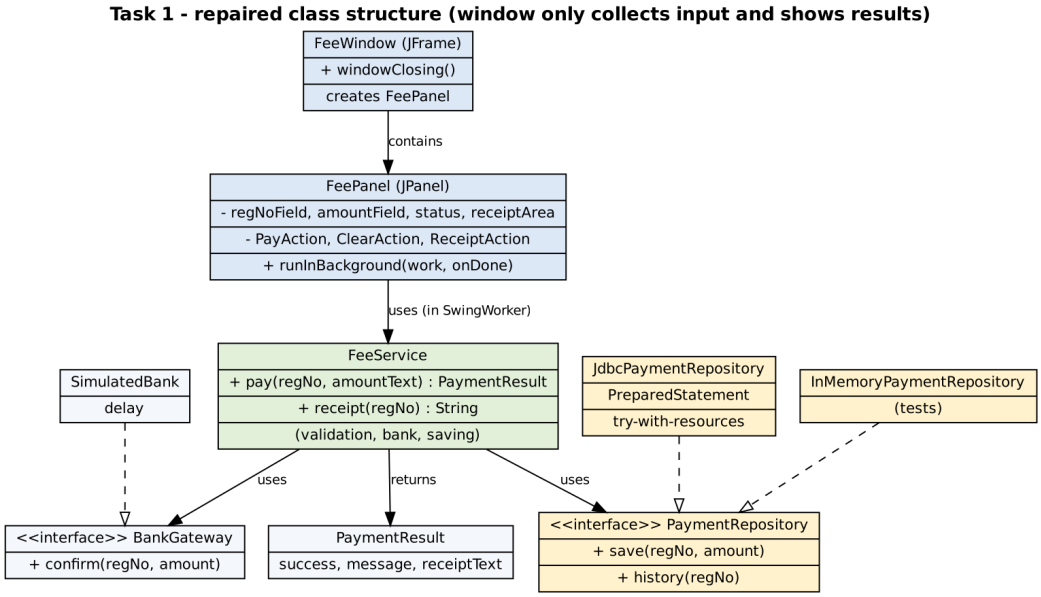


Figure 1.1 – Class structure of the repaired design

Class	Responsibility	Repairs
FeeWindow (JFrame)	Creates the panel; handles window closing (asks if a payment is running).	D10
FeePanel (JPanel)	Text fields, three buttons (one Action each), status line, receipt area. Starts SwingWorkers.	D1, D2, D3, D5, D9
FeeService	Validation, bank confirmation, saving, receipt text. Returns a PaymentResult; never throws to the window.	D1, D4, D8
PaymentRepository / JdbcPaymentRepository / InMemoryPaymentRepository	Data access. JDBC version uses PreparedStatement and try-with-resources.	D6, D7
BankGateway / SimulatedBank	The slow bank call, replacing Thread.sleep in the listener.	D5

Event handling – before and after.

```
// BEFORE: one listener, string compare with ==, everything inside, Receipt = Clear
public void actionPerformed(ActionEvent e) {
    if (e.getActionCommand() == "Pay") { ... JDBC ... Thread.sleep(3000); ... } else { clear
fields }
}

// AFTER: one small Action per button, no if-else chain, slow work in a SwingWorker
buttons.add(new JButton(payAction));        // PayAction     -> service.pay(...)      in the
background
buttons.add(new JButton(clearAction));      // ClearAction   -> clears fields only
```

```
buttons.add(new JButton(receiptAction));    // ReceiptAction -> service.receipt(...)  in the
background
```

Complete source code of the repaired design

Task01/PaymentResult.java

```
/** What the service tells the window after a payment attempt. The window only displays it. */
public class PaymentResult {
    public final boolean success;
    public final String message;
    public final String receiptText;      // empty when the payment failed

    private PaymentResult(boolean success, String message, String receiptText) {
        this.success = success;
        this.message = message;
        this.receiptText = receiptText;
    }
    public static PaymentResult ok(String message, String receiptText) { return new
PaymentResult(true, message, receiptText); }
    public static PaymentResult fail(String message) { return new PaymentResult(false, message,
""); }
}
```

Task01/PaymentRepository.java

```
import java.sql.SQLException;
import java.util.List;

/** Data access for fee payments. The service depends on this interface, not on JDBC. */
public interface PaymentRepository {
    /** Stores a payment and returns its payment id. */
    long save(String regNo, double amount) throws SQLException;
    /** Returns one text line per payment made by the student. */
    List<String> history(String regNo) throws SQLException;
}
```

Task01/JdbcPaymentRepository.java - PreparedStatement, try-with-resources

```
import java.sql.*;
import java.util.ArrayList;
import java.util.List;

/** JDBC implementation. Uses PreparedStatement (no SQL injection) and try-with-resources
(connection always closed). */
public class JdbcPaymentRepository implements PaymentRepository {
    private final String url, user, password;

    public JdbcPaymentRepository(String url, String user, String password) {
        this.url = url; this.user = user; this.password = password;
    }

    @Override
    public long save(String regNo, double amount) throws SQLException {
        String sql = "INSERT INTO fees(reg_no, amount) VALUES (?, ?)";
        try (Connection c = DriverManager.getConnection(url, user, password);
             PreparedStatement ps = c.prepareStatement(sql, Statement.RETURN_GENERATED_KEYS)) {
            ps.setString(1, regNo);                 // the value is sent as DATA, never as SQL
text
            ps.setDouble(2, amount);
            ps.executeUpdate();
            try (ResultSet keys = ps.getGeneratedKeys()) {
                return keys.next() ? keys.getLong(1) : -1;
            }
        }
    }
```

```
    @Override
    public List<String> history(String regNo) throws SQLException {
        String sql = "SELECT payment_id, amount, paid_on FROM fees WHERE reg_no = ? ORDER BY
payment_id";
        List<String> lines = new ArrayList<>();
        try (Connection c = DriverManager.getConnection(url, user, password);
             PreparedStatement ps = c.prepareStatement(sql)) {
            ps.setString(1, regNo);
            try (ResultSet rs = ps.executeQuery()) {
                while (rs.next()) {
                    lines.add(String.format("RCPT-%05d  Rs.%.2f  %s", rs.getLong(1),
rs.getDouble(2), rs.getTimestamp(3)));
                }
            }
        }
        return lines;
    }
}
```

Task01/InMemoryPaymentRepository.java

```
import java.util.ArrayList;
import java.util.List;

/** Used for unit tests and for running the window without a database. */
public class InMemoryPaymentRepository implements PaymentRepository {
    private final List<String[]> rows = new ArrayList<>();     // {id, regNo, amount}
    private long nextId = 1;

    @Override
    public synchronized long save(String regNo, double amount) {
        rows.add(new String[]{String.valueOf(nextId), regNo, String.format("%.2f", amount)});
        return nextId++;
    }

    @Override
    public synchronized List<String> history(String regNo) {
        List<String> lines = new ArrayList<>();
        for (String[] r : rows) {
            if (r[1].equals(regNo)) lines.add(String.format("RCPT-%05d  Rs.%s",
Long.parseLong(r[0]), r[2]));
        }
        return lines;
    }

    public synchronized int count() { return rows.size(); }
    public synchronized String lastRegNoStoredLiterally() { return rows.isEmpty() ? null :
rows.get(rows.size() - 1)[1]; }
}
```

Task01/BankGateway.java

```
/** The (slow) bank confirmation, hidden behind an interface so tests can use an instant one. */
public interface BankGateway {
    boolean confirm(String regNo, double amount) throws InterruptedException;
}
```

Task01/SimulatedBank.java

```
/** Pretends to call the bank. The delay replaces the junior developer's Thread.sleep(3000). */
public class SimulatedBank implements BankGateway {
    private final long delayMillis;
    public SimulatedBank(long delayMillis) { this.delayMillis = delayMillis; }

    @Override
    public boolean confirm(String regNo, double amount) throws InterruptedException {
        if (delayMillis > 0) Thread.sleep(delayMillis);
        return true;
```

```
    }
}
```

Task01/FeeService.java - validation + payment

```
import java.sql.SQLException;
import java.util.List;

/**
 * All the business rules of the fee window: validation, bank confirmation, saving, receipt text.
 * It has NO Swing / AWT code, so it can be tested with plain Java.
 */
public class FeeService {
    public static final double MIN_AMOUNT = 1.0;
    public static final double MAX_AMOUNT = 200000.0;

    private final PaymentRepository repository;
    private final BankGateway bank;

    public FeeService(PaymentRepository repository, BankGateway bank) {
        this.repository = repository;
        this.bank = bank;
    }

    /** Validates the two text values, asks the bank, saves the payment. Never throws: errors come
back as a result. */
    public PaymentResult pay(String regNoText, String amountText) {
        String regNo = regNoText == null ? "" : regNoText.trim();
        String amt = amountText == null ? "" : amountText.trim();

        if (regNo.isEmpty())              return PaymentResult.fail("Register number is
required.");
        if (!regNo.matches("\\d{9}"))     return PaymentResult.fail("Register number must be 9
digits (example 192311001).");
        if (amt.isEmpty())                return PaymentResult.fail("Amount is required.");

        double amount;
        try {
            amount = Double.parseDouble(amt);
        } catch (NumberFormatException e) {
            return PaymentResult.fail("Amount must be a number.");
        }
        if (Double.isNaN(amount) || Double.isInfinite(amount) || amount < MIN_AMOUNT || amount >
MAX_AMOUNT) {
            return PaymentResult.fail("Amount must be between " + (int) MIN_AMOUNT + " and " +
(int) MAX_AMOUNT + ".");
        }

        try {
            if (!bank.confirm(regNo, amount)) return PaymentResult.fail("The bank did not confirm
the payment.");
            long id = repository.save(regNo, amount);
            String message = String.format("Payment of Rs.%.2f received. Receipt RCPT-%05d.",
amount, id);
            return PaymentResult.ok(message, receipt(regNo));
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            return PaymentResult.fail("Payment was interrupted.");
        } catch (SQLException e) {
            return PaymentResult.fail("Database error: " + e.getMessage());
        }
    }

    /** Text of all receipts of a student (used by the Receipt button). */
    public String receipt(String regNoText) {
        String regNo = regNoText == null ? "" : regNoText.trim();
```

```
        if (!regNo.matches("\\d{9}")) return "Enter a valid register number to see receipts.";
        try {
            List<String> lines = repository.history(regNo);
            if (lines.isEmpty()) return "No payments found for " + regNo + ".";
            StringBuilder sb = new StringBuilder("Receipts for " + regNo + "\n");
            for (String line : lines) sb.append(line).append('\n');
            return sb.toString().trim();
        } catch (SQLException e) {
            return "Database error: " + e.getMessage();
        }
    }
}
```

Task01/FeePanel.java - the rewritten event handling

```
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.KeyEvent;
import java.util.concurrent.Callable;
import java.util.concurrent.ExecutionException;
import java.util.function.Consumer;
import javax.swing.*;

/**
 * The view: it only collects input and shows results. Each button has its OWN Action object
 * (no if-else chain on the action command). All slow work is done by SwingWorker, off the event-
dispatch thread.
 */
public class FeePanel extends JPanel {
    private final FeeService service;
    private final JTextField regNoField = new JTextField(14);
    private final JTextField amountField = new JTextField(14);
    private final JLabel status = new JLabel(" ");
    private final JTextArea receiptArea = new JTextArea(6, 32);
    private final Action payAction = new PayAction();
    private final Action clearAction = new ClearAction();
    private final Action receiptAction = new ReceiptAction();
    private volatile boolean busy = false;

    public FeePanel(FeeService service) {
        this.service = service;
        setLayout(new BorderLayout(8, 8));
        setBorder(BorderFactory.createEmptyBorder(10, 12, 10, 12));

        JPanel form = new JPanel(new GridLayout(2, 2, 6, 6));
        form.add(new JLabel("Register number:")); form.add(regNoField);
        form.add(new JLabel("Amount (Rs.):"));    form.add(amountField);

        JPanel buttons = new JPanel(new FlowLayout(FlowLayout.CENTER, 8, 4));
        buttons.add(new JButton(payAction));
        buttons.add(new JButton(clearAction));
        buttons.add(new JButton(receiptAction));

        JPanel north = new JPanel(new BorderLayout(4, 4));
        north.add(form, BorderLayout.CENTER);
        north.add(buttons, BorderLayout.SOUTH);

        receiptArea.setEditable(false);
        add(north, BorderLayout.NORTH);
        add(new JScrollPane(receiptArea), BorderLayout.CENTER);
        add(status, BorderLayout.SOUTH);
    }

    public boolean isBusy() { return busy; }
    public String statusText() { return status.getText(); }
    public String receiptText() { return receiptArea.getText(); }
```

```
    public JTextField regNoField() { return regNoField; }
    public JTextField amountField() { return amountField; }
    public Action payAction() { return payAction; }
    public Action clearAction() { return clearAction; }
    public Action receiptAction() { return receiptAction; }

    private void showStatus(String text, boolean error) {
        status.setForeground(error ? new Color(0xB00020) : new Color(0x1B5E20));
        status.setText(text);
    }

    private void setBusy(boolean b) {
        busy = b;
        payAction.setEnabled(!b);
        receiptAction.setEnabled(!b);
    }

    /** Runs slow work in a worker thread and delivers the result on the event-dispatch thread. */
    private <T> void runInBackground(Callable<T> work, Consumer<T> onDone) {
        setBusy(true);
        new SwingWorker<T, Void>() {
            @Override protected T doInBackground() throws Exception { return work.call(); }
            @Override protected void done() {
                try {
                    onDone.accept(get());
                } catch (InterruptedException | ExecutionException ex) {
                    showStatus("Unexpected error: " + ex.getMessage(), true);
                } finally {
                    setBusy(false);
                }
            }
        }.execute();
    }

    private class PayAction extends AbstractAction {
        PayAction() { super("Pay"); putValue(MNEMONIC_KEY, KeyEvent.VK_P); }
        @Override public void actionPerformed(ActionEvent e) {
            final String reg = regNoField.getText(), amt = amountField.getText();
            showStatus("Contacting the bank ...", false);
            runInBackground(() -> service.pay(reg, amt), result -> {
                showStatus(result.message, !result.success);
                if (result.success) receiptArea.setText(result.receiptText);
            });
        }
    }

    private class ClearAction extends AbstractAction {
        ClearAction() { super("Clear"); putValue(MNEMONIC_KEY, KeyEvent.VK_C); }
        @Override public void actionPerformed(ActionEvent e) {
            regNoField.setText(""); amountField.setText(""); receiptArea.setText(""); showStatus("
", false);
            regNoField.requestFocusInWindow();
        }
    }

    private class ReceiptAction extends AbstractAction {
        ReceiptAction() { super("Receipt"); putValue(MNEMONIC_KEY, KeyEvent.VK_R); }
        @Override public void actionPerformed(ActionEvent e) {
            final String reg = regNoField.getText();
            runInBackground(() -> service.receipt(reg), text -> { receiptArea.setText(text);
showStatus("Receipt list updated.", false); });
        }
    }
}
```

Task01/FeeWindow.java

```
import java.awt.event.WindowAdapter;
import java.awt.event.WindowEvent;
import javax.swing.*;

/** The window: only creates the panel and handles closing. */
public class FeeWindow extends JFrame {
    private final FeePanel panel;

    public FeeWindow(FeeService service) {
        super("Student Fee Payment");
        panel = new FeePanel(service);
        add(panel);
        setDefaultCloseOperation(DO_NOTHING_ON_CLOSE);           // we decide in windowClosing
        addWindowListener(new WindowAdapter() {
            @Override
            public void windowClosing(WindowEvent e) {
                if (panel.isBusy()) {
                    int answer = JOptionPane.showConfirmDialog(FeeWindow.this,
                            "A payment is still in progress. Close anyway?", "Payment in
progress", JOptionPane.YES_NO_OPTION);
                    if (answer != JOptionPane.YES_OPTION) return;
                }
                dispose();                                        // no connection is kept open,
nothing else to close
            }
        });
        pack();
        setLocationRelativeTo(null);
    }

    public static void main(String[] args) {
        FeeService service;
        if (Boolean.getBoolean("fee.demo")) {                     // java -Dfee.demo=true
FeeWindow   (no database needed)
            service = new FeeService(new InMemoryPaymentRepository(), new SimulatedBank(1500));
        } else {
            String url = System.getProperty("fee.url", "jdbc:mysql://localhost:3306/college");
            String user = System.getProperty("fee.user", "root");
            String pwd = System.getProperty("fee.pwd", "");
            service = new FeeService(new JdbcPaymentRepository(url, user, pwd), new
SimulatedBank(1500));
        }
        SwingUtilities.invokeLater(() -> new FeeWindow(service).setVisible(true));
    }
}
```

Task01/FeeServiceTest.java - automatic tests

```
/** Plain-Java tests of the service (no window needed). Run: java FeeServiceTest */
public class FeeServiceTest {
    static int passed = 0, failed = 0;

    static void check(String name, PaymentResult r, boolean expectSuccess, String expectedPart) {
        boolean ok = r.success == expectSuccess && r.message.contains(expectedPart);
        if (ok) passed++; else failed++;
        System.out.printf("%-4s %-38s success=%-5s message=%s%n", ok ? "PASS" : "FAIL", name,
r.success, r.message);
    }

    public static void main(String[] args) {
        InMemoryPaymentRepository repo = new InMemoryPaymentRepository();
        FeeService service = new FeeService(repo, new SimulatedBank(0));

        System.out.println("=== FeeService tests ===");
```

```
        check("valid payment",                 service.pay("192311001", "15000"),  true,
"received");
        check("valid payment with spaces",     service.pay("  192311002 ", " 2500.50 "), true,
"received");
        check("empty register number",         service.pay("", "100"),             false,
"required");
        check("short register number",         service.pay("1923", "100"),         false, "9
digits");
        check("empty amount",                  service.pay("192311001", ""),       false, "Amount
is required");
        check("amount is text",                service.pay("192311001", "abc"),    false, "must be
a number");
        check("negative amount",               service.pay("192311001", "-50"),    false,
"between");
        check("zero amount",                   service.pay("192311001", "0"),      false,
"between");
        check("amount too large",              service.pay("192311001", "999999"), false,
"between");
        check("NaN amount",                    service.pay("192311001", "NaN"),    false,
"between");
        check("SQL injection in reg number",   service.pay("1'); DROP TABLE fees;--", "100"),
false, "9 digits");
        check("null inputs",                   service.pay(null, null),            false,
"required");

        System.out.println();
        System.out.println("Payments stored after all tests : " + repo.count() + "  (expected 2 -
invalid input never reaches the database)");
        System.out.println("Receipt text for 192311001:");
        System.out.println(service.receipt("192311001"));
        System.out.println("Receipt text for 192311009 (no payments):");
        System.out.println(service.receipt("192311009"));
        System.out.println();
        System.out.println("Result: " + passed + " passed, " + failed + " failed");

        // why the original  e.getActionCommand() == "Pay"  is unreliable
        String fromResource = new String("Pay");
        System.out.println();
        System.out.println("\"Pay\" == \"Pay\" (same literal)          : " + ("Pay" == "Pay"));
        System.out.println("new String(\"Pay\") == \"Pay\"             : " + (fromResource ==
"Pay") + "   <- == compares references");
        System.out.println("new String(\"Pay\").equals(\"Pay\")        : " +
fromResource.equals("Pay"));

        // how the original built its SQL
        String evil = "x', 0); DROP TABLE fees; --";
        System.out.println();
        System.out.println("Original code builds : INSERT INTO fees VALUES('" + evil + "'," +
100.0 + ")");
        System.out.println("Repaired code sends  : INSERT INTO fees(reg_no, amount) VALUES (?, ?)
with ? = text \"" + evil + "\" as data only");
    }
}
```

Task01/schema.sql - MySQL tables

```
-- Task 1 : fee payment table (MySQL)
CREATE DATABASE IF NOT EXISTS college;
USE college;

CREATE TABLE IF NOT EXISTS student (
    reg_no  VARCHAR(12) PRIMARY KEY,
    name    VARCHAR(60) NOT NULL
);

CREATE TABLE IF NOT EXISTS fees (
```

```
    payment_id INT AUTO_INCREMENT PRIMARY KEY,
    reg_no     VARCHAR(12)   NOT NULL,
    amount     DECIMAL(10,2) NOT NULL CHECK (amount > 0),
    paid_on    TIMESTAMP     NOT NULL DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (reg_no) REFERENCES student(reg_no)
);
```

Database design (ER diagram)

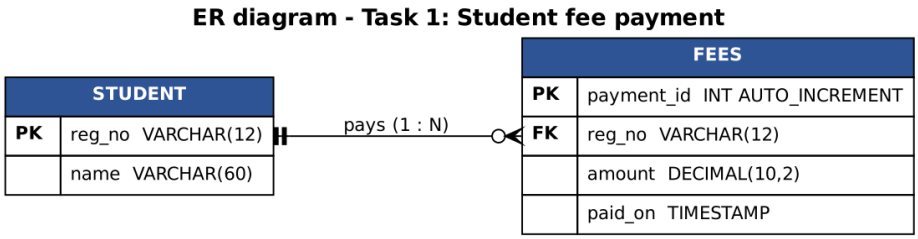


Figure 1.2 – ER diagram of the fee database (one student pays many times)

Evidence 1 – automatic tests of FeeService (each test is linked to the deficiency it repairs)

Output of: java FeeServiceTest

```
=== FeeService tests ===
PASS valid payment                          success=true  message=Payment of Rs.15000.00 received.
Receipt RCPT-00001.
PASS valid payment with spaces              success=true  message=Payment of Rs.2500.50 received. Receipt
RCPT-00002.
PASS empty register number                  success=false message=Register number is required.
PASS short register number                  success=false message=Register number must be 9 digits
(example 192311001).
PASS empty amount                           success=false message=Amount is required.
PASS amount is text                         success=false message=Amount must be a number.
PASS negative amount                        success=false message=Amount must be between 1 and 200000.
PASS zero amount                            success=false message=Amount must be between 1 and 200000.
PASS amount too large                       success=false message=Amount must be between 1 and 200000.
PASS NaN amount                             success=false message=Amount must be between 1 and 200000.
PASS SQL injection in reg number            success=false message=Register number must be 9 digits
(example 192311001).
PASS null inputs                            success=false message=Register number is required.

Payments stored after all tests : 2  (expected 2 - invalid input never reaches the database)
Receipt text for 192311001:
Receipts for 192311001
RCPT-00001  Rs.15000.00
Receipt text for 192311009 (no payments):
No payments found for 192311009.

Result: 12 passed, 0 failed

"Pay" == "Pay" (same literal)          : true
new String("Pay") == "Pay"             : false   <- == compares references
new String("Pay").equals("Pay")        : true

Original code builds : INSERT INTO fees VALUES('x', 0); DROP TABLE fees; --',100.0)
Repaired code sends  : INSERT INTO fees(reg_no, amount) VALUES (?, ?)   with ? = text "x', 0); DROP TABLE
fees; --" as data only
```

Test (output line)	Deficiency repaired	What it shows
empty register number, short register number, null inputs	D4	Invalid input is rejected with a clear message and never stored.
amount is text / negative / zero / too large / NaN	D4	Amount rules are enforced; no NumberFormatException reaches the user.
SQL injection in reg number	D6	Rejected by validation; and the repository stores values as data (PreparedStatement).

Test (output line)	Deficiency repaired	What it shows
"Payments stored after all tests: 2"	D4, D6	Only the 2 valid payments reached the repository.
"Pay" == "Pay" vs new String("Pay") == "Pay"	D2	Reference comparison is false for an equal string; equals() is true.
Original builds … / Repaired code sends …	D6	The original statement is rewritten by the input; the repaired statement keeps the input as data only.

Evidence 2 – the repaired window, driven by real events (off-screen render)

Output of the harness FeeHarness (Appendix B)

```
[1] window opened: status=" "
[2] Pay with empty fields -> status: Register number is required.
[3] Pay with amount 'abc'   -> status: Amount must be a number.
[4] Pay 15000 (bank takes 1.5 s): finished after 1624 ms; status: Payment of Rs.15000.00 received.
Receipt RCPT-00001.
    event-dispatch thread stayed responsive: 32 pings during the payment, slowest answer 1.28 ms
[5] Clear, then Receipt for 192311001 -> receipt area:
    Receipts for 192311001
    RCPT-00001  Rs.15000.00
```

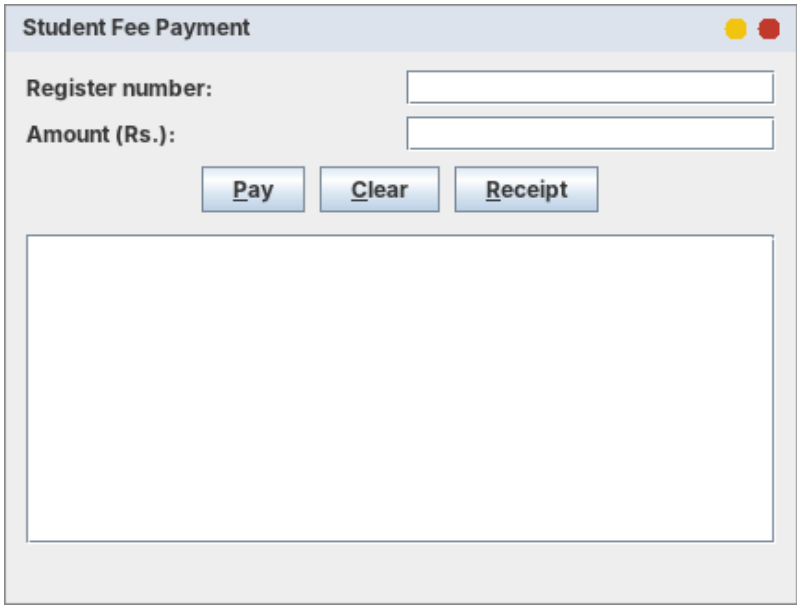


Figure 1.3 – Initial window

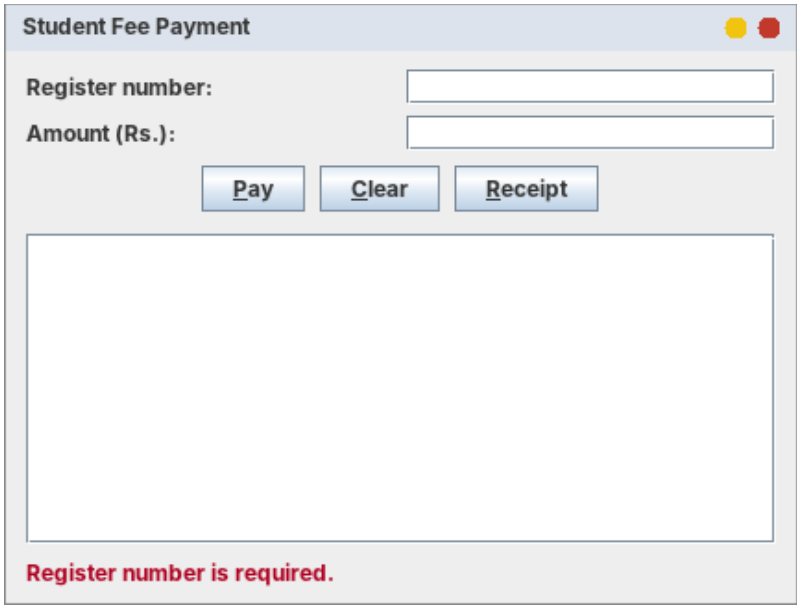


Figure 1.4 – Demonstrates D4/D9: Pay with empty fields shows a clear message

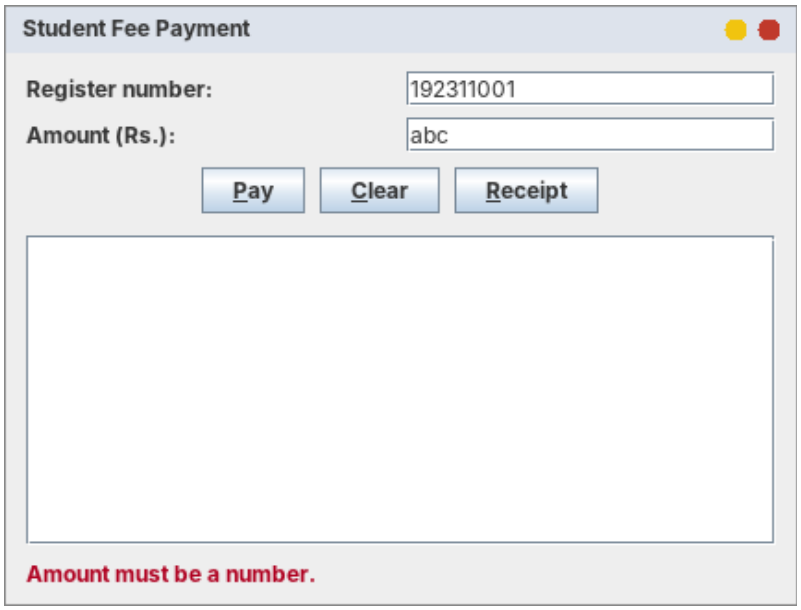


Figure 1.5 – Demonstrates D4: amount "abc" is rejected without an exception

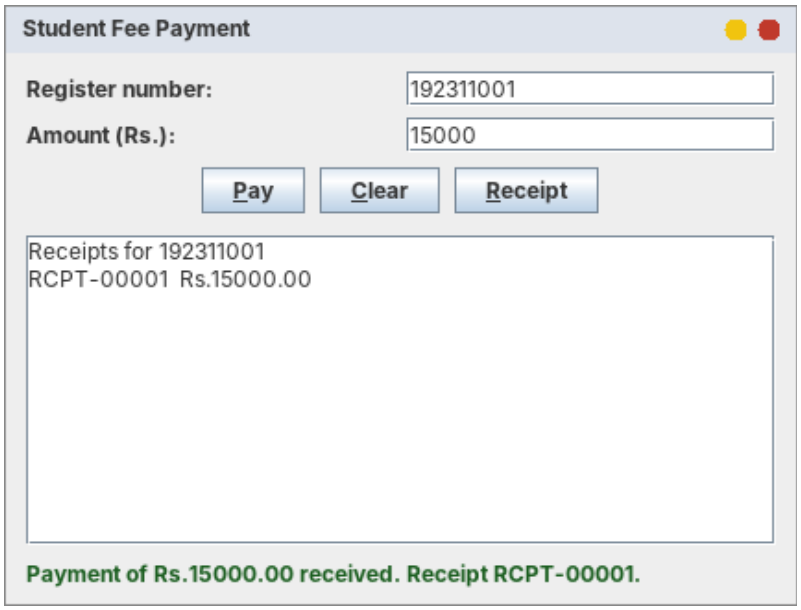


Figure 1.6 – Demonstrates D5/D9: payment accepted after the 1.5 s bank call; confirmation and receipt shown

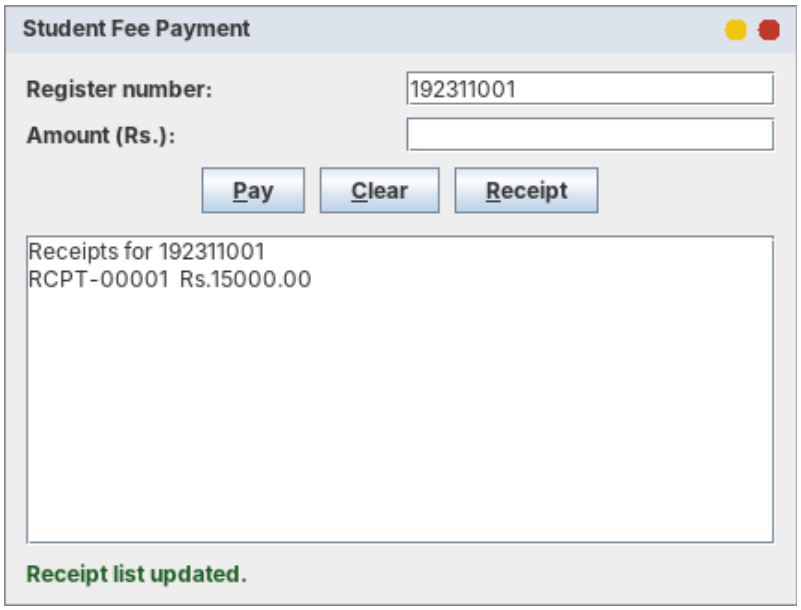


Figure 1.7 – Demonstrates D3: the Receipt button shows the receipts instead of clearing the form

D5 evidence: the bank call took 1.5 s. During that time the program asked the event-dispatch thread to do a tiny task 32 times, and the slowest answer took 1.28 ms (see the output above). With the original code the same request would have waited the whole 3 seconds.

Task 1.3 – Design defence (87 words)

The window now only reads text fields and displays a PaymentResult, so it holds no business rules. FeeService has no Swing code, so twelve automatic tests (valid, empty, text, negative, injection) run in milliseconds without a window or database, using an in-memory repository. Database access sits behind an interface, so MySQL can be replaced without touching the service or the window. Each button has its own small Action, so changing Pay cannot break Clear or Receipt. Slow work runs in a SwingWorker, so the window stays responsive.

Alternative concept and complexity analysis

The alternative solution. Instead of one listener with an if–else chain on the action command, every button gets its own listener object (or a Swing Action). This is what the repaired FeePanel already does. The two solutions in code:

```
// one listener, if-else chain on the command (k comparisons in the worst case)
public void actionPerformed(ActionEvent e) {
    String cmd = e.getActionCommand();
    if (cmd.equals("Pay")) { ... } else if (cmd.equals("Clear")) { ... } else if
(cmd.equals("Receipt")) { ... }  // ... k branches
}

// one listener (Action) per button: the event goes straight to the right object
payButton.addActionListener(e -> pay());          // or  new JButton(payAction)
clearButton.addActionListener(e -> clear());
receiptButton.addActionListener(e -> receipt());
```

	One listener + if-else chain	One listener / Action per button
Time to dispatch one event (k buttons)	O(k) comparisons in the worst case (O(1) for the first branch)	O(1): Swing calls the listener registered on the button that fired
Space	One listener object + k command strings	k small listener objects (about 20 bytes each) + 0 command strings
Adding a button	Edit the shared method (risk to the others)	Add one registration; nothing else changes

Measurements (DispatchBenchmark.java; time for the LAST button, the worst case; average of 5 runs after warm-up):

k buttons	if-else chain (ns/event)	HashMap lookup (ns/event)	own listener (ns/event)	bytes: k listeners
3	33.2	113.0	0.1	872
10	81.7	16.4	0.1	1,008
100	402.5	13.5	0.1	2,808
1,000	3022.3	8.9	0.1	20,808
10,000	37026.9	15.6	0.2	200,808

Table 1.2 – Dispatch time and listener memory (from Task01/dispatch_results.csv)

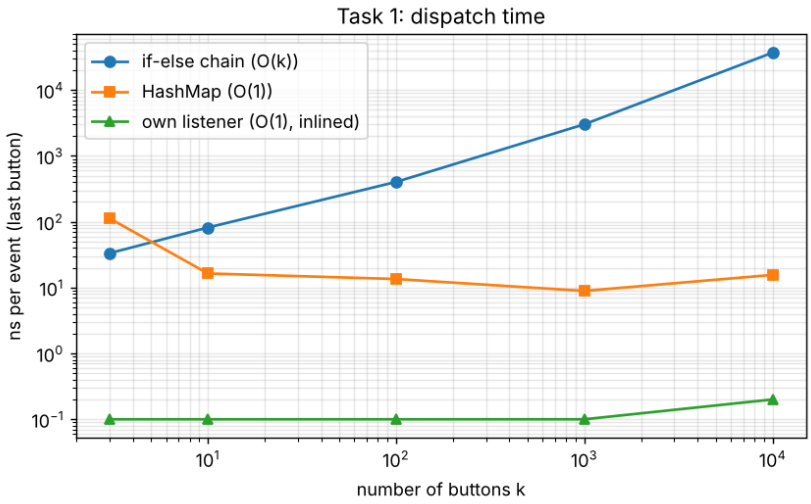


Figure 1.8 – Dispatch time against the number of buttons k (logarithmic axes)

Analysis. The chain grows with k: from 33.2 ns (k = 3) to 37,027 ns (k = 10,000), i.e. about 1,100 times for 3,333 times more buttons, which is the linear growth O(k). The HashMap and the own-listener approaches stay flat (O(1)); the first HashMap value (113.0 ns) is warm-up noise, and the own-listener time of about 0.1 ns means the JIT compiler inlined the call completely. The space for k separate listeners is about 20 bytes each (a 16-byte object and a 4-byte array slot), i.e. 200,808 bytes for k = 10,000. **Conclusion:** the O(k) against O(1) difference is real, but for a window with 3–20 buttons both costs are far below one microsecond and 1 KB. The decisive reasons for the redesign are testability and maintainability (D1, D2, D3), not the speed of dispatch.

Task 2 – From Requirement to Event-Driven Design: Hospital Outpatient Appointment Desk

Bloom's level K4 – Analyse. 8 marks. The receptionist enters a patient ID, selects a department, chooses a doctor (the doctors shown depend on the department), picks a time slot and books. The window shows a confirmation or an error and asks for confirmation before closing if a booking is in progress.

Task 2.1 – Model the events

Event source	Event object generated	Listener interface	Action taken
Window (title-bar close button)	WindowEvent (WINDOW_CLOSING)	WindowListener (via WindowAdapter)	If a booking is in progress ask “Close anyway?”; if yes dispose(), otherwise stay open.
Window (when first shown)	WindowEvent (WINDOW_OPENED, WINDOW_ACTIVATED)	WindowListener	Not needed for the logic; used for the trace in Task 2.4.
Department JComboBox	ItemEvent (SELECTED, and DESELECTED for the old item)	ItemListener	Only for SELECTED: clear the doctor list and fill it with the doctors of the chosen department from the cached Map.
Patient-ID JTextField (typing)	DocumentEvent (insert / remove)	DocumentListener	The form becomes “in progress” (used when closing). (KeyEvent / KeyListener would also work but sees keys, not text changes.)
Book JButton (click, or Enter on the button)	ActionEvent	ActionListener	Validate and book in a SwingWorker; show the confirmation or the error; clear the form on success.
SwingWorker (booking finished)	done() callback on the event-dispatch thread	(SwingWorker)	Show the result, re-enable the button.

Table 2.1 – Event model

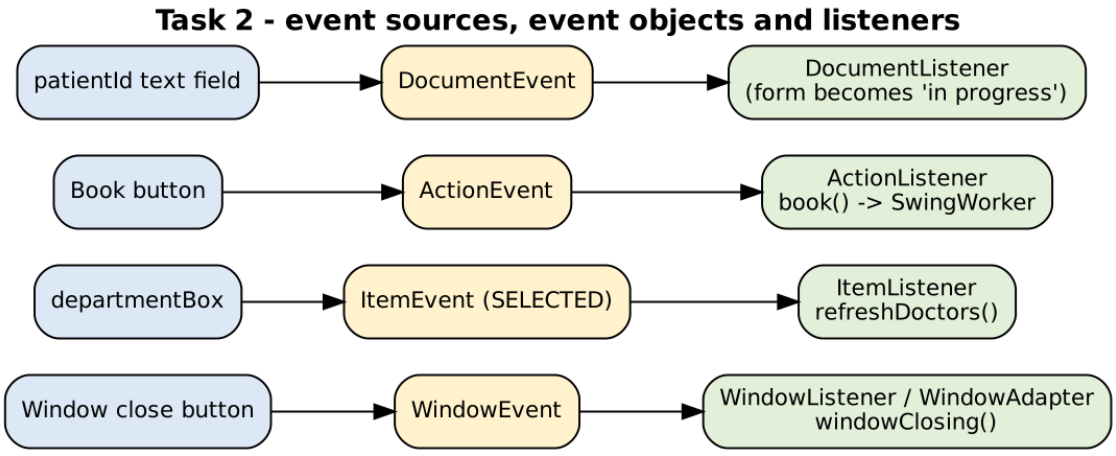


Figure 2.1 – Event sources, event objects and listeners

Task 2.2 – Component hierarchy

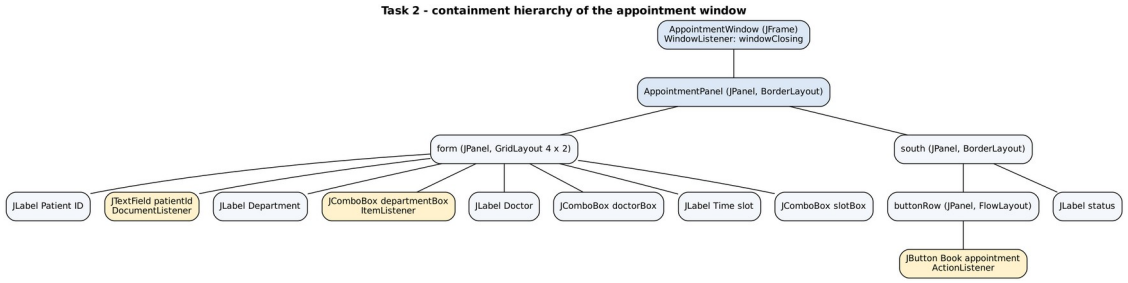


Figure 2.2 – Containment tree with the Swing class of every node

Node	Swing class	Layout manager
Top-level window	JFrame (AppointmentWindow)	BorderLayout (content pane)
Whole form	JPanel (AppointmentPanel)	BorderLayout: form in CENTER, south panel in SOUTH
Four label/field rows	JPanel “form”	GridLayout 4 × 2 (equal rows, label left, field right)
Patient ID / Department / Doctor / Time slot	JTextField, JComboBox<String>, JComboBox<Doctor>, JComboBox<String>	–
Button row + status	JPanel (FlowLayout, centred) with a JButton; JLabel	BorderLayout (button row NORTH, status SOUTH)

Task 2.3 – Handler logic (pseudocode)

The pseudocode below uses numbered steps with the keywords ALGORITHM, INPUT, OUTPUT, BEGIN, END, IF/THEN/ELSE/ENDIF, FOR EACH/ENDFOR, CALL, SET and RETURN. (The text of the “SIMATS Engineering – Standard for Algorithmic Pseudocode” was not available while writing; if it prescribes different keywords, only the keywords need to be changed.)

```
ALGORITHM RefreshDoctorList
INPUT   : selectedDepartment, doctorsByDepartment (Map from department name to list of Doctor)
OUTPUT  : the doctor combo box shows only the doctors of the selected department
BEGIN
  1.  CALL doctorBox.removeAllItems()
  2.  IF selectedDepartment is the prompt "-- select --" THEN
  3.      RETURN                                        // nothing to show
  4.  ENDIF
  5.  SET doctors <- doctorsByDepartment.get(selectedDepartment)     // empty list if the key is
missing
  6.  FOR EACH doctor IN doctors DO
  7.      CALL doctorBox.addItem(doctor)
  8.  ENDFOR
  9.  CALL statusLabel.setText("")
END
```

```
ALGORITHM ValidateAndBook
INPUT   : patientId, department, doctor, slot
OUTPUT  : a confirmation or an error message on the status label
BEGIN
  1.  SET id <- trim(patientId)
  2.  IF id is empty THEN  RETURN error "Enter the patient ID"                    ENDIF
  3.  IF id does not match the pattern P followed by 4 digits THEN  RETURN error "Patient ID must
look like P1001"  ENDIF
  4.  IF department is not selected THEN  RETURN error "Select a department"      ENDIF
  5.  IF doctor is not selected THEN  RETURN error "Select a doctor"              ENDIF
  6.  IF doctor does not belong to department THEN  RETURN error                  ENDIF
  7.  IF slot is not selected THEN  RETURN error "Select a time slot"             ENDIF
  8.  IF repository.isSlotTaken(doctor, slot) THEN  RETURN error "already booked" ENDIF
  9.  CALL repository.save(id, doctor, slot)
```

```
  10. CALL resetForm()                                  // so the window is no longer "in progress"
  11. RETURN confirmation "Appointment confirmed: id with doctor at slot"
END                                                    // steps 1-11 run in a SwingWorker, not on
the event-dispatch thread
```

Task 2.4 – Trace the event sequence

The sequence below is taken from the documented behaviour of the AWT/Swing window events (it requires a real window, so it was not executed on the server used for this report). The file Task02/WindowTrace.java prints the real order; run it, minimise, restore and close the window, and paste the output here to confirm.

Step	What happens	Methods called
1	main() starts; the window object is created on the event-dispatch thread	AppointmentWindow constructor → JFrame constructor → AppointmentPanel constructor (components created, listeners registered) → pack() → setLocationRelativeTo()
2	setVisible(true): the window appears	addNotify() (native peer) → paint() / paintComponent() → windowOpened() → windowActivated() → windowGainedFocus()
3	The user types the patient ID and selects a department	DocumentListener.insertUpdate() for each character; ItemListener.itemStateChanged() (DESELECTED, then SELECTED) → refreshDoctors()
4	The user minimises the window	windowIconified() → windowDeactivated() (windowStateChanged() on the state listener)
5	The user restores it	windowDeiconified() → windowActivated() → paint() (the whole window is painted again)
6	The user clicks Book	ActionListener.actionPerformed() → SwingWorker.execute() → doInBackground() (worker thread) → done() (event-dispatch thread)
7	The user clicks the close button	windowClosing() → confirmClose() → JOptionPane.showConfirmDialog() → (if yes) dispose() → windowDeactivated() → windowClosed()

Table 2.2 – Order of calls from open to close

Comparison with the life-cycle of an applet

Applet method	When it is called	Counterpart in the window application
init()	Once, when the applet is loaded	The constructor (create components, register listeners)
start()	When the applet becomes visible / the page is shown again	setVisible(true) → windowOpened(); windowDeiconified() / windowActivated() when restored
paint(Graphics)	Whenever the area must be drawn	paint() / paintComponent() – same idea, called by the system
stop()	When the page is left (the applet should pause)	windowIconified() / windowDeactivated()
destroy()	Once, when the applet is removed	windowClosing() → dispose() → windowClosed()

The applet life cycle is driven by the browser; a window application is driven by the window system through WindowEvents, and the program decides in windowClosing() whether the window really closes. (The Applet API is deprecated in current Java versions, which is another reason to use JFrame.)

Implementation

Task02/Doctor.java

```
/** A doctor shown in the doctor list. toString() is what the JComboBox displays. */
public record Doctor(String id, String name, String department) {
    @Override public String toString() { return name; }
}
```

Task02/AppointmentRepository.java

```
import java.util.List;
import java.util.Map;
```

```
public interface AppointmentRepository {
    /** One query for everything: used once at start-up (cache approach). */
    Map<String, List<Doctor>> loadAllDoctors();
    /** One query per call: used when the doctors are fetched on every department change. */
    List<Doctor> findDoctors(String department);
    List<String> slots();
    boolean isSlotTaken(String doctorId, String slot);
    void save(String patientId, Doctor doctor, String slot);
}
```

Task02/InMemoryAppointmentRepository.java

```
import java.util.*;

/** Stand-in for the database (used for tests / demo). latencyMillis imitates one network round
trip per query. */
public class InMemoryAppointmentRepository implements AppointmentRepository {
    private final List<Doctor> doctors = new ArrayList<>();
    private final List<String> slots = List.of("09:00", "09:30", "10:00", "10:30", "11:00",
"11:30", "12:00", "12:30");
    private final Set<String> booked = new HashSet<>();           // doctorId + "|" + slot
    private final long latencyMillis;

    public InMemoryAppointmentRepository() { this(0); seedDemoData(); }
    public InMemoryAppointmentRepository(long latencyMillis) { this.latencyMillis = latencyMillis;
}

    public void addDoctor(Doctor d) { doctors.add(d); }

    private void seedDemoData() {
        String[][] data = {
            {"Cardiology", "Dr. Anand Rao", "Dr. Meera Nair", "Dr. S. Kumar"},
            {"Orthopaedics", "Dr. Rajesh Iyer", "Dr. Latha Menon", "Dr. A. Fernandes"},
            {"Paediatrics", "Dr. Kavitha Rao", "Dr. Imran Khan", "Dr. Sneha Pillai"},
            {"Dermatology", "Dr. Priya Sharma", "Dr. V. Balaji", "Dr. Nisha George"}};
        int n = 1;
        for (String[] row : data)
            for (int i = 1; i < row.length; i++) doctors.add(new Doctor("D" + (n++), row[i],
row[0]));
    }

    private void pause() {
        if (latencyMillis > 0) try { Thread.sleep(latencyMillis); } catch (InterruptedException e)
{ Thread.currentThread().interrupt(); }
    }

    @Override public Map<String, List<Doctor>> loadAllDoctors() {
        pause();
        Map<String, List<Doctor>> map = new TreeMap<>();
        for (Doctor d : doctors) map.computeIfAbsent(d.department(), k -> new
ArrayList<>()).add(d);
        return map;
    }

    @Override public List<Doctor> findDoctors(String department) {
        pause();
        List<Doctor> result = new ArrayList<>();
        for (Doctor d : doctors) if (d.department().equals(department)) result.add(d);   // full
scan, like a query with no index
        return result;
    }

    @Override public List<String> slots() { return slots; }
    @Override public boolean isSlotTaken(String doctorId, String slot) { return
booked.contains(doctorId + "|" + slot); }
```

```
    @Override public void save(String patientId, Doctor doctor, String slot)
{ booked.add(doctor.id() + "|" + slot); }
}
```

Task02/JdbcAppointmentRepository.java - MySQL version

```
import java.sql.*;
import java.util.*;

/** MySQL version of the repository (compile-checked; run it with the MySQL Connector/J jar on the
class path). */
public class JdbcAppointmentRepository implements AppointmentRepository {
    private final String url, user, password;
    public JdbcAppointmentRepository(String url, String user, String password) { this.url = url;
this.user = user; this.password = password; }

    private Connection open() throws SQLException { return DriverManager.getConnection(url, user,
password); }

    @Override public Map<String, List<Doctor>> loadAllDoctors() {
        String sql = "SELECT d.doctor_id, d.name, p.name FROM doctor d JOIN department p ON
p.dept_id = d.dept_id ORDER BY p.name, d.name";
        Map<String, List<Doctor>> map = new TreeMap<>();
        try (Connection c = open(); Statement st = c.createStatement(); ResultSet rs =
st.executeQuery(sql)) {
            while (rs.next()) map.computeIfAbsent(rs.getString(3), k -> new ArrayList<>()).add(new
Doctor(rs.getString(1), rs.getString(2), rs.getString(3)));
        } catch (SQLException e) { throw new IllegalStateException("Cannot load doctors: " +
e.getMessage(), e); }
        return map;
    }

    @Override public List<Doctor> findDoctors(String department) {
        String sql = "SELECT d.doctor_id, d.name FROM doctor d JOIN department p ON p.dept_id =
d.dept_id WHERE p.name = ? ORDER BY d.name";
        List<Doctor> list = new ArrayList<>();
        try (Connection c = open(); PreparedStatement ps = c.prepareStatement(sql)) {
            ps.setString(1, department);
            try (ResultSet rs = ps.executeQuery()) { while (rs.next()) list.add(new
Doctor(rs.getString(1), rs.getString(2), department)); }
        } catch (SQLException e) { throw new IllegalStateException("Cannot load doctors: " +
e.getMessage(), e); }
        return list;
    }

    @Override public List<String> slots() { return List.of("09:00", "09:30", "10:00", "10:30",
"11:00", "11:30", "12:00", "12:30"); }

    @Override public boolean isSlotTaken(String doctorId, String slot) {
        String sql = "SELECT COUNT(*) FROM appointment WHERE doctor_id = ? AND appt_date =
CURDATE() AND slot_time = ?";
        try (Connection c = open(); PreparedStatement ps = c.prepareStatement(sql)) {
            ps.setString(1, doctorId); ps.setString(2, slot);
            try (ResultSet rs = ps.executeQuery()) { rs.next(); return rs.getInt(1) > 0; }
        } catch (SQLException e) { throw new IllegalStateException("Cannot check slot: " +
e.getMessage(), e); }
    }

    @Override public void save(String patientId, Doctor doctor, String slot) {
        String sql = "INSERT INTO appointment(patient_id, doctor_id, appt_date, slot_time) VALUES
(?, ?, CURDATE(), ?)";
        try (Connection c = open(); PreparedStatement ps = c.prepareStatement(sql)) {
            ps.setString(1, patientId); ps.setString(2, doctor.id()); ps.setString(3, slot);
            ps.executeUpdate();
        } catch (SQLException e) { throw new IllegalStateException("Cannot save appointment: " +
e.getMessage(), e); }
```

```
    }
}
```

Task02/BookingResult.java

```
public class BookingResult {
    public final boolean success;
    public final String message;
    private BookingResult(boolean s, String m) { success = s; message = m; }
    public static BookingResult ok(String m) { return new BookingResult(true, m); }
    public static BookingResult fail(String m) { return new BookingResult(false, m); }
}
```

Task02/AppointmentService.java - doctors cached once in a Map<String, List<Doctor>>

```
import java.util.*;

/** Rules of the appointment desk. The doctors of every department are loaded ONCE into a Map and
reused. */
public class AppointmentService {
    public static final String PROMPT = "-- select --";
    private final AppointmentRepository repository;
    private final Map<String, List<Doctor>> doctorsByDepartment;

    public AppointmentService(AppointmentRepository repository) {
        this.repository = repository;
        this.doctorsByDepartment = new TreeMap<>(repository.loadAllDoctors());     // one database
access
    }

    public List<String> departments() { return new ArrayList<>(doctorsByDepartment.keySet()); }
    public List<String> slots() { return repository.slots(); }

    /** O(1) lookup in the cached Map (no database access). */
    public List<Doctor> doctorsFor(String department) {
        return doctorsByDepartment.getOrDefault(department, Collections.emptyList());
    }

    public BookingResult book(String patientId, String department, Doctor doctor, String slot) {
        String id = patientId == null ? "" : patientId.trim();
        if (id.isEmpty())                                    return BookingResult.fail("Enter the
patient ID.");
        if (!id.matches("P\\d{4}"))                          return BookingResult.fail("Patient ID
must look like P1001.");
        if (department == null || department.equals(PROMPT)) return BookingResult.fail("Select a
department.");
        if (doctor == null)                                  return BookingResult.fail("Select a
doctor.");
        if (!doctor.department().equals(department))       return BookingResult.fail("The doctor
does not belong to " + department + ".");
        if (slot == null || slot.equals(PROMPT))             return BookingResult.fail("Select a
time slot.");
        if (repository.isSlotTaken(doctor.id(), slot))      return BookingResult.fail(doctor + "
is already booked at " + slot + ".");
        repository.save(id, doctor, slot);
        return BookingResult.ok("Appointment confirmed: " + id + " with " + doctor + " (" +
department + ") at " + slot + ".");
    }
}
```

Task02/AppointmentPanel.java - listeners for ItemEvent, ActionEvent, DocumentEvent

```
import java.awt.*;
import java.awt.event.ItemEvent;
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.ExecutionException;
import javax.swing.*;
```

```
import javax.swing.event.DocumentEvent;
import javax.swing.event.DocumentListener;

/** The appointment form (view + event handlers). All rules are in AppointmentService. */
public class AppointmentPanel extends JPanel {
    private final AppointmentService service;
    private final JTextField patientId = new JTextField(10);
    private final JComboBox<String> departmentBox = new JComboBox<>();
    private final JComboBox<Doctor> doctorBox = new JComboBox<>();
    private final JComboBox<String> slotBox = new JComboBox<>();
    private final JButton bookButton = new JButton("Book appointment");
    private final JLabel status = new JLabel(" ");
    private final List<String> trace = new ArrayList<>();         // one line per event handled
(used in the report)
    private volatile boolean busy = false;

    public AppointmentPanel(AppointmentService service) {
        this.service = service;
        setLayout(new BorderLayout(8, 8));
        setBorder(BorderFactory.createEmptyBorder(10, 12, 10, 12));

        departmentBox.addItem(AppointmentService.PROMPT);
        for (String d : service.departments()) departmentBox.addItem(d);
        slotBox.addItem(AppointmentService.PROMPT);
        for (String s : service.slots()) slotBox.addItem(s);

        JPanel form = new JPanel(new GridLayout(4, 2, 6, 6));
        form.add(new JLabel("Patient ID:"));  form.add(patientId);
        form.add(new JLabel("Department:"));  form.add(departmentBox);
        form.add(new JLabel("Doctor:"));      form.add(doctorBox);
        form.add(new JLabel("Time slot:"));   form.add(slotBox);
        JPanel south = new JPanel(new BorderLayout(4, 4));
        JPanel buttonRow = new JPanel(new FlowLayout(FlowLayout.CENTER));
        buttonRow.add(bookButton);
        south.add(buttonRow, BorderLayout.NORTH);
        south.add(status, BorderLayout.SOUTH);
        add(form, BorderLayout.CENTER);
        add(south, BorderLayout.SOUTH);

        // ---- event sources and their listeners ----
        departmentBox.addItemListener(e -> {                                    // ItemEvent  ->
ItemListener
            if (e.getStateChange() == ItemEvent.SELECTED) {                     // ignore the
DESELECTED event of the old item
                trace.add("ItemEvent(SELECTED) from departmentBox -> " + e.getItem());
                refreshDoctors();
            }
        });
        bookButton.addActionListener(e -> {                                     // ActionEvent ->
ActionListener
            trace.add("ActionEvent from bookButton");
            book();
        });
        patientId.getDocument().addDocumentListener(new DocumentListener() {   // DocumentEvent ->
DocumentListener
            public void insertUpdate(DocumentEvent e)  { trace.add("DocumentEvent(insert) from
patientId"); }
            public void removeUpdate(DocumentEvent e)  { trace.add("DocumentEvent(remove) from
patientId"); }
            public void changedUpdate(DocumentEvent e) { }
        });
    }

    /** (a) refresh the doctor list when the department changes */
    private void refreshDoctors() {
```

```
        String dept = (String) departmentBox.getSelectedItem();
        doctorBox.removeAllItems();
        for (Doctor d : service.doctorsFor(dept)) doctorBox.addItem(d);        // empty list when
the prompt is selected
        status.setText(" ");
    }

    /** (b) validate and book (the database work runs in a SwingWorker, not on the event-dispatch
thread) */
    private void book() {
        final String id = patientId.getText();
        final String dept = (String) departmentBox.getSelectedItem();
        final Doctor doc = (Doctor) doctorBox.getSelectedItem();
        final String slot = (String) slotBox.getSelectedItem();
        busy = true;
        bookButton.setEnabled(false);
        new SwingWorker<BookingResult, Void>() {
            @Override protected BookingResult doInBackground() { return service.book(id, dept,
doc, slot); }
            @Override protected void done() {
                try {
                    BookingResult r = get();
                    if (r.success) resetForm();             // reset first: it fires the
department listener, which clears the status line
                    status.setForeground(r.success ? new Color(0x1B5E20) : new Color(0xB00020));
                    status.setText(r.message);
                } catch (InterruptedException | ExecutionException ex) {
                    status.setForeground(new Color(0xB00020));
                    status.setText("Unexpected error: " + ex.getMessage());
                } finally {
                    busy = false;
                    bookButton.setEnabled(true);
                }
            }
        }.execute();
    }

    private void resetForm() {
        patientId.setText("");
        departmentBox.setSelectedIndex(0);
        slotBox.setSelectedIndex(0);
    }

    /** True while a booking is being typed in or is being saved: the window then asks before
closing. */
    public boolean isBookingInProgress() {
        return busy || !patientId.getText().trim().isEmpty()
                || departmentBox.getSelectedIndex() > 0 || slotBox.getSelectedIndex() > 0;
    }

    public boolean isBusy() { return busy; }
    public List<String> trace() { return trace; }
    public String statusText() { return status.getText(); }
    public JTextField patientIdField() { return patientId; }
    public JComboBox<String> departmentBox() { return departmentBox; }
    public JComboBox<Doctor> doctorBox() { return doctorBox; }
    public JComboBox<String> slotBox() { return slotBox; }
    public JButton bookButton() { return bookButton; }
}
```

Task02/AppointmentWindow.java - WindowAdapter + confirmation before closing

```
import java.awt.event.WindowAdapter;
import java.awt.event.WindowEvent;
import javax.swing.*;
```

```
public class AppointmentWindow extends JFrame {
    private final AppointmentPanel panel;

    public AppointmentWindow(AppointmentService service) {
        super("Outpatient Appointment Desk");
        panel = new AppointmentPanel(service);
        add(panel);
        setDefaultCloseOperation(DO_NOTHING_ON_CLOSE);
        addWindowListener(new WindowAdapter() {
            @Override public void windowClosing(WindowEvent e) { if (confirmClose()) dispose(); }
        });
        pack();
        setLocationRelativeTo(null);
    }

    /** Asks the receptionist only when a booking is in progress. */
    boolean confirmClose() {
        if (!panel.isBookingInProgress()) return true;
        int answer = JOptionPane.showConfirmDialog(this, "A booking is in progress. Close the
window anyway?",
                "Confirm close", JOptionPane.YES_NO_OPTION);
        return answer == JOptionPane.YES_OPTION;
    }

    public AppointmentPanel panel() { return panel; }

    public static void main(String[] args) {
        AppointmentService service = new AppointmentService(new InMemoryAppointmentRepository());
        SwingUtilities.invokeLater(() -> new AppointmentWindow(service).setVisible(true));
    }
}
```

Task02/WindowTrace.java - run on a real display to print the real window-event order

```
import java.awt.*;
import java.awt.event.*;

/**
 * Run this on a machine WITH a display to print the real order of the window events (Task 2.4):
 *   javac WindowTrace.java  &&  java WindowTrace
 * Then minimise, restore and close the window and paste the console output into the report.
 */
public class WindowTrace extends Frame implements WindowListener, WindowFocusListener,
WindowStateListener {
    public WindowTrace() {
        super("Window trace");
        System.out.println("constructor finished");
        addWindowListener(this);
        addWindowFocusListener(this);
        addWindowStateListener(this);
        setSize(300, 150);
    }
    public void windowOpened(WindowEvent e)      { System.out.println("windowOpened"); }
    public void windowClosing(WindowEvent e)     { System.out.println("windowClosing"); dispose();
}
    public void windowClosed(WindowEvent e)      { System.out.println("windowClosed");
System.exit(0); }
    public void windowIconified(WindowEvent e)   { System.out.println("windowIconified"); }
    public void windowDeiconified(WindowEvent e) { System.out.println("windowDeiconified"); }
    public void windowActivated(WindowEvent e)   { System.out.println("windowActivated"); }
    public void windowDeactivated(WindowEvent e) { System.out.println("windowDeactivated"); }
    public void windowGainedFocus(WindowEvent e) { System.out.println("windowGainedFocus"); }
    public void windowLostFocus(WindowEvent e)   { System.out.println("windowLostFocus"); }
    public void windowStateChanged(WindowEvent e){ System.out.println("windowStateChanged"); }
    public void paint(Graphics g)                { System.out.println("paint"); }
```

```
    public static void main(String[] args) { System.out.println("main: setVisible(true)"); new
WindowTrace().setVisible(true); }
}
```

Task02/AppointmentServiceTest.java

```
import java.util.List;

public class AppointmentServiceTest {
    static int pass = 0, fail = 0;
    static void check(String name, BookingResult r, boolean expect, String part) {
        boolean ok = r.success == expect && r.message.contains(part);
        if (ok) pass++; else fail++;
        System.out.printf("%-4s %-34s %s%n", ok ? "PASS" : "FAIL", name, r.message);
    }
    public static void main(String[] args) {
        AppointmentService s = new AppointmentService(new InMemoryAppointmentRepository());
        System.out.println("Departments : " + s.departments());
        for (String d : s.departments()) System.out.println("Doctors of " + d + " : " +
s.doctorsFor(d));
        System.out.println();
        List<Doctor> card = s.doctorsFor("Cardiology");
        Doctor anand = card.get(0), meera = card.get(1), paed =
s.doctorsFor("Paediatrics").get(0);
        check("valid booking",               s.book("P1001", "Cardiology", anand, "10:00"), true,
"confirmed");
        check("same slot, same doctor",      s.book("P1002", "Cardiology", anand, "10:00"),
false, "already booked");
        check("same slot, other doctor",     s.book("P1002", "Cardiology", meera, "10:00"), true,
"confirmed");
        check("empty patient id",            s.book("", "Cardiology", anand, "11:00"), false,
"Enter the patient ID");
        check("bad patient id",              s.book("1001", "Cardiology", anand, "11:00"), false,
"must look like");
        check("no department",               s.book("P1003", AppointmentService.PROMPT, anand,
"11:00"), false, "Select a department");
        check("no doctor",                   s.book("P1003", "Cardiology", null, "11:00"), false,
"Select a doctor");
        check("doctor of another department", s.book("P1003", "Cardiology", paed, "11:00"), false,
"does not belong");
        check("no slot",                     s.book("P1003", "Cardiology", anand,
AppointmentService.PROMPT), false, "Select a time slot");
        System.out.println("\nResult: " + pass + " passed, " + fail + " failed");
    }
}
```

Task02/schema.sql

```
-- Task 2 : outpatient appointment desk (MySQL)
CREATE DATABASE IF NOT EXISTS hospital;
USE hospital;

CREATE TABLE department (
    dept_id INT AUTO_INCREMENT PRIMARY KEY,
    name    VARCHAR(40) NOT NULL UNIQUE
);
CREATE TABLE doctor (
    doctor_id VARCHAR(8)  PRIMARY KEY,
    name      VARCHAR(60) NOT NULL,
    dept_id   INT         NOT NULL,
    FOREIGN KEY (dept_id) REFERENCES department(dept_id),
    INDEX idx_doctor_dept (dept_id)            -- makes "doctors of a department" an index lookup
);
CREATE TABLE patient (
    patient_id VARCHAR(8) PRIMARY KEY,
    name       VARCHAR(60) NOT NULL,
    phone      VARCHAR(15)
```

```
);
CREATE TABLE appointment (
    appt_id    INT AUTO_INCREMENT PRIMARY KEY,
    patient_id VARCHAR(8) NOT NULL,
    doctor_id  VARCHAR(8) NOT NULL,
    appt_date  DATE       NOT NULL,
    slot_time  CHAR(5)    NOT NULL,
    FOREIGN KEY (patient_id) REFERENCES patient(patient_id),
    FOREIGN KEY (doctor_id)  REFERENCES doctor(doctor_id),
    UNIQUE KEY uq_doctor_slot (doctor_id, appt_date, slot_time)   -- a doctor cannot be booked
twice for one slot
);
```

Database design (ER diagram)

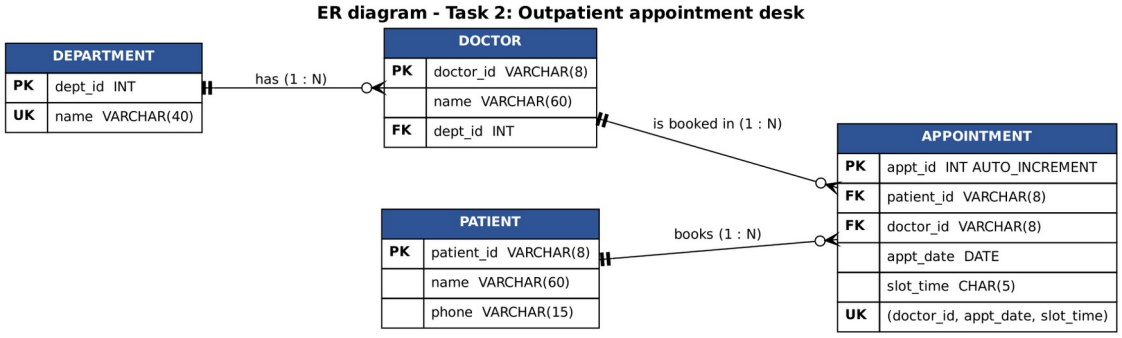


Figure 2.3 – ER diagram of the hospital database (the UNIQUE key on doctor, date and slot prevents double booking)

Evidence – service tests

Output of: java AppointmentServiceTest

```
Departments : [Cardiology, Dermatology, Orthopaedics, Paediatrics]
Doctors of Cardiology : [Dr. Anand Rao, Dr. Meera Nair, Dr. S. Kumar]
Doctors of Dermatology : [Dr. Priya Sharma, Dr. V. Balaji, Dr. Nisha George]
Doctors of Orthopaedics : [Dr. Rajesh Iyer, Dr. Latha Menon, Dr. A. Fernandes]
Doctors of Paediatrics : [Dr. Kavitha Rao, Dr. Imran Khan, Dr. Sneha Pillai]

PASS valid booking                     Appointment confirmed: P1001 with Dr. Anand Rao (Cardiology) at
10:00.
PASS same slot, same doctor            Dr. Anand Rao is already booked at 10:00.
PASS same slot, other doctor           Appointment confirmed: P1002 with Dr. Meera Nair (Cardiology) at
10:00.
PASS empty patient id                  Enter the patient ID.
PASS bad patient id                    Patient ID must look like P1001.
PASS no department                     Select a department.
PASS no doctor                         Select a doctor.
PASS doctor of another department      The doctor does not belong to Cardiology.
PASS no slot                           Select a time slot.

Result: 9 passed, 0 failed
```

Evidence – the real window classes driven by real events (off-screen render)

Output of the harness AppointmentHarness (includes the list of events handled, in order)

```
[1] window created. Doctor list: []
[2] department = Cardiology  -> doctor list: [Dr. Anand Rao, Dr. Meera Nair, Dr. S. Kumar]
[3] department = Paediatrics -> doctor list: [Dr. Kavitha Rao, Dr. Imran Khan, Dr. Sneha Pillai]
[4] Book with empty patient ID -> Enter the patient ID.
[5] in-progress booking? true  -> closing asks for confirmation
    receptionist answers NO  -> window closes? false
    receptionist answers YES -> window closes? true
    [confirm:Confirm close] A booking is in progress. Close the window anyway? -> 1
    [confirm:Confirm close] A booking is in progress. Close the window anyway? -> 0
[6] Book -> Appointment confirmed: P1001 with Dr. Imran Khan (Paediatrics) at 10:00.
[7] form cleared after success; in-progress now? false  -> closing needs no question: true
```

```
[8] Same doctor and slot, other patient -> Dr. Imran Khan is already booked at 10:00.

Events handled by the panel, in order:
  1. ItemEvent(SELECTED) from departmentBox -> Cardiology
  2. ItemEvent(SELECTED) from departmentBox -> Paediatrics
  3. ActionEvent from bookButton
  4. DocumentEvent(insert) from patientId
  5. ActionEvent from bookButton
  6. DocumentEvent(remove) from patientId
  7. ItemEvent(SELECTED) from departmentBox -> -- select --
  8. DocumentEvent(insert) from patientId
  9. ItemEvent(SELECTED) from departmentBox -> Paediatrics
  10. ActionEvent from bookButton
```

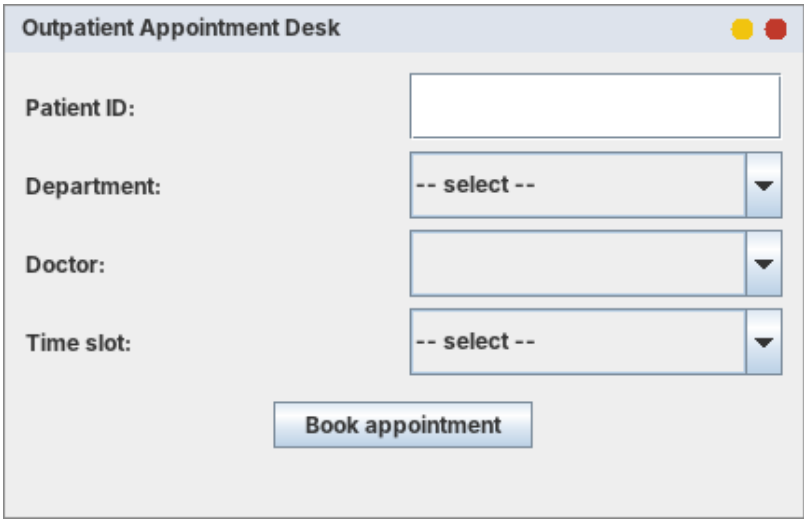


Figure 2.4 – Initial window: the doctor list is empty until a department is chosen

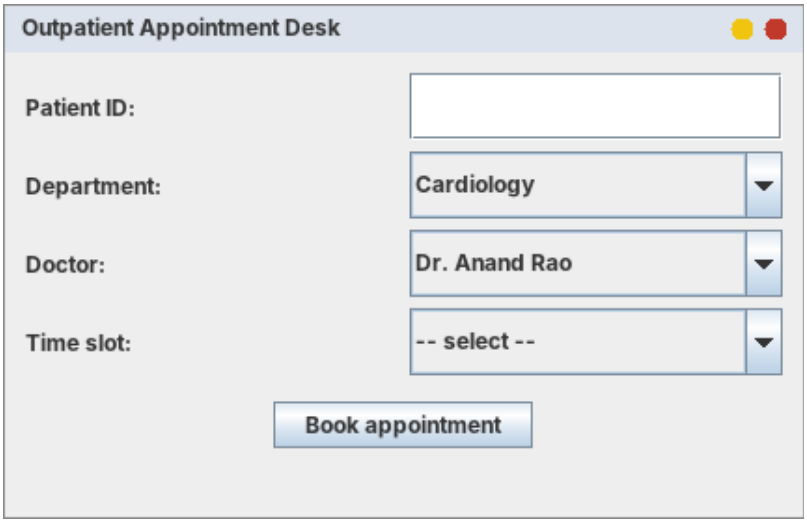


Figure 2.5 – Demonstrates RefreshDoctorList: Cardiology selected → its three doctors

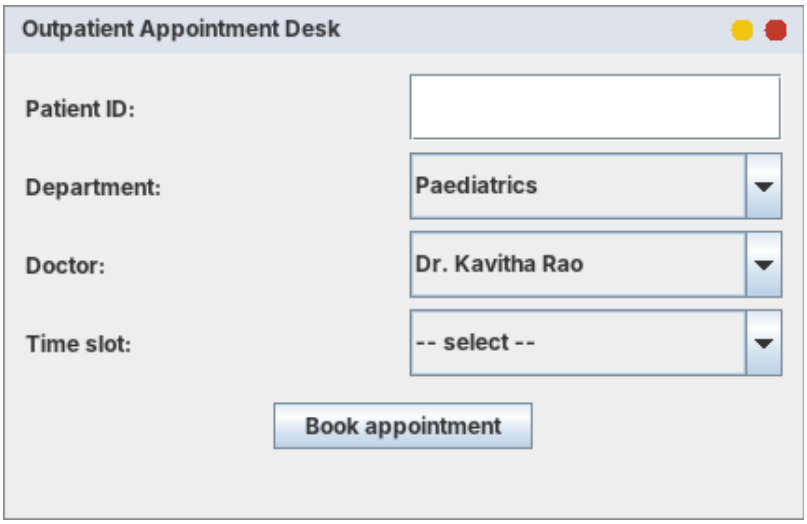


Figure 2.6 – Department changed to Paediatrics → the list is replaced

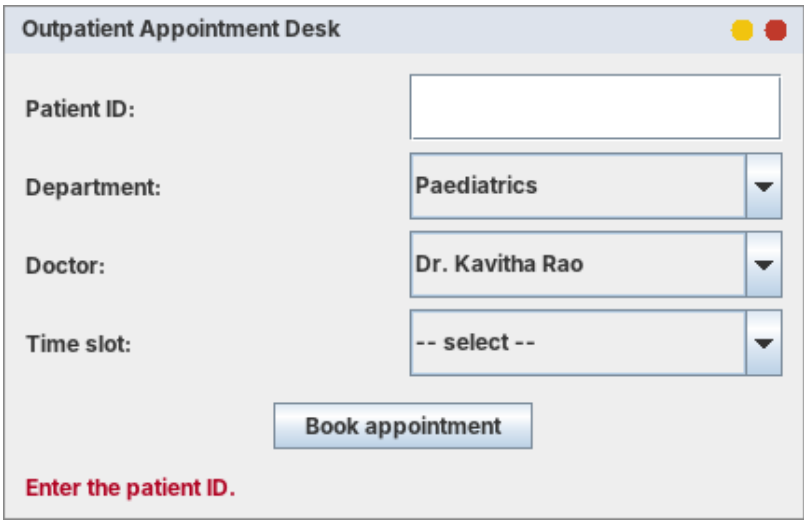


Figure 2.7 – Demonstrates ValidateAndBook step 2: empty patient ID

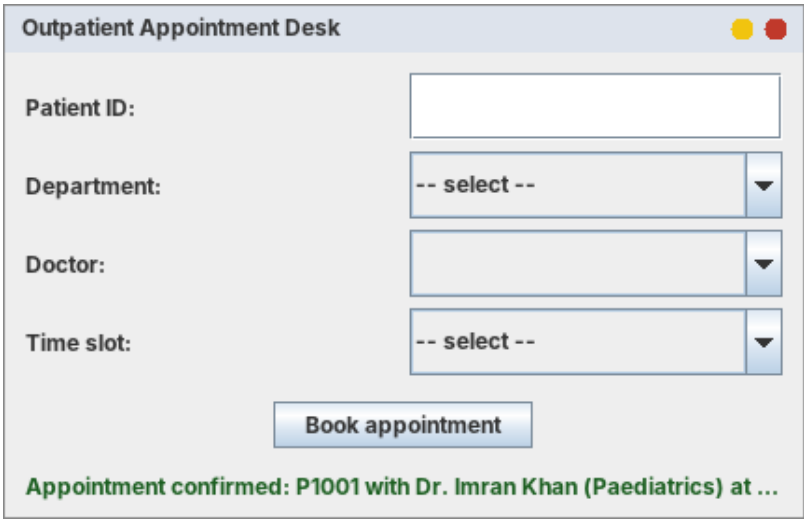


Figure 2.8 – Demonstrates ValidateAndBook step 11: confirmation shown, form cleared

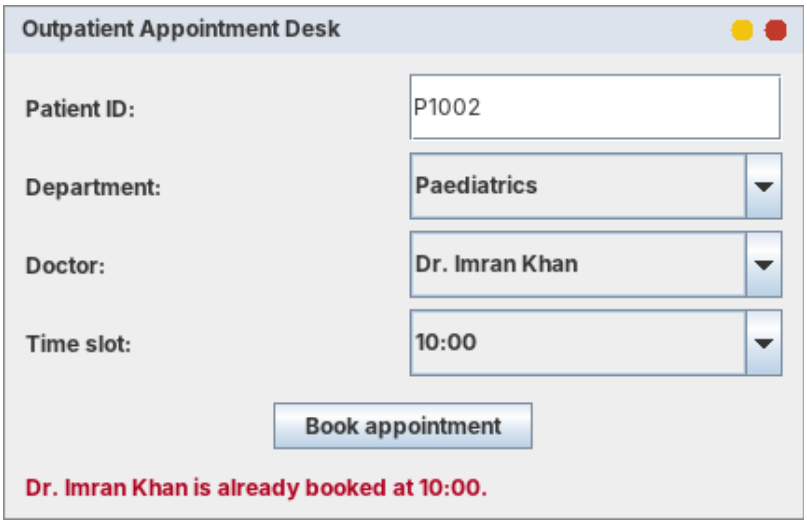


Figure 2.9 – Demonstrates step 8: the same doctor and slot cannot be booked twice

Closing confirmation. Output [5] above shows the window logic: with a booking in progress, the answer NO keeps the window open (confirmClose() = false) and YES closes it; after a successful booking the form is empty, so closing needs no question. **A defect found by the event trace:** in the first version the success message was blank. Resetting the form changes the department combo box, which fires its ItemListener, and that listener cleared the status line right after the message was set. The fix (AppointmentPanel.done(): reset the form first, then show the message) is in the code above; this is the kind of defect that only shows when the order of events is traced.

Alternative concept and complexity analysis

The alternative solution. Instead of loading all doctors once into a Map<String, List<Doctor>> (the cache used above), the doctor list is fetched from the database every time the department changes:

```
// cache (used): one query at start-up, then a Map lookup per change
private final Map<String, List<Doctor>> doctorsByDepartment = new
TreeMap<>(repository.loadAllDoctors());
public List<Doctor> doctorsFor(String department) { return
doctorsByDepartment.getOrDefault(department, List.of()); }

// alternative: one query per change (SELECT d.doctor_id, d.name FROM doctor d JOIN department
p ... WHERE p.name = ?)
public List<Doctor> doctorsFor(String department) { return repository.findDoctors(department); }
// must run in a SwingWorker
```

d departments, m doctors	Cached Map	Query on every change
Time per department change	O(1) lookup in the Map, then O(m/d) to fill the combo box (the same fill is needed in both)	O(log m + m/d) with an index on dept_id (O(m) without one) + one network round trip; must run off the event thread
Time at start-up	one query, O(m)	none
Memory	O(m): all m doctors stay in memory (d lists + m references; the Doctor objects, about 100 bytes each, are the larger part)	O(m/d): only the doctors of the current department
Freshness	stale until reloaded (a new doctor is not seen)	always current

Measurements (CacheBenchmark.java). The “database” is the in-memory repository, whose query scans all doctors like a query without an index; one real network round trip is assumed to cost 2 ms (an assumption, not a measurement).

d	m	Map lookup (ns)	DB query scan (ns)	DB + 2 ms round trip (ms)	Map overhead (MB, references only)
10	1,000	370.7	51,427	2.051	0.01

d	m	Map lookup (ns)	DB query scan (ns)	DB + 2 ms round trip (ms)	Map overhead (MB, references only)
10	100,000	113.0	971,032	2.971	0.54
100	100,000	328.4	1,187,616	3.188	0.48
1,000	200,000	297.7	4,781,786	6.782	1.01

Table 2.3 – Department change: cache against query (from Task02/cache_results.csv)

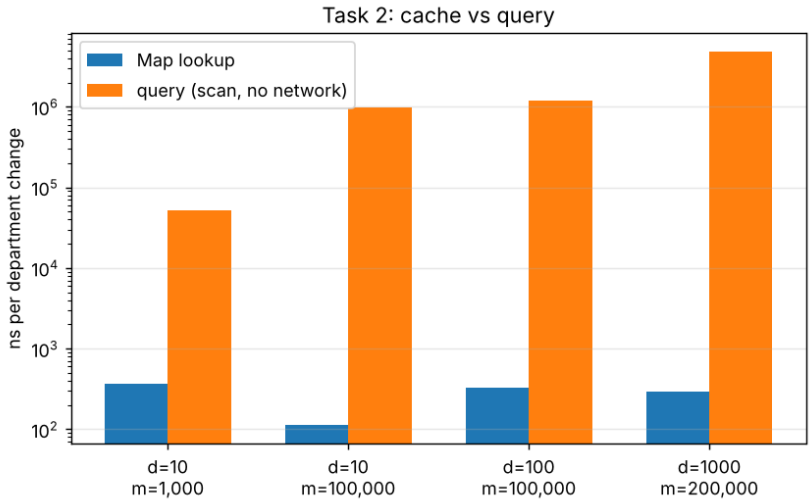


Figure 2.10 – Time per department change (logarithmic axis)

Analysis. The cached lookup takes about 0.1–0.4 µs whatever m is (O(1); the first row is affected by warm-up). The query without an index grows with m: 51 µs for 1,000 doctors, about 971 µs for 100,000 and 4.8 ms for 200,000 (linear, O(m)); with a database index and a network trip it is a few milliseconds. The Map therefore answers 100 to 10,000 times faster. The Map overhead measured here (0.01–1.01 MB) counts only the lists and references because the Doctor objects already exist in the simulated database; in a real application the m Doctor objects also stay in memory (an estimate of 100 bytes each gives about 10 MB for 100,000 doctors). **Evaluation:** a hospital has tens or a few hundred doctors that change rarely, so the cache is the better design (the receptionist sees the list instantly); a “Refresh doctors” menu item or a time limit on the cache solves the freshness problem. Fetching on every change is better only for very large or frequently changing data.

Task 3 – From Sketch to Layout: Campus Library Search Screen

Bloom's level K4 – Analyse. 9 marks. The screen has a title bar at the top; a search row (text field, category selection and Search button); a results area in the centre; a row of Prev, Next and Reserve buttons at the bottom; and a status message. When the window is resized, **the results area must grow while the search row and the buttons keep their size.** (The category "Choice" of AWT is implemented with its Swing equivalent, JComboBox.)

Task 3.1 – Choose the layout managers

Container	Layout manager	Justification with respect to the resizing requirement
Screen (root panel)	BorderLayout	NORTH and SOUTH keep their preferred height; CENTER receives ALL the extra width and height. Putting the results area in CENTER is exactly "results grow, the rest keeps its size".
north panel (title + search row)	BorderLayout	The title bar goes to NORTH (stretches across the full width, fixed height); the search row goes to CENTER of this small panel. Nesting is needed because the root has only one NORTH slot.
searchRow (label, text field, Choice, Search)	FlowLayout (LEFT)	FlowLayout gives every component its preferred size and never stretches it, so the field and the button keep their size when the window is resized. GridLayout would force equal cell sizes and stretch them, which breaks the requirement.
south panel (buttons + status)	BorderLayout	Button row in CENTER, status message in SOUTH; both keep their preferred height.
buttonRow (Prev, Next, Reserve)	FlowLayout (CENTER)	The three buttons keep their size and stay centred when the width changes.
Results area	JScrollPane (JTextArea inside)	Placed in the BorderLayout CENTER; scroll bars appear when the list is longer than the area.

CardLayout was not used: it shows only one component of a container at a time (useful for switching between whole screens, as in Task 6), and this screen must show all its parts together.

Task 3.2 – Nested containers

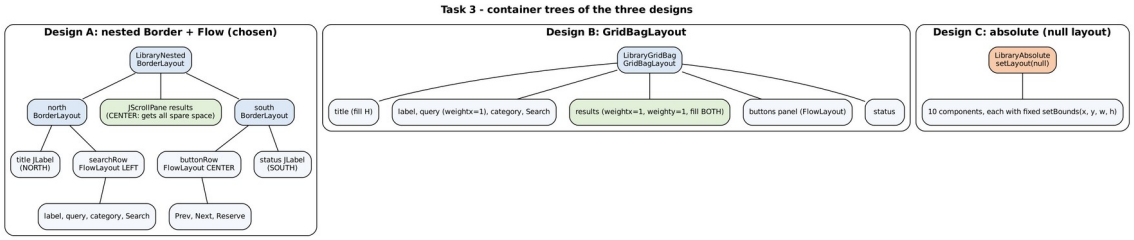


Figure 3.1 – Nested panel structure with the layout manager of each panel (Design A is the chosen design; B and C are the alternatives analysed below)

```
+-------------------------------------------------------------+  <- LibraryNested (BorderLayout)
| north (BorderLayout)                                        |
|   NORTH : [            Campus Library Search             ]  |
|   CENTER: searchRow (FlowLayout LEFT)                       |
|           Search: [_______________] [All v] [Search]        |
+-------------------------------------------------------------+
|                                                             |
|   CENTER: results (JScrollPane)  <- takes all the extra space |
|                                                            |
+-------------------------------------------------------------+
| south (BorderLayout)                                        |
|   CENTER: buttonRow (FlowLayout CENTER)  [Prev] [Next] [Reserve]|
|   SOUTH : status message                                    |
+-------------------------------------------------------------+
```

Task 3.3 – Implementation

The three layout designs share one base class (LibraryScreen) that creates the widgets and the behaviour (search, paging, reserve), so that they differ ONLY in their layout. Design A (LibraryNested) is the chosen solution; B and C are used in the alternative analysis.

Task03/BookCatalog.java

```
import java.util.ArrayList;
import java.util.List;

/** A very small in-memory catalogue so the search screen has something to show. */
public class BookCatalog {
    private static final String[][] BOOKS = {
        {"Programming in Java", "Balagurusamy", "Computer Science", "9780070141691"},
        {"Head First Java", "Kathy Sierra", "Computer Science", "9780596009205"},
        {"Effective Java", "Joshua Bloch", "Computer Science", "9780134685991"},
        {"Database System Concepts", "Silberschatz", "Computer Science", "9780078022159"},
        {"Operating System Concepts", "Silberschatz", "Computer Science", "9781119800361"},
        {"Engineering Mathematics", "B. S. Grewal", "Mathematics", "9788193328491"},
        {"Digital Electronics", "Morris Mano", "Electronics", "9780132774208"},
        {"Wings of Fire", "A. P. J. Abdul Kalam", "Biography", "9788173711466"},
        {"The Discovery of India", "Jawaharlal Nehru", "History", "9780143031031"},
        {"Data Structures Using C", "Reema Thareja", "Computer Science", "9780198099307"},
        {"Discrete Mathematics", "Kenneth Rosen", "Mathematics", "9780073383095"},
        {"Clean Code", "Robert C. Martin", "Computer Science", "9780132350884"}};

    public static List<String> search(String category, String text) {
        String q = text == null ? "" : text.trim().toLowerCase();
        List<String> out = new ArrayList<>();
        for (String[] b : BOOKS) {
            String field;
            switch (category) {
                case "Title":   field = b[0]; break;
                case "Author":  field = b[1]; break;
                case "Subject": field = b[2]; break;
                case "ISBN":    field = b[3]; break;
                default:        field = String.join(" ", b);
            }
            if (q.isEmpty() || field.toLowerCase().contains(q)) out.add(b[0] + "  -  " + b[1] + "
(" + b[2] + ")");
        }
        return out;
    }
}
```

Task03/LibraryScreen.java - common widgets and behaviour

```
import java.awt.event.ActionEvent;
import java.util.ArrayList;
import java.util.List;
import javax.swing.*;

/** Common widgets and behaviour of the search screen. The three subclasses differ ONLY in their
layout. */
public abstract class LibraryScreen extends JPanel {
    protected final JLabel title = new JLabel("Campus Library Search", SwingConstants.CENTER);
    protected final JTextField query = new JTextField(20);
    protected final JComboBox<String> category = new JComboBox<>(new String[]{"All", "Title",
"Author", "Subject", "ISBN"});
    protected final JButton search = new JButton("Search");
    protected final JTextArea results = new JTextArea();
    protected final JScrollPane resultsScroll = new JScrollPane(results);
    protected final JButton prev = new JButton("Prev");
    protected final JButton next = new JButton("Next");
    protected final JButton reserve = new JButton("Reserve");
```

```
    protected final JLabel status = new JLabel("Ready");

    private static final int PAGE = 5;
    private List<String> found = new ArrayList<>();
    private int page = 0;

    protected LibraryScreen() {
        title.setFont(title.getFont().deriveFont(java.awt.Font.BOLD, 18f));
        title.setOpaque(true);
        title.setBackground(new java.awt.Color(0x2F5496));
        title.setForeground(java.awt.Color.WHITE);
        results.setEditable(false);
        search.addActionListener(this::doSearch);
        query.addActionListener(this::doSearch);
        prev.addActionListener(e -> { if (page > 0) { page--; showPage(); } });
        next.addActionListener(e -> { if ((page + 1) * PAGE < found.size()) { page++;
showPage(); } });
        reserve.addActionListener(e -> status.setText("Reserve requested for page " + (page +
1)));
    }

    private void doSearch(ActionEvent e) {
        found = BookCatalog.search((String) category.getSelectedItem(), query.getText());
        page = 0;
        showPage();
    }

    private void showPage() {
        StringBuilder sb = new StringBuilder();
        for (int i = page * PAGE; i < Math.min(found.size(), (page + 1) * PAGE); i++) sb.append(i
+ 1).append(". ").append(found.get(i)).append('\n');
        results.setText(sb.toString());
        status.setText(found.size() + " result(s) - page " + (page + 1) + " of " + Math.max(1,
(found.size() + PAGE - 1) / PAGE));
    }

    // used by the tests
    public JButton searchButton() { return search; }
    public JTextField queryField() { return query; }
    public JScrollPane resultsPane() { return resultsScroll; }
    public JButton prevButton() { return prev; }
    public JButton nextButton() { return next; }
    public JButton reserveButton() { return reserve; }
    public JLabel statusLabel() { return status; }
    public JLabel titleLabel() { return title; }
    public JComboBox<String> categoryBox() { return category; }
}
```

Task03/LibraryNested.java - DESIGN A – nested BorderLayout + FlowLayout (chosen)

```
import java.awt.*;
import javax.swing.*;

/**
 * DESIGN A (chosen): nested BorderLayout + FlowLayout panels.
 *
 *   this (BorderLayout)
 *     NORTH  : north  (BorderLayout)  -> NORTH: title,  CENTER: searchRow (FlowLayout LEFT)
 *     CENTER : results (JScrollPane)                  <- gets ALL the extra space
 *     SOUTH  : south  (BorderLayout)  -> CENTER: buttonRow (FlowLayout CENTER),  SOUTH: status
 */
public class LibraryNested extends LibraryScreen {
    public LibraryNested() {
        setLayout(new BorderLayout(0, 4));

        JPanel searchRow = new JPanel(new FlowLayout(FlowLayout.LEFT, 8, 6));
```

```
        searchRow.add(new JLabel("Search:"));
        searchRow.add(query);
        searchRow.add(category);
        searchRow.add(search);

        JPanel north = new JPanel(new BorderLayout());
        north.add(title, BorderLayout.NORTH);
        north.add(searchRow, BorderLayout.CENTER);

        JPanel buttonRow = new JPanel(new FlowLayout(FlowLayout.CENTER, 10, 6));
        buttonRow.add(prev);
        buttonRow.add(next);
        buttonRow.add(reserve);

        JPanel south = new JPanel(new BorderLayout());
        south.add(buttonRow, BorderLayout.CENTER);
        south.add(status, BorderLayout.SOUTH);
        status.setBorder(BorderFactory.createEmptyBorder(2, 8, 4, 8));

        add(north, BorderLayout.NORTH);
        add(resultsScroll, BorderLayout.CENTER);
        add(south, BorderLayout.SOUTH);
    }
}
```

Task03/LibraryGridBag.java - DESIGN B – GridBagLayout

```
import java.awt.*;
import javax.swing.*;

/** DESIGN B: one panel with GridBagLayout (weights and fill decide what grows). */
public class LibraryGridBag extends LibraryScreen {
    public LibraryGridBag() {
        setLayout(new GridBagLayout());
        GridBagConstraints c = new GridBagConstraints();
        c.insets = new Insets(4, 6, 4, 6);

        c.gridx = 0; c.gridy = 0; c.gridwidth = 4; c.weightx = 1; c.weighty = 0; c.fill =
GridBagConstraints.HORIZONTAL;
        add(title, c);                                                    // title bar: stretches
sideways only

        c.gridy = 1; c.gridwidth = 1; c.weightx = 0; c.fill = GridBagConstraints.NONE; c.anchor =
GridBagConstraints.WEST;
        add(new JLabel("Search:"), c);
        c.gridx = 1; c.weightx = 1; c.fill = GridBagConstraints.HORIZONTAL;
        add(query, c);                                                    // the text field takes
the spare width
        c.gridx = 2; c.weightx = 0; c.fill = GridBagConstraints.NONE;
        add(category, c);
        c.gridx = 3;
        add(search, c);

        c.gridx = 0; c.gridy = 2; c.gridwidth = 4; c.weightx = 1; c.weighty = 1; c.fill =
GridBagConstraints.BOTH;
        add(resultsScroll, c);                                            // results: grows both
ways

        JPanel buttons = new JPanel(new FlowLayout(FlowLayout.CENTER, 10, 0));   // one small
helper panel for the 3 buttons
        buttons.add(prev); buttons.add(next); buttons.add(reserve);
        c.gridy = 3; c.weighty = 0; c.fill = GridBagConstraints.NONE; c.anchor =
GridBagConstraints.CENTER;
        add(buttons, c);

        c.gridy = 4; c.fill = GridBagConstraints.HORIZONTAL; c.anchor = GridBagConstraints.WEST;
```

```
        add(status, c);
    }
}
```

Task03/LibraryAbsolute.java - DESIGN C – absolute positioning

```
import javax.swing.*;

/** DESIGN C: absolute positioning (setLayout(null) + setBounds). Positions are fixed numbers, so
nothing reacts to resizing. */
public class LibraryAbsolute extends LibraryScreen {
    public LibraryAbsolute() {
        setLayout(null);
        title.setBounds(0, 0, 600, 34);
        JLabel label = new JLabel("Search:");
        label.setBounds(10, 44, 55, 26);
        query.setBounds(70, 44, 250, 26);
        category.setBounds(330, 44, 100, 26);
        search.setBounds(440, 44, 90, 26);
        resultsScroll.setBounds(10, 80, 580, 240);
        prev.setBounds(150, 330, 90, 28);
        next.setBounds(255, 330, 90, 28);
        reserve.setBounds(360, 330, 90, 28);
        status.setBounds(10, 368, 580, 22);
        add(title); add(label); add(query); add(category); add(search);
        add(resultsScroll); add(prev); add(next); add(reserve); add(status);
    }
    @Override public java.awt.Dimension getPreferredSize() { return new java.awt.Dimension(600,
400); }
}
```

Task03/LibraryFrame.java - run with: java LibraryFrame [nested|gridbag|absolute]

```
import javax.swing.*;

/** Runs the chosen design in a real window:  java LibraryFrame [nested|gridbag|absolute] */
public class LibraryFrame extends JFrame {
    public LibraryFrame(LibraryScreen screen) {
        super("Campus Library Search");
        add(screen);
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setSize(700, 450);
        setLocationRelativeTo(null);
    }
    public static void main(String[] args) {
        String design = args.length > 0 ? args[0] : "nested";
        SwingUtilities.invokeLater(() -> {
            LibraryScreen s = design.equals("gridbag") ? new LibraryGridBag() :
design.equals("absolute") ? new LibraryAbsolute() : new LibraryNested();
            new LibraryFrame(s).setVisible(true);
        });
    }
}
```

Task 3.4 – Test the resizing behaviour

The harness (ResizeHarness, Appendix B) lays out each design at three window sizes, prints the size of every region and saves a picture. The printed sizes are the evidence; the pictures show the same thing.

Output of ResizeHarness (region sizes in pixels: width x height)

```
Region sizes (width x height in pixels) after layout at three window sizes

DESIGN: nested
window       results    search fld Search btn Prev btn   status     title
520x330      520x204    224x19     75x25      61x25      520x21     520x23
800x500      800x374    224x19     75x25      61x25      800x21     800x23
```

```
            results grew: true   Search button kept its size: true
1100x650    1100x524   224x19     75x25      61x25      1100x21    1100x23
            results grew: true   Search button kept its size: true

DESIGN: gridbag
window      results    search fld Search btn Prev btn   status     title
520x330     508x202    280x19     75x25      61x25      508x15     508x23
800x500     788x372    560x19     75x25      61x25      788x15     788x23
            results grew: true   Search button kept its size: true
1100x650    1088x522   860x19     75x25      61x25      1088x15    1088x23
            results grew: true   Search button kept its size: true

DESIGN: absolute
window      results    search fld Search btn Prev btn   status     title
520x330     580x240    250x26     90x26      90x28      580x22     600x34
800x500     580x240    250x26     90x26      90x28      580x22     600x34
            results grew: false   Search button kept its size: true
1100x650    580x240    250x26     90x26      90x28      580x22     600x34
            results grew: false   Search button kept its size: true
```

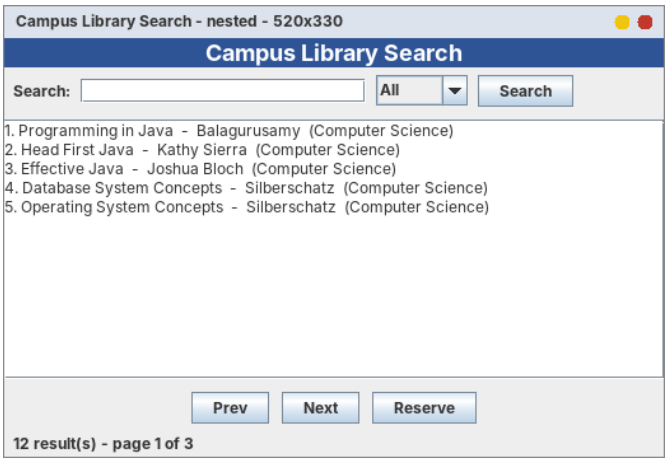


Figure 3.2 – Design A at 520 × 330 (small). Demonstrates the requirement: all regions visible

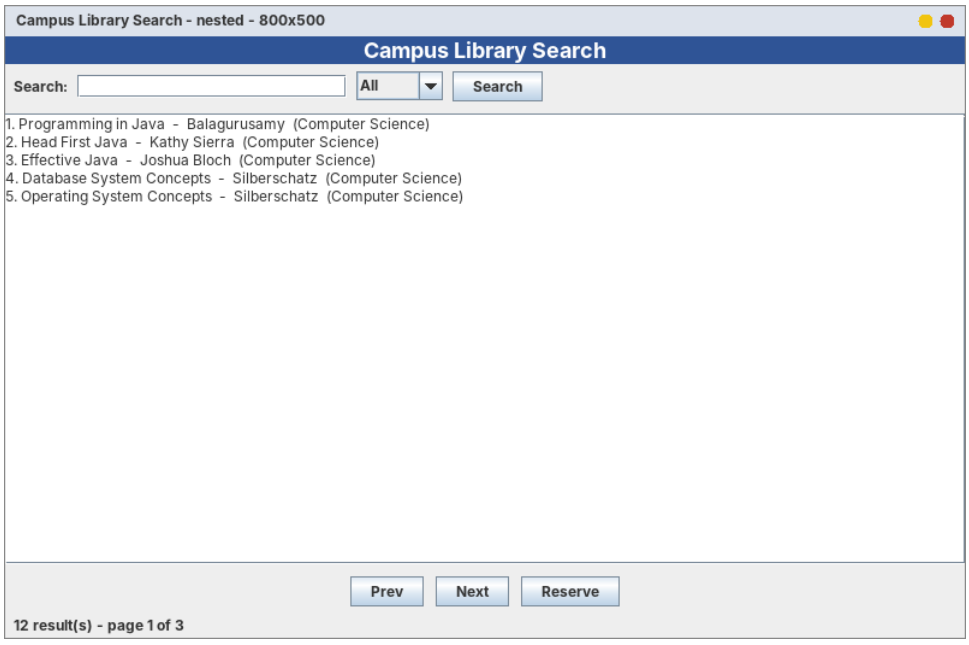


Figure 3.3 – Design A at 800 × 500 (medium)

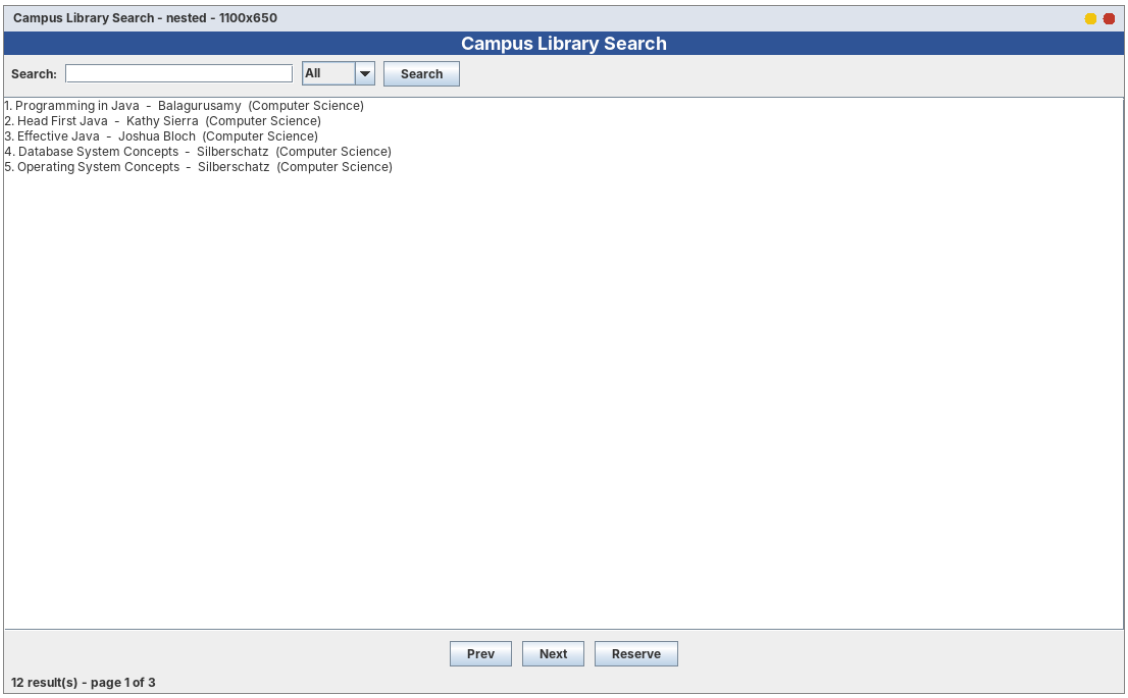


Figure 3.4 – Design A at 1100 × 650 (large)

Region	Behaved as designed?	Evidence (520×330 → 800×500 → 1100×650)
Title bar	Yes: stretches sideways, height fixed	width 520 → 800 → 1100, height 23 at all sizes
Search row (field, Choice, Search button)	Yes: keeps its size	text field 224×19 and Search button 75×25 at all three sizes; the row stays at the left
Results area	Yes: grows in both directions	height 204 → 374 → 524 and width 520 → 800 → 1100 (it takes all the extra space)
Prev / Next / Reserve buttons	Yes: keep their size and stay centred	Prev button 61×25 at all three sizes
Status message	Yes: fixed height, follows the width	height 21, width 520 → 800 → 1100

Table 3.1 – Resizing behaviour of Design A: every region behaved as designed

Alternative concept and complexity analysis

The alternative solutions. (B) One panel with GridBagLayout: every component gets a cell, and weightx / weighty / fill decide what grows. (C) Absolute positioning: setLayout(null) and setBounds(x, y, w, h) for each of the ten components. The key code is in LibraryGridBag.java and LibraryAbsolute.java above.

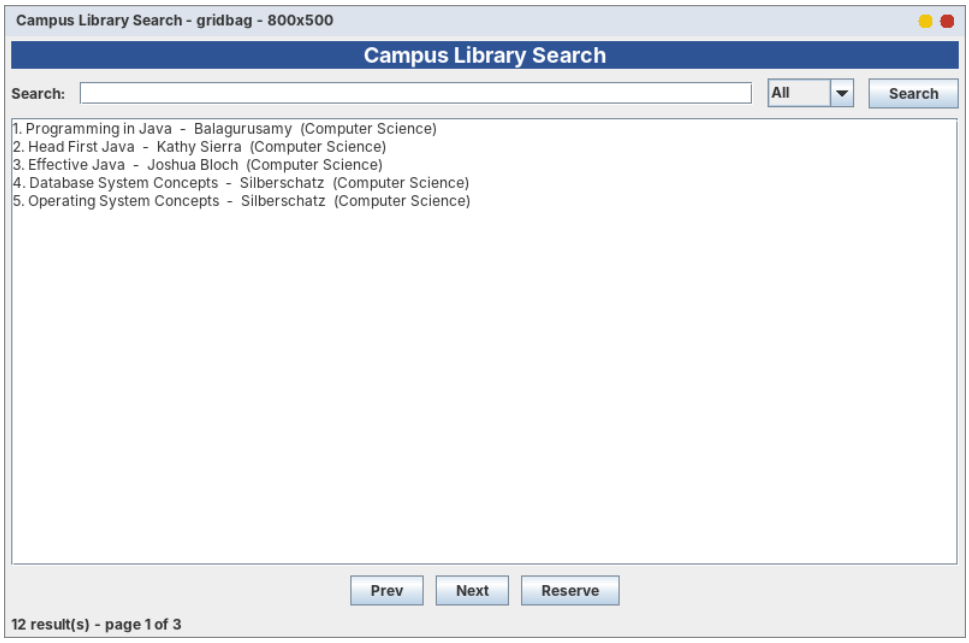


Figure 3.5 – Design B (GridBagLayout) at 800 × 500: results grow; the search text field also stretches because it was given weightx = 1

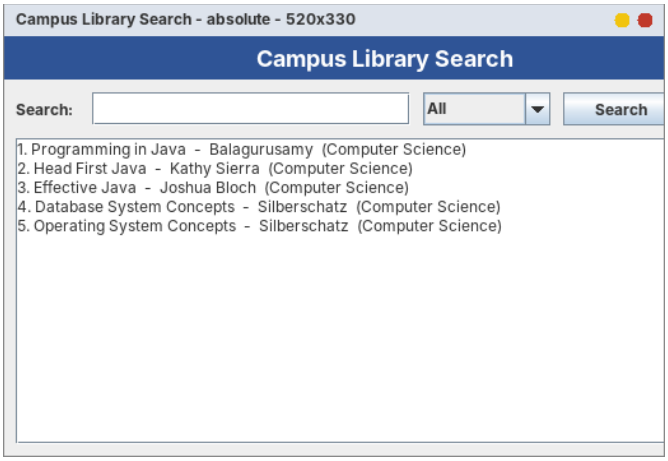


Figure 3.6 – Design C (absolute) at 520 × 330: the window is smaller than the fixed coordinates, so the right side and the bottom (status, Search button) are cut off

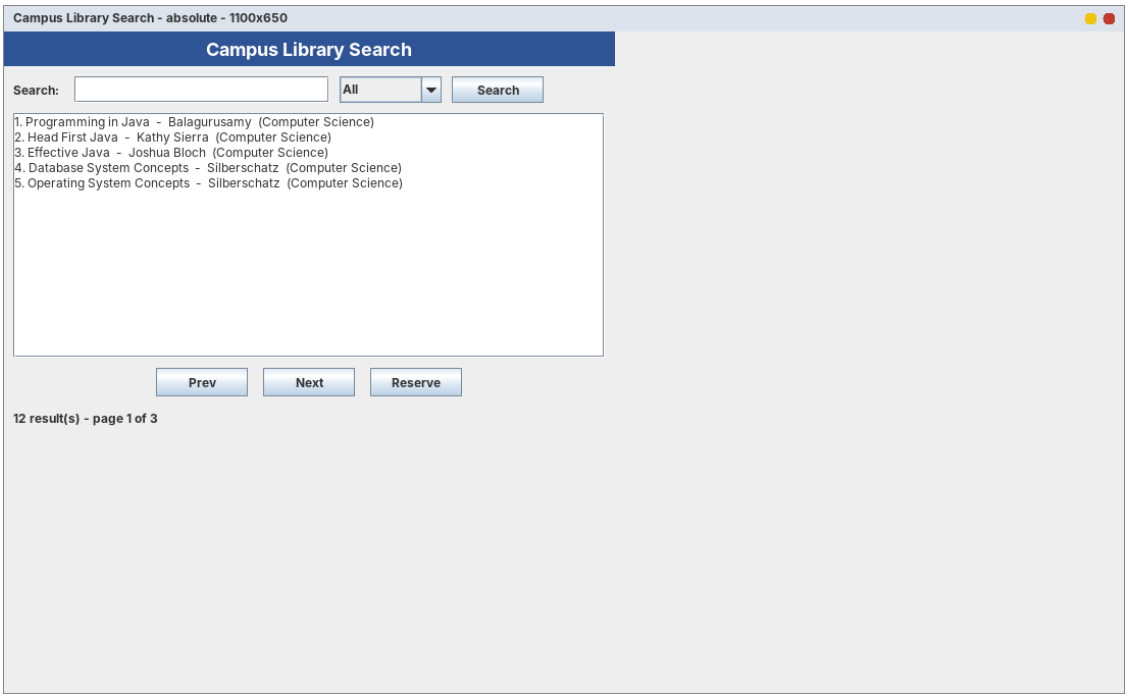


Figure 3.7 – Design C at 1100 × 650: nothing moved or grew, the results area is still 580 × 240 and the rest is empty

	A: nested Border + Flow	B: GridBagLayout	C: absolute (null layout)
Resizing behaviour	As required: results grow, search row and buttons keep their size	As required if weights are set correctly; the field can stretch if wanted (more control)	None: positions and sizes are fixed numbers; clipped when smaller, empty when larger
Layout computation per resize	O(c) – a few simple managers, c = number of components	O(c) with a larger constant – the manager builds a grid matrix and distributes weights	O(c) – only reads the stored bounds (no computation)
Container objects (space)	4 helper JPanels + the root	1 helper JPanel (buttons) + the root	0 helper panels; only the root
Change effort	Easy to read: one manager per panel	Powerful but the constraints are hard to read and tune	Breaks on other fonts, screen sizes and languages

Measurements (LayoutBenchmark.java: layout of the whole component tree after 20,000 resizes, average of 5 runs):

Design	Helper JPanels	All containers in the tree	Components	Layout time (µs per resize)
nested	4	25	25	22.21
gridbag	1	22	22	28.45
absolute	0	21	21	4.51

Table 3.2 – Layout cost and number of container objects (from Task03/layout_results.csv)

Analysis. All three are linear in the number of components. Absolute positioning is fastest (4.51 µs) because it does no computation, GridBagLayout is slowest (28.45 µs) because it builds the grid, and the nested design is in between (22.21 µs). The nested and GridBag designs need about 0.02–0.03 ms per resize and the absolute design about 0.005 ms; even 0.03 ms is only 0.2% of the 16 ms available for one frame at 60 frames per second, so the speed difference is of no practical importance. The “all containers” column counts the containers inside the Swing components themselves (scroll pane, combo box), which is why it is 21–25 for every design; the number of helper panels we created is 4, 1 and 0. Each JPanel costs only a few hundred bytes, so the extra memory of Design A is negligible. **Evaluation:** Design A is the best choice because it satisfies the resizing requirement with simple managers that are easy to read; GridBagLayout is the alternative when finer control is needed; absolute positioning fails the requirement.

Task 4 – Event-Handling Forensics: The Drawing Pad

Bloom's level K4 – Analyse. 7 marks. A drawing pad should draw a dot wherever the user clicks and show the click count in a label. Users report that nothing is drawn and the count stays at 0. The original program (class renamed DrawPadBuggy so that it can live beside the repaired DrawPad):

Task04/DrawPadBuggy.java - the original program

```
import java.awt.*;
import java.awt.event.*;

/** The ORIGINAL program from the assignment (renamed DrawPadBuggy so that it can live beside the
repaired DrawPad). */
public class DrawPadBuggy extends Frame {
    int count = 0;
    Label status = new Label("Clicks: 0");

    public DrawPadBuggy() {
        add(status, BorderLayout.SOUTH);
        addMouseListener(new MouseAdapter() {
            public void mouseclicked(MouseEvent e) {            // DEFECT 1: wrong spelling (small
c)
                count++;
                Graphics g = getGraphics();
                g.fillOval(e.getX(), e.getY(), 6, 6);
            }
        });
        setSize(400, 300);
        setVisible(true);
    }
}
```

Task 4.1 – Locate the defect

Why the click is never handled. The listener class extends MouseAdapter, whose method is called **mouseClicked** (capital C). The program declares **mouseclicked** (small c). Java is case sensitive, so this is a *new, unrelated method*, not an override. When the click happens, AWT calls mouseClicked(MouseEvent), which in MouseAdapter is an empty method, so nothing happens. The compiler does not complain, because declaring an extra method is legal.

Why the label never changes. Two separate reasons: (1) the method that would change it is never called (above); (2) even if it were called, its body only does count++ and draws a dot; no statement ever calls status.setText(...), so the label would still read “Clicks: 0”. The defect list is therefore: D1 wrong method name; D2 the label is never updated; D3 the dot is drawn with getGraphics() (see 4.2); D4 the click position is used as the top-left corner of the dot, not the centre.

Evidence (real events sent to the real class):

Output of DrawPadHarness (section 1 = the original program)

```
=== 1. Original DrawPadBuggy: 5 mouse clicks sent to the window ===
count = 0      label = "Clicks: 0"
methods declared by the listener class : [mouseclicked]
listener overrides mouseClicked(MouseEvent) : false  -> the inherited empty method of MouseAdapter runs

=== 2. Spelling fixed, dots drawn with getGraphics() ===
after 5 clicks              : count = 5, label = "Clicks: 5", dots on screen = 5
after minimise + restore    : paint() runs again; Frame.paint knows nothing about the dots -> dots on
screen = 0, label still "Clicks: 5"

=== 3. Completed DrawPad (TODO 1 and TODO 2) ===
after 5 clicks              : clickCount() = 5, dots painted from the stored points
after minimise + restore    : paint() draws the stored points again -> still 5 dots, same picture
one more click              : clickCount() = 6
```

```
=== 4. DrawPadBuffered (off-screen image) ===
after 5 clicks              : clickCount() = 5
```

Five MOUSE_CLICKED events were dispatched to the window: count = 0, the label is unchanged, and the listener class declares only mouseclicked – it does not override mouseClicked(MouseEvent).

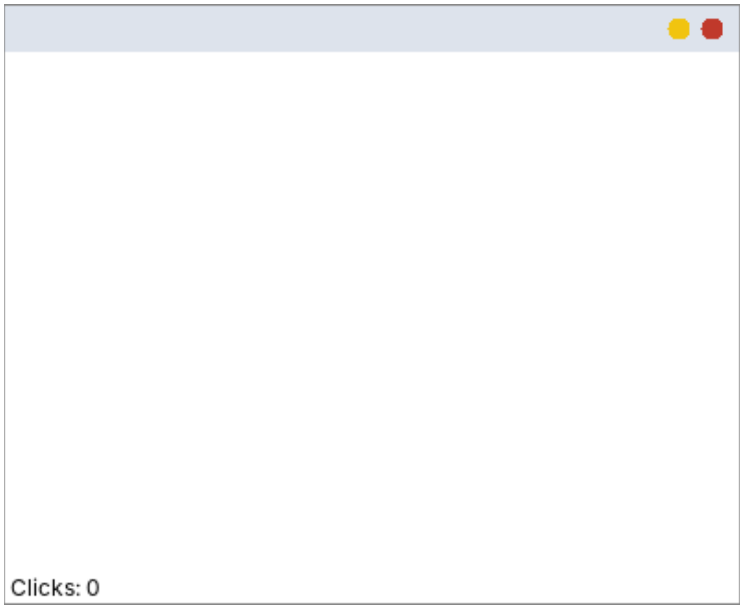


Figure 4.1 – Demonstrates Task 4.1: original program after five clicks – no dots, label still "Clicks: 0"

Task 4.2 – Construct a failing interaction

Reproducible steps. (1) Fix only the spelling (mouseClicked) and add status.setText(...) – this is the class DrawPadStep2 below. (2) Run it and click five times in the window: five dots appear and the label shows "Clicks: 5". (3) Minimise the window and restore it (or drag another window over it, or resize it). (4) **All dots are gone**, while the label still says "Clicks: 5".

Why it happens. getGraphics() returns a graphics context for the *window surface*, so the dot is only pixels on the screen; the program stores nothing. When a window is minimised, covered or resized, the system throws those pixels away. When the window is shown again AWT calls update() and paint(), which redraw the window from the program's data. DrawPadStep2 does not override paint(), so the default paint() draws only the components (the label), and the dots are lost. (The label survives because the label component redraws itself.) The rule: **everything that must stay on screen has to be drawn from stored data inside paint().**

Task04/DrawPadStep2.java - defect D1 and D2 repaired, but still drawing with getGraphics()

```
import java.awt.*;
import java.awt.event.*;

/**
 * Defect 1 repaired (mouseClicked, label updated) but the dots are still drawn straight onto the
screen with getGraphics().
 * This is the version that loses its dots when the window is minimised and restored (Task 4.2).
 */
public class DrawPadStep2 extends Frame {
    int count = 0;
    Label status = new Label("Clicks: 0");

    public DrawPadStep2() {
        add(status, BorderLayout.SOUTH);
        addMouseListener(new MouseAdapter() {
            @Override
            public void mouseClicked(MouseEvent e) {
                count++;
```

```
                status.setText("Clicks: " + count);
                Graphics g = getGraphics();                     // a temporary graphics context
for the window surface
                g.fillOval(e.getX(), e.getY(), 6, 6);           // DEFECT 2: nothing remembers
this dot
                g.dispose();
            }
        });
        setSize(400, 300);
        setVisible(true);
    }
}
```

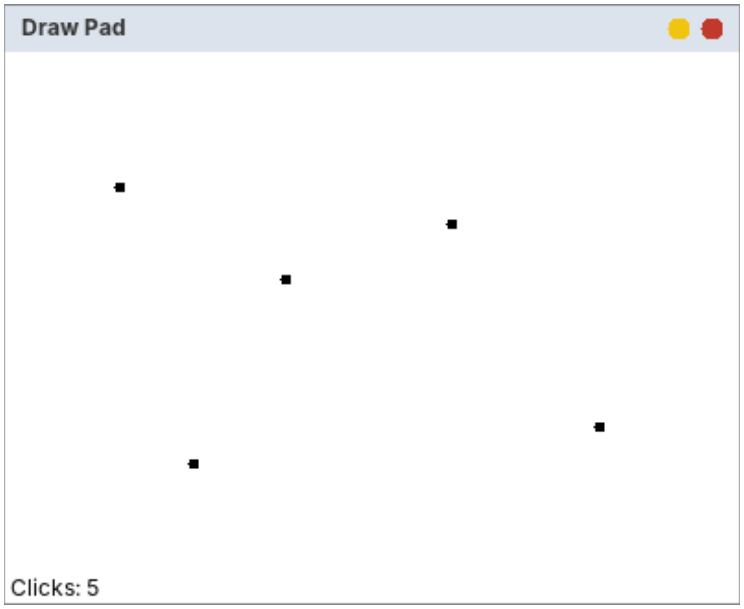


Figure 4.2 – Demonstrates Task 4.2 (step 2): five dots, "Clicks: 5"

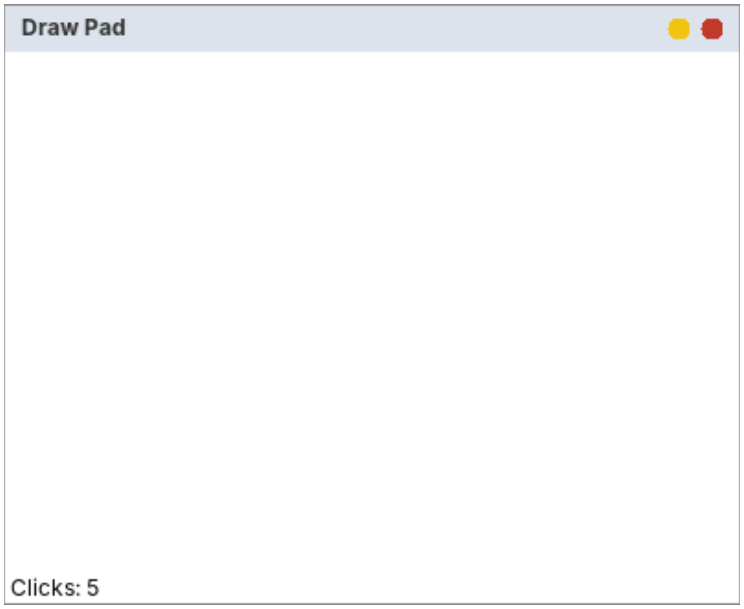


Figure 4.3 – Demonstrates Task 4.2 (step 4): after minimise + restore (imitated by repainting the window) the dots are gone, the label still says 5

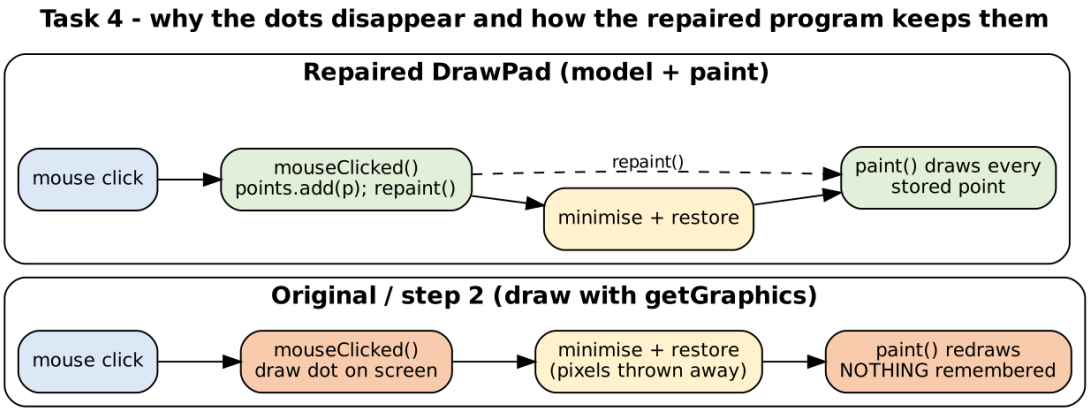


Figure 4.4 – Why the dots disappear and how the repaired design keeps them

Task 4.3 – Repair the program (also: Complete the Code, TODO 1 and TODO 2)

The points are stored in a List<Point>; the click handler stores the point, updates the label and calls repaint(); paint() draws every stored point as a 6 × 6 filled circle centred on the point (x − 3, y − 3). The two TODOs of the Complete-the-Code program are marked in the code. Nothing else in the given program was changed (a small accessor clickCount() and a call to super.paint(g) were added; super.paint(g) lets AWT draw the label as usual).

Task04/DrawPad.java - TODO 1 and TODO 2 completed

```
import java.awt.BorderLayout;
import java.awt.Frame;
import java.awt.Graphics;
import java.awt.Label;
import java.awt.Point;
import java.awt.event.MouseAdapter;
import java.awt.event.MouseEvent;
import java.awt.event.WindowAdapter;
import java.awt.event.WindowEvent;
import java.util.ArrayList;
import java.util.List;

public class DrawPad extends Frame {
    private final List<Point> points = new ArrayList<>();
    private final Label status = new Label("Clicks: 0");

    public DrawPad() {
        super("Draw Pad");
        add(status, BorderLayout.SOUTH);
        addMouseListener(new MouseAdapter() {
            // TODO 1 (completed): override mouseClicked correctly: store the point, update the
label and request a repaint
            @Override
            public void mouseClicked(MouseEvent e) {
                points.add(e.getPoint());
                status.setText("Clicks: " + points.size());
                repaint();
            }
        });
        addWindowListener(new WindowAdapter() {
            @Override
            public void windowClosing(WindowEvent e) {
                dispose();
            }
        });
        setSize(400, 300);
    }
```

```
    @Override
    public void paint(Graphics g) {
        // TODO 2 (completed): draw every stored point as a 6 x 6 filled circle centred on the
point
        super.paint(g);
        for (Point p : points) {
            g.fillOval(p.x - 3, p.y - 3, 6, 6);
        }
    }

    public int clickCount() { return points.size(); }          // small accessor, used only by the
tests

    public static void main(String[] args) {
        new DrawPad().setVisible(true);
    }
}
```

Output of DrawPadHarness (section 3 = the completed DrawPad)

```
=== 1. Original DrawPadBuggy: 5 mouse clicks sent to the window ===
count = 0      label = "Clicks: 0"
methods declared by the listener class : [mouseclicked]
listener overrides mouseClicked(MouseEvent) : false  -> the inherited empty method of MouseAdapter runs

=== 2. Spelling fixed, dots drawn with getGraphics() ===
after 5 clicks              : count = 5, label = "Clicks: 5", dots on screen = 5
after minimise + restore    : paint() runs again; Frame.paint knows nothing about the dots -> dots on
screen = 0, label still "Clicks: 5"

=== 3. Completed DrawPad (TODO 1 and TODO 2) ===
after 5 clicks              : clickCount() = 5, dots painted from the stored points
after minimise + restore    : paint() draws the stored points again -> still 5 dots, same picture
one more click              : clickCount() = 6

=== 4. DrawPadBuffered (off-screen image) ===
after 5 clicks              : clickCount() = 5
```

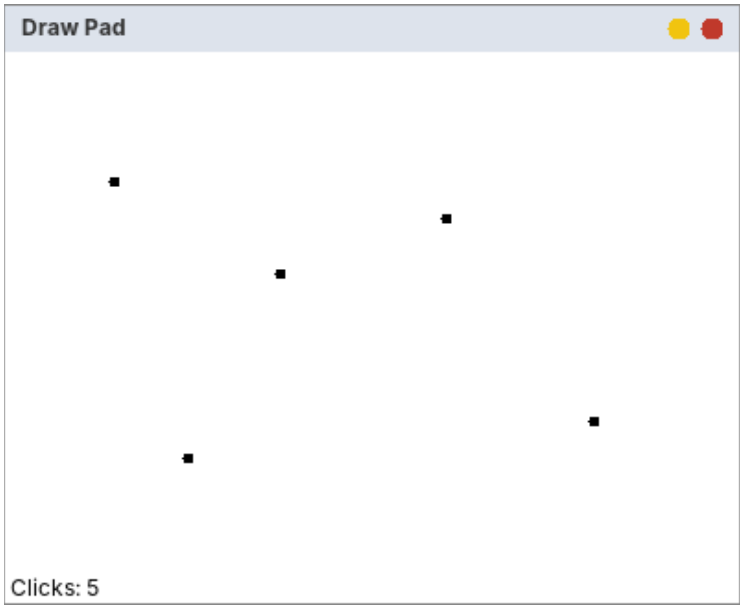


Figure 4.5 – Complete the Code: DrawPad after five clicks (BEFORE minimising)

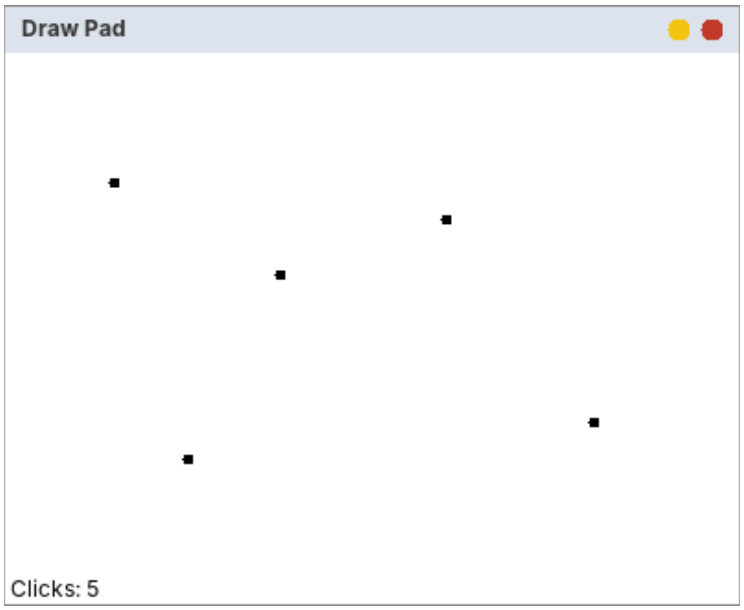


Figure 4.6 – Complete the Code: the same window AFTER minimise + restore: all five dots are still there (painted again from the stored points)

Note on the screenshots. On a real desktop, take the two screenshots yourself with the real window (click five times, minimise, restore). The two pictures above come from the headless test harness: it sends five MouseEvents to the real DrawPad object and asks it to paint itself, once before and once after the imitated "minimise/restore".

Task 4.4 – Prevent recurrence with @Override

The experiment from the Complete-the-Code section was performed with the real compiler. (1) With **@Override removed** and the method misspelled as mouseclicked, the program compiles without any message – the bug is invisible. (2) With **@Override** written above the misspelled method, javac stops with an error. The compiler output:

(1) javac DrawPadNoOverride.java (no @Override, misspelled)

```
(javac printed nothing: the misspelled method without @Override compiles without any warning or error)
```

(2) javac DrawPadWithOverride.java (@Override, misspelled)

```
Task04/errors/DrawPadWithOverride.java:20: error: method does not override or implement a method from a
supertype
            @Override
            ^
1 error
```

@Override tells the compiler "this method is meant to replace a method of the superclass or interface". If no such method exists with that name and parameter list, the compiler reports "method does not override or implement a method from a supertype". The original defect would therefore have been found at compile time, in the first second, instead of by users who see nothing happen. The same applies to wrong parameter types (for example mouseClicked(ActionEvent)). Using an IDE that adds @Override automatically, or a MouseAdapter lambda-free listener with @Override on every handler, removes this whole class of mistakes.

Task04/errors/DrawPadNoOverride.java - experiment (1)

```
import java.awt.BorderLayout;
import java.awt.Frame;
import java.awt.Graphics;
import java.awt.Label;
import java.awt.Point;
import java.awt.event.MouseAdapter;
import java.awt.event.MouseEvent;
import java.util.ArrayList;
```

```
import java.util.List;

// Experiment (Task 4 Complete the Code): @Override REMOVED and the method misspelled as
mouseclicked.
public class DrawPadNoOverride extends Frame {
    private final List<Point> points = new ArrayList<>();
    private final Label status = new Label("Clicks: 0");

    public DrawPadNoOverride() {
        super("Draw Pad");
        add(status, BorderLayout.SOUTH);
        addMouseListener(new MouseAdapter() {
            public void mouseclicked(MouseEvent e) {          // no @Override and small c: it is
just a NEW, unused method
                points.add(e.getPoint());
                status.setText("Clicks: " + points.size());
                repaint();
            }
        });
        setSize(400, 300);
    }

    @Override
    public void paint(Graphics g) {
        for (Point p : points) g.fillOval(p.x - 3, p.y - 3, 6, 6);
    }

    public static void main(String[] args) { new DrawPadNoOverride().setVisible(true); }
}
```

Task04/errors/DrawPadWithOverride.java - experiment (2)

```
import java.awt.BorderLayout;
import java.awt.Frame;
import java.awt.Graphics;
import java.awt.Label;
import java.awt.Point;
import java.awt.event.MouseAdapter;
import java.awt.event.MouseEvent;
import java.util.ArrayList;
import java.util.List;

// Experiment (Task 4 Complete the Code): @Override PUT BACK but the method is still misspelled as
mouseclicked.
public class DrawPadWithOverride extends Frame {
    private final List<Point> points = new ArrayList<>();
    private final Label status = new Label("Clicks: 0");

    public DrawPadWithOverride() {
        super("Draw Pad");
        add(status, BorderLayout.SOUTH);
        addMouseListener(new MouseAdapter() {
            @Override
            public void mouseclicked(MouseEvent e) {          // the compiler now checks the name
                points.add(e.getPoint());
                status.setText("Clicks: " + points.size());
                repaint();
            }
        });
        setSize(400, 300);
    }

    @Override
    public void paint(Graphics g) {
        for (Point p : points) g.fillOval(p.x - 3, p.y - 3, 6, 6);
    }
```

```
    public static void main(String[] args) { new DrawPadWithOverride().setVisible(true); }
}
```

Alternative concept and complexity analysis

The alternative solution. Instead of storing the points and redrawing all of them in paint(), draw each dot **once** onto an off-screen BufferedImage and let paint() copy that image to the window with one drawImage call:

```
// repaired design (stored points)                    // alternative (off-screen image)
points.add(e.getPoint());                             Graphics2D g2 = canvas.createGraphics();
repaint();                                            g2.fillOval(e.getX() - 3, e.getY() - 3, 6,
6); g2.dispose();
paint(g): for (Point p : points)                      paint(g): g.drawImage(canvas, 0, 0, null);
// one copy
              g.fillOval(p.x - 3, p.y - 3, 6, 6);
```

Task04/DrawPadBuffered.java - the alternative

```
import java.awt.*;
import java.awt.event.*;
import java.awt.image.BufferedImage;

/** Alternative concept: each dot is drawn ONCE onto an off-screen image; paint() only copies that
image to the window. */
public class DrawPadBuffered extends Frame {
    private BufferedImage canvas = new BufferedImage(800, 600, BufferedImage.TYPE_INT_ARGB);
    private int clicks = 0;
    private final Label status = new Label("Clicks: 0");

    public DrawPadBuffered() {
        super("Draw Pad (BufferedImage)");
        add(status, BorderLayout.SOUTH);
        addMouseListener(new MouseAdapter() {
            @Override
            public void mouseClicked(MouseEvent e) {
                Graphics2D g = canvas.createGraphics();
                g.setColor(Color.BLACK);
                g.fillOval(e.getX() - 3, e.getY() - 3, 6, 6);     // drawn once, never again
                g.dispose();
                clicks++;
                status.setText("Clicks: " + clicks);
                repaint();
            }
        });
        addWindowListener(new WindowAdapter() {
            @Override public void windowClosing(WindowEvent e) { dispose(); }
        });
        setSize(400, 300);
    }

    @Override
    public void paint(Graphics g) {
        super.paint(g);
        g.drawImage(canvas, 0, 0, null);                          // one image copy, no matter how
many clicks
    }

    public int clickCount() { return clicks; }
    public static void main(String[] args) { new DrawPadBuffered().setVisible(true); }
}
```

	Stored points + paint()	Off-screen BufferedImage
Time per click	O(1) (add a point) + one repaint	O(1) (draw one dot) + one repaint
Time per repaint (n clicks)	O(n): every point is drawn again	O(W × H): one image copy, independent of n
Space (n clicks)	O(n): about 40 bytes per point	O(W × H): 4 bytes per pixel (400 × 300 = 480,000 bytes; 1920 × 1080 = 8.3 MB), independent of n
Other properties	Data survive: can undo, save, redraw at another size	Pixels only: no undo, size fixed (must be enlarged when the window grows)

Measurements (RepaintBenchmark.java; window 400 × 300 painted into an off-screen surface):

n clicks	Redraw n points (µs)	Copy one image (µs)	Points memory (bytes)	Image memory (bytes)
100	323.8	76.6	4,912	480,000
1,000	1407.2	276.8	39,816	480,000
10,000	10179.3	333.0	409,816	480,000
100,000	110829.2	117.2	3,681,744	480,000
1,000,000	843875.2	110.0	38,587,208	480,000

Table 4.1 – Repaint time and memory (from Task04/repaint_results.csv)

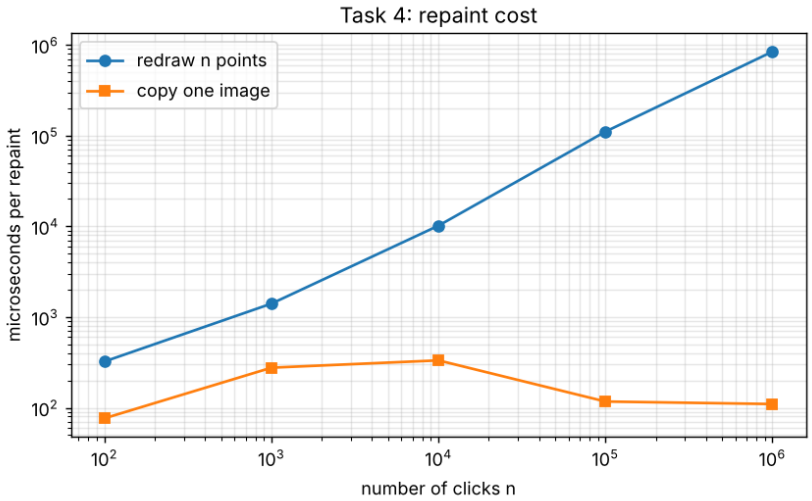


Figure 4.7 – Time per repaint against the number of clicks (logarithmic axes)

Analysis. Redrawing the points costs about 1 µs per point: 1407.2 µs for 1,000 points, 110.8 ms for 100,000 and 843.9 ms (almost a second) for 1,000,000 – linear, O(n). Copying the image costs about 0.1–0.3 ms whatever n is (O(W × H); the small differences are noise). With about 1 µs per point against 0.1–0.3 ms per image copy, the two curves cross somewhere between 100 and 300 points (the n = 100 row is noisy), after which the image is faster; at 100,000 clicks it is about 900 times faster. For memory the points need about 39 bytes each (38,587,208 bytes for 1,000,000), the image always 480,000 bytes for a 400 × 300 window; the break-even is about 13,000 points for 400 × 300, and about 225,000 points for a 1920 × 1080 image (8.3 MB). **Evaluation:** for a drawing pad where a user makes some hundreds of clicks, the stored points are simpler and keep the data (undo, save); for thousands of strokes or freehand drawing the off-screen image is the better design. (These are software-rendered times on an off-screen surface; a real window adds the cost of copying to the screen in both cases.)

Task 5 – Same Screen, Different Event-Handling Styles: Currency Converter

Bloom's level K5 – Evaluate. 7 marks. A currency converter has an amount field, a currency selection, a Convert button and a result label (INR to USD or EUR; rates 0.012 and 0.011). The result is shown with two decimals, or “Invalid amount” when the text is not a number.

Task 5.1 – Four implementations

All four versions have the same window, the same convert() method and the same window-closing code (a WindowAdapter). They differ only in how the Convert button's listener is written.

Task05/ConverterA.java - style (a): the frame itself implements ActionListener

```
import java.awt.FlowLayout;
import java.awt.event.*;
import javax.swing.*;

/** Style (a): the frame implements ActionListener */
public class ConverterA extends JFrame implements ActionListener {
    private final JTextField amount = new JTextField(10);
    private final JComboBox<String> target = new JComboBox<>(new String[]{"USD", "EUR"});
    private final JLabel result = new JLabel(" ");
    private final JButton convertButton = new JButton("Convert");

    private static double rate(String currency) { return currency.equals("USD") ? 0.012 : 0.011; }

    public ConverterA() {
        super("Style (a): the frame implements ActionListener");
        setLayout(new FlowLayout());
        add(new JLabel("INR")); add(amount); add(target); add(convertButton); add(result);
        convertButton.addActionListener(this);
        setDefaultCloseOperation(DO_NOTHING_ON_CLOSE);
        addWindowListener(new WindowAdapter() {
            @Override
            public void windowClosing(WindowEvent e) {
                dispose();
                System.exit(0);
            }
        });
        pack();
    }

    @Override
    public void actionPerformed(ActionEvent e) {
        convert();
    }

    private void convert() {
        try {
            double inr = Double.parseDouble(amount.getText().trim());
            if (Double.isNaN(inr) || Double.isInfinite(inr)) throw new NumberFormatException();
            String currency = (String) target.getSelectedItem();
            String text = String.format("%.2f %s", inr * rate(currency), currency);
            result.setText(text);
        } catch (NumberFormatException ex) {
            result.setText("Invalid amount");
        }
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new ConverterA().setVisible(true));
    }
```

```
}
```

Task05/ConverterB.java - style (b): named inner class

```
import java.awt.FlowLayout;
import java.awt.event.*;
import javax.swing.*;

/** Style (b): named inner class */
public class ConverterB extends JFrame {
    private final JTextField amount = new JTextField(10);
    private final JComboBox<String> target = new JComboBox<>(new String[]{"USD", "EUR"});
    private final JLabel result = new JLabel(" ");
    private final JButton convertButton = new JButton("Convert");

    private static double rate(String currency) { return currency.equals("USD") ? 0.012 : 0.011; }

    public ConverterB() {
        super("Style (b): named inner class");
        setLayout(new FlowLayout());
        add(new JLabel("INR")); add(amount); add(target); add(convertButton); add(result);
        convertButton.addActionListener(new ConvertListener());
        setDefaultCloseOperation(DO_NOTHING_ON_CLOSE);
        addWindowListener(new WindowAdapter() {
            @Override
            public void windowClosing(WindowEvent e) {
                dispose();
                System.exit(0);
            }
        });
        pack();
    }

    private class ConvertListener implements ActionListener {
        @Override
        public void actionPerformed(ActionEvent e) {
            convert();
        }
    }

    private void convert() {
        try {
            double inr = Double.parseDouble(amount.getText().trim());
            if (Double.isNaN(inr) || Double.isInfinite(inr)) throw new NumberFormatException();
            String currency = (String) target.getSelectedItem();
            String text = String.format("%.2f %s", inr * rate(currency), currency);
            result.setText(text);
        } catch (NumberFormatException ex) {
            result.setText("Invalid amount");
        }
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new ConverterB().setVisible(true));
    }
}
```

Task05/ConverterC.java - style (c): anonymous inner class

```
import java.awt.FlowLayout;
import java.awt.event.*;
import javax.swing.*;

/** Style (c): anonymous inner class */
public class ConverterC extends JFrame {
    private final JTextField amount = new JTextField(10);
    private final JComboBox<String> target = new JComboBox<>(new String[]{"USD", "EUR"});
```

```
    private final JLabel result = new JLabel(" ");
    private final JButton convertButton = new JButton("Convert");

    private static double rate(String currency) { return currency.equals("USD") ? 0.012 : 0.011; }

    public ConverterC() {
        super("Style (c): anonymous inner class");
        setLayout(new FlowLayout());
        add(new JLabel("INR")); add(amount); add(target); add(convertButton); add(result);
        convertButton.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                convert();
            }
        });
        setDefaultCloseOperation(DO_NOTHING_ON_CLOSE);
        addWindowListener(new WindowAdapter() {
            @Override
            public void windowClosing(WindowEvent e) {
                dispose();
                System.exit(0);
            }
        });
        pack();
    }

    private void convert() {
        try {
            double inr = Double.parseDouble(amount.getText().trim());
            if (Double.isNaN(inr) || Double.isInfinite(inr)) throw new NumberFormatException();
            String currency = (String) target.getSelectedItem();
            String text = String.format("%.2f %s", inr * rate(currency), currency);
            result.setText(text);
        } catch (NumberFormatException ex) {
            result.setText("Invalid amount");
        }
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new ConverterC().setVisible(true));
    }
}
```

Task05/ConverterD.java - style (d): lambda expression

```
import java.awt.FlowLayout;
import java.awt.event.*;
import javax.swing.*;

/** Style (d): lambda expression */
public class ConverterD extends JFrame {
    private final JTextField amount = new JTextField(10);
    private final JComboBox<String> target = new JComboBox<>(new String[]{"USD", "EUR"});
    private final JLabel result = new JLabel(" ");
    private final JButton convertButton = new JButton("Convert");

    private static double rate(String currency) { return currency.equals("USD") ? 0.012 : 0.011; }

    public ConverterD() {
        super("Style (d): lambda expression");
        setLayout(new FlowLayout());
        add(new JLabel("INR")); add(amount); add(target); add(convertButton); add(result);
        convertButton.addActionListener(e -> convert());
        setDefaultCloseOperation(DO_NOTHING_ON_CLOSE);
        addWindowListener(new WindowAdapter() {
            @Override
```

```
            public void windowClosing(WindowEvent e) {
                dispose();
                System.exit(0);
            }
        });
        pack();
    }

    private void convert() {
        try {
            double inr = Double.parseDouble(amount.getText().trim());
            if (Double.isNaN(inr) || Double.isInfinite(inr)) throw new NumberFormatException();
            String currency = (String) target.getSelectedItem();
            String text = String.format("%.2f %s", inr * rate(currency), currency);
            result.setText(text);
        } catch (NumberFormatException ex) {
            result.setText("Invalid amount");
        }
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new ConverterD().setVisible(true));
    }
}
```

Results (from the harness; amount 5000 INR to EUR, and the text “abc”). The first four rows of the table are these versions:

Output of ConverterHarness (rows ConverterA … ConverterD = Task 5.1)

```
=== Task 5: every version, amount 5000 INR -> EUR (index 1) and invalid text 'abc' ===
class        button click Enter key      invalid 'abc'  dialog shown?  dialog text
ConverterA   55.00 EUR    (nothing)      Invalid amount no             -
ConverterB   55.00 EUR    (nothing)      Invalid amount no             -
ConverterC   55.00 EUR    (nothing)      Invalid amount no             -
ConverterD   55.00 EUR    (nothing)      Invalid amount no             -
ConverterA2  55.00 EUR    55.00 EUR      Invalid amount yes            55.00 EUR
ConverterB2  55.00 EUR    55.00 EUR      Invalid amount yes            55.00 EUR
ConverterC2  55.00 EUR    55.00 EUR      Invalid amount yes            55.00 EUR
ConverterD2  55.00 EUR    55.00 EUR      Invalid amount yes            55.00 EUR
ConverterE   55.00 EUR    55.00 EUR      Invalid amount yes            55.00 EUR
ConverterF   55.00 EUR    55.00 EUR      Invalid amount yes            55.00 EUR

=== Complete the Code: Converter.java (lambda, one listener for button and Enter) ===
2500 INR -> USD, button click : 30.00 USD
5000 INR -> EUR, Enter key    : 55.00 EUR
'12abc'                       : Invalid amount
empty field                   : Invalid amount
listeners on the button / on the text field : 1 / 1   same object? true

=== Listener objects registered per version (distinct objects) ===
ConverterA2  registrations: 2   distinct listener objects: 1
ConverterB2  registrations: 2   distinct listener objects: 2
ConverterC2  registrations: 2   distinct listener objects: 1
ConverterD2  registrations: 2   distinct listener objects: 1
ConverterE   registrations: 2   distinct listener objects: 2
ConverterF   registrations: 3   distinct listener objects: 1
ConverterF menu item click (same Action) -> result label: 1.20 USD; Action shared by button and menu
item: true
```

Task 5.2 – Change the requirement (Enter key + result dialog)

The converter must now also convert when the user presses Enter in the amount field, and show the result in a dialog box. In the output table above, the rows ConverterA … D (before) show “(nothing)” for the Enter key and “no” dialog; the rows ConverterA2 … D2 (after) convert on Enter and show the dialog (“dialog text 55.00 EUR”).

What had to change in each version (measured with a line diff between the base file and the modified file):

Task05/change_effort.txt

```
Task 5.2 - change effort: base version -> version with Enter key + result dialog
style  LOC before  LOC after  lines added  lines removed  lines changed in total
  A         44        47          3           0              3
  B         46        49          3           0              3
  C         45        49          6           2              8
  D         40        44          5           1              6
```

Style	Changes needed for Enter key	Changes needed for the dialog	Total lines changed
(a) frame implements ActionListener	1 line: amount.addActionListener(this)	2 lines in convert() (result and error)	3 (3 added, 0 removed)
(b) named inner class	1 line: amount.addActionListener(new ConvertListener())	2 lines in convert()	3 (3 added, 0 removed)
(c) anonymous inner class	The anonymous object cannot be reused as written: it had to be stored in a variable first, then registered twice (the registration code was rewritten)	2 lines in convert()	8 (6 added, 2 removed)
(d) lambda	Store the lambda in a variable, register it twice (a lambda also has no name)	2 lines in convert()	6 (5 added, 1 removed)

Table 5.1 – Change effort (the "lines" numbers are from the diff above)

The modified versions

Task05/ConverterA2.java

```
import java.awt.FlowLayout;
import java.awt.event.*;
import javax.swing.*;

/** Style (a): the frame implements ActionListener */
public class ConverterA2 extends JFrame implements ActionListener {
    private final JTextField amount = new JTextField(10);
    private final JComboBox<String> target = new JComboBox<>(new String[]{"USD", "EUR"});
    private final JLabel result = new JLabel(" ");
    private final JButton convertButton = new JButton("Convert");

    private static double rate(String currency) { return currency.equals("USD") ? 0.012 : 0.011; }

    public ConverterA2() {
        super("Style (a): the frame implements ActionListener");
        setLayout(new FlowLayout());
        add(new JLabel("INR")); add(amount); add(target); add(convertButton); add(result);
        convertButton.addActionListener(this);
        amount.addActionListener(this);
        setDefaultCloseOperation(DO_NOTHING_ON_CLOSE);
        addWindowListener(new WindowAdapter() {
            @Override
            public void windowClosing(WindowEvent e) {
                dispose();
                System.exit(0);
            }
        });
        pack();
    }

    @Override
    public void actionPerformed(ActionEvent e) {
        convert();
    }

    private void convert() {
```

```
        try {
            double inr = Double.parseDouble(amount.getText().trim());
            if (Double.isNaN(inr) || Double.isInfinite(inr)) throw new NumberFormatException();
            String currency = (String) target.getSelectedItem();
            String text = String.format("%.2f %s", inr * rate(currency), currency);
            result.setText(text);
            JOptionPane.showMessageDialog(this, text, "Result", JOptionPane.INFORMATION_MESSAGE);
        } catch (NumberFormatException ex) {
            result.setText("Invalid amount");
            JOptionPane.showMessageDialog(this, "Invalid amount", "Result",
JOptionPane.ERROR_MESSAGE);
        }
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new ConverterA2().setVisible(true));
    }
}
```

Task05/ConverterB2.java

```
import java.awt.FlowLayout;
import java.awt.event.*;
import javax.swing.*;

/** Style (b): named inner class */
public class ConverterB2 extends JFrame {
    private final JTextField amount = new JTextField(10);
    private final JComboBox<String> target = new JComboBox<>(new String[]{"USD", "EUR"});
    private final JLabel result = new JLabel(" ");
    private final JButton convertButton = new JButton("Convert");

    private static double rate(String currency) { return currency.equals("USD") ? 0.012 : 0.011; }

    public ConverterB2() {
        super("Style (b): named inner class");
        setLayout(new FlowLayout());
        add(new JLabel("INR")); add(amount); add(target); add(convertButton); add(result);
        convertButton.addActionListener(new ConvertListener());
        amount.addActionListener(new ConvertListener());
        setDefaultCloseOperation(DO_NOTHING_ON_CLOSE);
        addWindowListener(new WindowAdapter() {
            @Override
            public void windowClosing(WindowEvent e) {
                dispose();
                System.exit(0);
            }
        });
        pack();
    }

    private class ConvertListener implements ActionListener {
        @Override
        public void actionPerformed(ActionEvent e) {
            convert();
        }
    }

    private void convert() {
        try {
            double inr = Double.parseDouble(amount.getText().trim());
            if (Double.isNaN(inr) || Double.isInfinite(inr)) throw new NumberFormatException();
            String currency = (String) target.getSelectedItem();
            String text = String.format("%.2f %s", inr * rate(currency), currency);
            result.setText(text);
            JOptionPane.showMessageDialog(this, text, "Result", JOptionPane.INFORMATION_MESSAGE);
```

```
        } catch (NumberFormatException ex) {
            result.setText("Invalid amount");
            JOptionPane.showMessageDialog(this, "Invalid amount", "Result",
JOptionPane.ERROR_MESSAGE);
        }
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new ConverterB2().setVisible(true));
    }
}
```

Task05/ConverterC2.java

```
import java.awt.FlowLayout;
import java.awt.event.*;
import javax.swing.*;

/** Style (c): anonymous inner class */
public class ConverterC2 extends JFrame {
    private final JTextField amount = new JTextField(10);
    private final JComboBox<String> target = new JComboBox<>(new String[]{"USD", "EUR"});
    private final JLabel result = new JLabel(" ");
    private final JButton convertButton = new JButton("Convert");

    private static double rate(String currency) { return currency.equals("USD") ? 0.012 : 0.011; }

    public ConverterC2() {
        super("Style (c): anonymous inner class");
        setLayout(new FlowLayout());
        add(new JLabel("INR")); add(amount); add(target); add(convertButton); add(result);
        ActionListener convertListener = new ActionListener() {   // must be stored in a variable
to be reused
            @Override
            public void actionPerformed(ActionEvent e) {
                convert();
            }
        };
        convertButton.addActionListener(convertListener);
        amount.addActionListener(convertListener);
        setDefaultCloseOperation(DO_NOTHING_ON_CLOSE);
        addWindowListener(new WindowAdapter() {
            @Override
            public void windowClosing(WindowEvent e) {
                dispose();
                System.exit(0);
            }
        });
        pack();
    }

    private void convert() {
        try {
            double inr = Double.parseDouble(amount.getText().trim());
            if (Double.isNaN(inr) || Double.isInfinite(inr)) throw new NumberFormatException();
            String currency = (String) target.getSelectedItem();
            String text = String.format("%.2f %s", inr * rate(currency), currency);
            result.setText(text);
            JOptionPane.showMessageDialog(this, text, "Result", JOptionPane.INFORMATION_MESSAGE);
        } catch (NumberFormatException ex) {
            result.setText("Invalid amount");
            JOptionPane.showMessageDialog(this, "Invalid amount", "Result",
JOptionPane.ERROR_MESSAGE);
        }
    }
```

```
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new ConverterC2().setVisible(true));
    }
}
```

Task05/ConverterD2.java

```
import java.awt.FlowLayout;
import java.awt.event.*;
import javax.swing.*;

/** Style (d): lambda expression */
public class ConverterD2 extends JFrame {
    private final JTextField amount = new JTextField(10);
    private final JComboBox<String> target = new JComboBox<>(new String[]{"USD", "EUR"});
    private final JLabel result = new JLabel(" ");
    private final JButton convertButton = new JButton("Convert");

    private static double rate(String currency) { return currency.equals("USD") ? 0.012 : 0.011; }

    public ConverterD2() {
        super("Style (d): lambda expression");
        setLayout(new FlowLayout());
        add(new JLabel("INR")); add(amount); add(target); add(convertButton); add(result);
        ActionListener convertListener = e -> convert();
        convertButton.addActionListener(convertListener);
        amount.addActionListener(convertListener);
        setDefaultCloseOperation(DO_NOTHING_ON_CLOSE);
        addWindowListener(new WindowAdapter() {
            @Override
            public void windowClosing(WindowEvent e) {
                dispose();
                System.exit(0);
            }
        });
        pack();
    }

    private void convert() {
        try {
            double inr = Double.parseDouble(amount.getText().trim());
            if (Double.isNaN(inr) || Double.isInfinite(inr)) throw new NumberFormatException();
            String currency = (String) target.getSelectedItem();
            String text = String.format("%.2f %s", inr * rate(currency), currency);
            result.setText(text);
            JOptionPane.showMessageDialog(this, text, "Result", JOptionPane.INFORMATION_MESSAGE);
        } catch (NumberFormatException ex) {
            result.setText("Invalid amount");
            JOptionPane.showMessageDialog(this, "Invalid amount", "Result",
JOptionPane.ERROR_MESSAGE);
        }
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new ConverterD2().setVisible(true));
    }
}
```

Task 5.3 – Compare the styles

Criterion	(a) frame implements	(b) named inner class	(c) anonymous class	(d) lambda
Readability	The listener code is far from the button (the whole frame is the listener); with many buttons actionPerformed becomes an if-else chain	Handler has a name and a place; more lines (boilerplate)	Handler is next to the button that uses it; verbose (6 lines to call one method)	Shortest; handler next to the button; best when the body is one call

Criterion	(a) frame implements	(b) named inner class	(c) anonymous class	(d) lambda
Access to the frame's fields	Direct (it is the frame)	Direct: an inner class can use the outer fields	Direct (captures the frame)	Direct (captures this)
Reuse of the handler	One handler for all sources: need e.getSource() or the command to tell them apart	Best: a named class can be instantiated for several components or reused	Poor: no name; must be kept in a variable to be reused	Poor-fair: keep it in a variable (an ActionListener) or call a shared method
Ease of change	Every new trigger changes the shared method	Add a registration; add a class for a new behaviour	Duplicates code or needs refactoring (measured: 8 lines for the changed requirement)	Small edits; refactoring needed for reuse (6 lines)

Table 5.2 – Comparison of the four styles

Which style for a large application? I would not let the frame implement the listener (a), because with 30 buttons one actionPerformed() becomes a long if-else chain (as in Task 1). For a large application I would use **small lambdas or method references that only call a method of a controller/service class** for one-trigger actions (readable, no boilerplate, handler next to the component), and **a shared Action object** (alternative f below) for any command that has several triggers (button, menu item, keyboard shortcut). Named inner classes are justified when a handler is big, has its own state or is reused. This keeps the interface separate from the logic, which is the main lesson of the assignment.

Complete the Code – Converter.java (style d)

TODO 1 registers **one** lambda for both the Convert button and the Enter key of the amount field (the same listener object is attached to both: the harness reports “same object? true”). TODO 2 parses the amount and shows the value with two decimals, or “Invalid amount”.

Task05/Converter.java - TODO 1 and TODO 2 completed

```
import java.awt.FlowLayout;
import java.awt.event.ActionListener;
import javax.swing.JButton;
import javax.swing.JComboBox;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JTextField;
import javax.swing.SwingUtilities;

public class Converter extends JFrame {
    private final JTextField amount = new JTextField(10);
    private final JComboBox<String> target = new JComboBox<>(new String[]{"USD", "EUR"});
    private final JLabel result = new JLabel(" ");

    private static double rate(String currency) {
        return currency.equals("USD") ? 0.012 : 0.011;
    }

    public Converter() {
        super("Currency Converter");
        setLayout(new FlowLayout());
        JButton convertButton = new JButton("Convert");
        add(new JLabel("INR"));
        add(amount);
        add(target);
        add(convertButton);
        add(result);

        // TODO 1 (completed): ONE lambda listener for both the Convert button and the Enter key
in the amount field
        ActionListener listener = e -> convert();
        convertButton.addActionListener(listener);
        amount.addActionListener(listener);
```

```
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        pack();
    }

    private void convert() {
        // TODO 2 (completed): parse the amount and show the converted value with 2 decimals, or
"Invalid amount"
        try {
            double inr = Double.parseDouble(amount.getText().trim());
            String currency = (String) target.getSelectedItem();
            result.setText(String.format("%.2f %s", inr * rate(currency), currency));
        } catch (NumberFormatException ex) {
            result.setText("Invalid amount");
        }
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new Converter().setVisible(true));
    }
}
```



Figure 5.1 – Complete the Code: a valid conversion (2500 INR → 30.00 USD, Convert button clicked)



Figure 5.2 – Complete the Code: an invalid amount ("12abc") shows "Invalid amount"

Results printed by the harness for this program: 2500 INR → 30.00 USD (button); 5000 INR → 55.00 EUR (Enter key); "12abc" → Invalid amount; empty field → Invalid amount.

Alternative concept and complexity analysis

The alternative solutions. (e) A **method reference**: convertButton.addActionListener(this::onConvert), where onConvert(ActionEvent e) is an ordinary method. (f) **One shared Action object** (AbstractAction) holding the name, the keyboard shortcut and the code, used by the button, a menu item and the Enter key:

```
// (e) method reference
convertButton.addActionListener(this::onConvert);
amount.addActionListener(this::onConvert);

// (f) one shared Action for three triggers
Action convertAction = new ConvertAction();            // name "Convert", accelerator Ctrl+Enter,
the code
convertButton.setAction(convertAction);                // trigger 1: button
amount.addActionListener(convertAction);               // trigger 2: Enter in the field
menu.add(new JMenuItem(convertAction));                // trigger 3: menu item (shows Ctrl+Enter)
```

Task05/ConverterE.java - alternative (e): method reference

```
import java.awt.FlowLayout;
import java.awt.event.*;
import javax.swing.*;
```

```
/** Alternative (e): method reference this::onConvert (Enter key and dialog included). */
public class ConverterE extends JFrame {
    private final JTextField amount = new JTextField(10);
    private final JComboBox<String> target = new JComboBox<>(new String[]{"USD", "EUR"});
    private final JLabel result = new JLabel(" ");
    private final JButton convertButton = new JButton("Convert");

    private static double rate(String currency) { return currency.equals("USD") ? 0.012 : 0.011; }

    public ConverterE() {
        super("Style (e): method reference");
        setLayout(new FlowLayout());
        add(new JLabel("INR")); add(amount); add(target); add(convertButton); add(result);
        convertButton.addActionListener(this::onConvert);
        amount.addActionListener(this::onConvert);
        setDefaultCloseOperation(DO_NOTHING_ON_CLOSE);
        addWindowListener(new WindowAdapter() {
            @Override
            public void windowClosing(WindowEvent e) {
                dispose();
                System.exit(0);
            }
        });
        pack();
    }

    private void onConvert(ActionEvent e) {                 // a method reference needs the
ActionEvent parameter
        try {
            double inr = Double.parseDouble(amount.getText().trim());
            if (Double.isNaN(inr) || Double.isInfinite(inr)) throw new NumberFormatException();
            String currency = (String) target.getSelectedItem();
            String text = String.format("%.2f %s", inr * rate(currency), currency);
            result.setText(text);
            JOptionPane.showMessageDialog(this, text, "Result", JOptionPane.INFORMATION_MESSAGE);
        } catch (NumberFormatException ex) {
            result.setText("Invalid amount");
            JOptionPane.showMessageDialog(this, "Invalid amount", "Result",
JOptionPane.ERROR_MESSAGE);
        }
    }

    public static void main(String[] args) { SwingUtilities.invokeLater(() -> new
ConverterE().setVisible(true)); }
}
```

Task05/ConverterF.java - alternative (f): shared Action, menu item, shortcut

```
import java.awt.FlowLayout;
import java.awt.event.*;
import javax.swing.*;

/** Alternative (f): ONE shared Action used by the button, a menu item and the Enter key. A new
trigger = one line. */
public class ConverterF extends JFrame {
    private final JTextField amount = new JTextField(10);
    private final JComboBox<String> target = new JComboBox<>(new String[]{"USD", "EUR"});
    private final JLabel result = new JLabel(" ");
    private final JButton convertButton = new JButton("Convert");

    private static double rate(String currency) { return currency.equals("USD") ? 0.012 : 0.011; }
    private final JMenuItem convertMenuItem;

    /** The Action carries the name, the keyboard shortcut and the code: every trigger shares
them. */
    private class ConvertAction extends AbstractAction {
```

```
        ConvertAction() {
            super("Convert");
            putValue(ACCELERATOR_KEY, KeyStroke.getKeyStroke(KeyEvent.VK_ENTER,
InputEvent.CTRL_DOWN_MASK));
        }
        @Override
        public void actionPerformed(ActionEvent e) {
            try {
                double inr = Double.parseDouble(amount.getText().trim());
                if (Double.isNaN(inr) || Double.isInfinite(inr)) throw new
NumberFormatException();
                String currency = (String) target.getSelectedItem();
                String text = String.format("%.2f %s", inr * rate(currency), currency);
                result.setText(text);
                JOptionPane.showMessageDialog(ConverterF.this, text, "Result",
JOptionPane.INFORMATION_MESSAGE);
            } catch (NumberFormatException ex) {
                result.setText("Invalid amount");
                JOptionPane.showMessageDialog(ConverterF.this, "Invalid amount", "Result",
JOptionPane.ERROR_MESSAGE);
            }
        }
    }

    public ConverterF() {
        super("Style (f): shared Action");
        setLayout(new FlowLayout());
        Action convertAction = new ConvertAction();
        convertButton.setAction(convertAction);                         // trigger 1: button
        amount.addActionListener(convertAction);                        // trigger 2: Enter key
in the field
        JMenu menu = new JMenu("Tools");
        convertMenuItem = new JMenuItem(convertAction);                 // trigger 3: menu item
(with Ctrl+Enter shown)
        menu.add(convertMenuItem);
        JMenuBar bar = new JMenuBar();
        bar.add(menu);
        setJMenuBar(bar);
        add(new JLabel("INR")); add(amount); add(target); add(convertButton); add(result);
        setDefaultCloseOperation(DO_NOTHING_ON_CLOSE);
        addWindowListener(new WindowAdapter() {
            @Override
            public void windowClosing(WindowEvent e) {
                dispose();
                System.exit(0);
            }
        });
        pack();
    }

    public static void main(String[] args) { SwingUtilities.invokeLater(() -> new
ConverterF().setVisible(true)); }
}
```

The harness output (section “Listener objects registered per version”) gives the number of listener objects. Combined with the change-effort table, the comparison is:

Version (with Enter key + dialog)	Registrations on triggers	Distinct listener objects	Work to add one more trigger (for example a menu item)
(a) frame implements	2	1 (the frame itself)	1 line: item.addActionListener(this)
(b) named inner class	2	2 (new ConvertListener() each time) – or 1 if the instance is stored	1 line + a new object (or store the instance)
(c) anonymous class	2	1 (after storing it in a variable)	1 line, but only after the refactoring to a variable

Version (with Enter key + dialog)	Registrations on triggers	Distinct listener objects	Work to add one more trigger (for example a menu item)
(d) lambda	2	1 (after storing it in a variable)	1 line, after the same refactoring
(e) method reference	2	2 (a method reference is evaluated, so a new object, for every registration)	1 line: item.addActionListener(this::onConvert)
(f) shared Action	3 (button, field, menu item)	1 (the Action is shared)	1 line: new JMenuItem(action) – the name, shortcut and code come with it

Table 5.3 – Listener objects (counted by the harness) and effort for a new trigger

Analysis. Space: a listener object is small (about 16–24 bytes), so the difference between one and two objects is irrelevant for memory; the Action version also keeps a few property-change registrations (a button that uses an Action listens to changes of the Action's text and enabled state), but these are also tiny. Time to dispatch: O(1) in every style. The real difference is **work and risk when requirements change**: the shared Action is the only version where a new trigger needs a single line and automatically gets the same name, shortcut and enabled/disabled state (disabling the Action disables the button and the menu item together). That is why it is the best choice when one command has several triggers.

Task 6 – Campus Event Registration Desktop Application (Swing + JDBC + MySQL)

Bloom's level K5 – Evaluate. 13 marks. The cultural committee needs a desktop application in which students register for campus events and the committee views, filters and exports the registrations. The application is built in the progression required by the assignment: Interpret the requirement → Identify components and events → Design the layout → Design the handlers → Implement → Trace → Challenge → Refine.

Stage 1 – Interpret the Problem (2 marks)

Users: (1) a **student** (or the committee desk operator on the student's behalf) who fills the registration form; (2) a **committee member** who looks at the registrations, filters them by event and exports them. **Data:** students (register number, name, e-mail), events (title, venue, date) and registrations (which student registered for which event, and when).

Each requirement below is a **testable statement**; the Test column says where in this report it is demonstrated (the final figure captions refer to these identifiers).

ID	Testable requirement	Demonstrated by
REQ-01	The register number must be exactly 9 digits; anything else is rejected with a message.	Test T1 – ServiceDemo "register number too short / with letters"
REQ-02	The name may contain only letters, spaces and dots (2–60 characters).	T2 – "name with digits", "SQL text in the name"
REQ-03	The e-mail address must have a valid form.	T3 – "e-mail without @"; Figure 6.8
REQ-04	All fields are required and an event must be selected.	T4 – "all fields empty", "no event selected"; Figure 6.7
REQ-05	A student can register only once for the same event (duplicate is rejected, nothing is stored twice).	T5 – "same student, same event again", direct insert blocked by UNIQUE key; SQL check; Figure 6.10
REQ-06	The same student can register for different events.	T6 – "same student, another event"; SQL check
REQ-07	Registrations are stored in a MySQL database through JDBC; the student row and the registration row are saved together or not at all.	Code: JdbcRegistrationDao (PreparedStatement, transaction); SQL script; SQL check (SQLite stand-in)
REQ-08	All registrations are shown in a table, one page at a time, and can be filtered by event.	T8 – Figures 6.11 to 6.13
REQ-09	The selection can be exported to a CSV file with the same rows.	T9 – ServiceDemo "Export"
REQ-10	A menu bar offers File (Export, Exit), View (screens, refresh) and Help (About).	T10 – menu output and Figure 6.6
REQ-11	The window never freezes while the database is working.	T11 – responsiveness measurement (slowest answer 2.00 ms during an 855 ms load)
REQ-12	Closing the window while work is running asks for confirmation.	Code: MainFrame.requestExit() (same pattern as Tasks 1 and 2)

Table 6.1 – Requirements and tests

Stage 2 – Propose the Screen and Event Flow (2 marks)

Screen flow. The application has two screens inside one window (a CardLayout) and three dialogs.

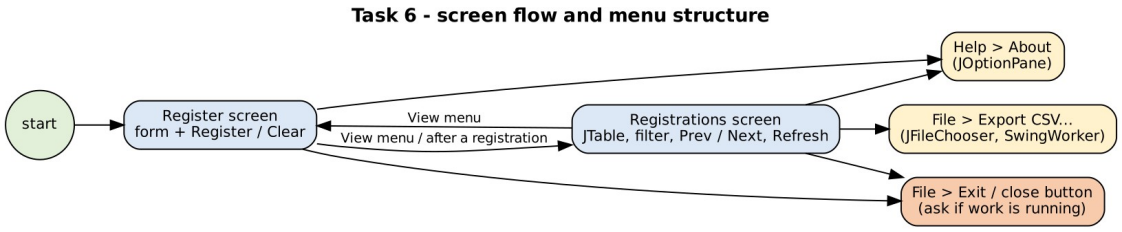


Figure 6.1 – Screen flow and menu structure

```
Screen 1 - Register                                  Screen 2 - Registrations
+--------------------------------------------+
+--------------------------------------------+
| File   View   Help                         |      | File   View   Help
|
+--------------------------------------------+
+--------------------------------------------+
| Register for an event                      |      | Show event: [All events v]  [Refresh]
|
| Register number: [_____________]           |      | +-----+----------+---------+-------+------
+|
| Student name:    [_____________]           |      | |RegID|Register N|Student  |Event  |Date
||
| E-mail:          [_____________]           |      | |  1  |192311001 |Arun K.  |Tech.. |...
||
| Event:           [-- select -- v]          |      | |  2  |192311002 |Divya S  |Cultu..|...
||
|        [ Register ]   [ Clear ]            |      | +-----+----------+---------+-------+------
+|
| status message (green = ok, red = error)   |      |      [Prev]  Page 1 of 3  [Next]
|
+--------------------------------------------+      | 13 registration(s) found
|

+--------------------------------------------+
Menu:  File > Export CSV... (Ctrl+E), Exit | View > Register screen, Registrations screen, Refresh
table | Help > About
```

Component	Swing class	Event → listener → action
Register number, name, e-mail	JTextField	typed text is read when Register is clicked
Event	JComboBox<CampusEvent>	item chosen is read when Register is clicked
Register / Clear	JButton	ActionEvent → ActionListener (lambda) → register() in a SwingWorker / clear()
Filter by event	JComboBox	ActionEvent → load(1) in a SwingWorker
Prev / Next / Refresh	JButton	ActionEvent → load(page − 1 / page + 1 / page)
Registrations table	JTable + DefaultTableModel (not editable) in a JScrollPane	filled in done() of the SwingWorker (event-dispatch thread)
Menu bar	JMenuBar, JMenu, JMenuItem (with Ctrl+E accelerator)	ActionEvent → showScreen / exportCsv / requestExit / About dialog
Export, About, Exit dialogs	JFileChooser, JOptionPane	modal dialogs started from menu actions
Main window	JFrame + WindowAdapter	windowClosing → requestExit()

Stage 3 – Identify the Critical Design Decisions (2 marks)

Decision	What I decided	Why (alternative rejected)
Where is input validated?	In RegistrationService (one place). The window only passes the text and shows the message.	Validation in the listener cannot be tested without a window and is repeated for every screen.

Decision	What I decided	Why (alternative rejected)
Where is “already registered” enforced?	Twice: the service asks the DAO first (friendly message), and the database has UNIQUE (reg_no, event_id) as the final guard; a violation (MySQL error 1062) becomes DuplicateRegistrationException.	The check alone is not safe when two desks register the same student at the same moment (a race between “check” and “insert”); only the database constraint is.
Where does the SQL live?	Only in JdbcRegistrationDao, behind the RegistrationDao interface.	SQL in listeners ties the window to MySQL and cannot be tested; the interface also allows the in-memory DAO used for tests.
How is the database accessed?	PreparedStatement for every statement; try-with-resources closes ResultSet, Statement and Connection; a transaction saves student + registration together (commit / rollback).	Concatenated SQL allows injection; unclosed connections leak (Task 1, D6 and D7).
Which thread does the database work?	A SwingWorker: doInBackground() calls the service; done() (event-dispatch thread) updates the window.	JDBC on the event thread freezes the window (Task 1, D5).
How are errors shown?	Expected problems (invalid input, duplicate) come back as a ServiceResult message; unexpected database problems are caught as DataAccessException and shown as “Database problem: …”.	A stack trace on the console is invisible to the user.
How is a big table shown?	Paging: SQL LIMIT/OFFSET, one page (10 rows; 5 in the demo) at a time, plus Prev/Next.	Loading all rows blocks the window and uses memory proportional to the table size (measured below).
How are the two screens switched?	CardLayout in one JFrame, menu items View > Register / Registrations.	Two windows would need more window management; the screens are used one at a time.

Table 6.2 – Design decisions

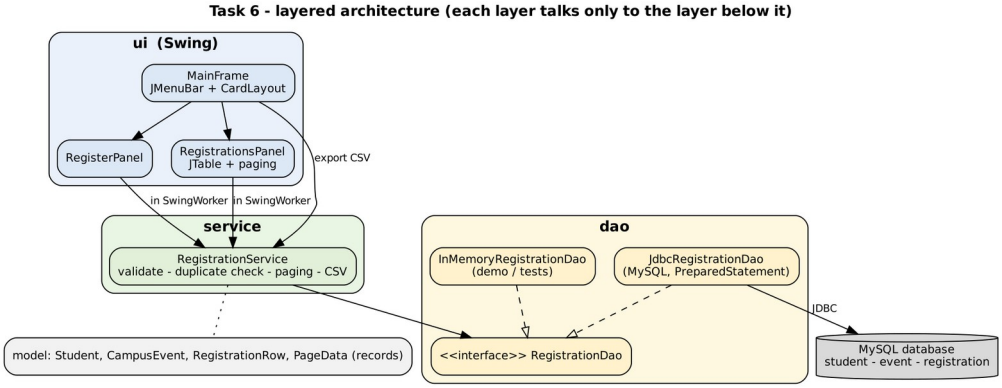


Figure 6.2 – Layered architecture: ui → service → dao → MySQL

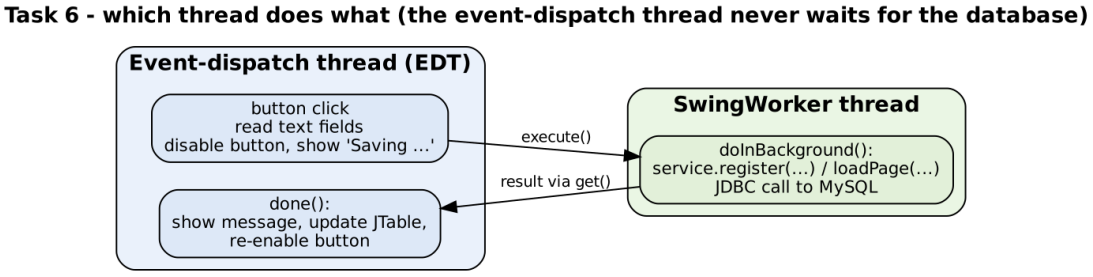


Figure 6.3 – Which thread does what

Stage 4 – Design the Classes and Handlers (4 marks)

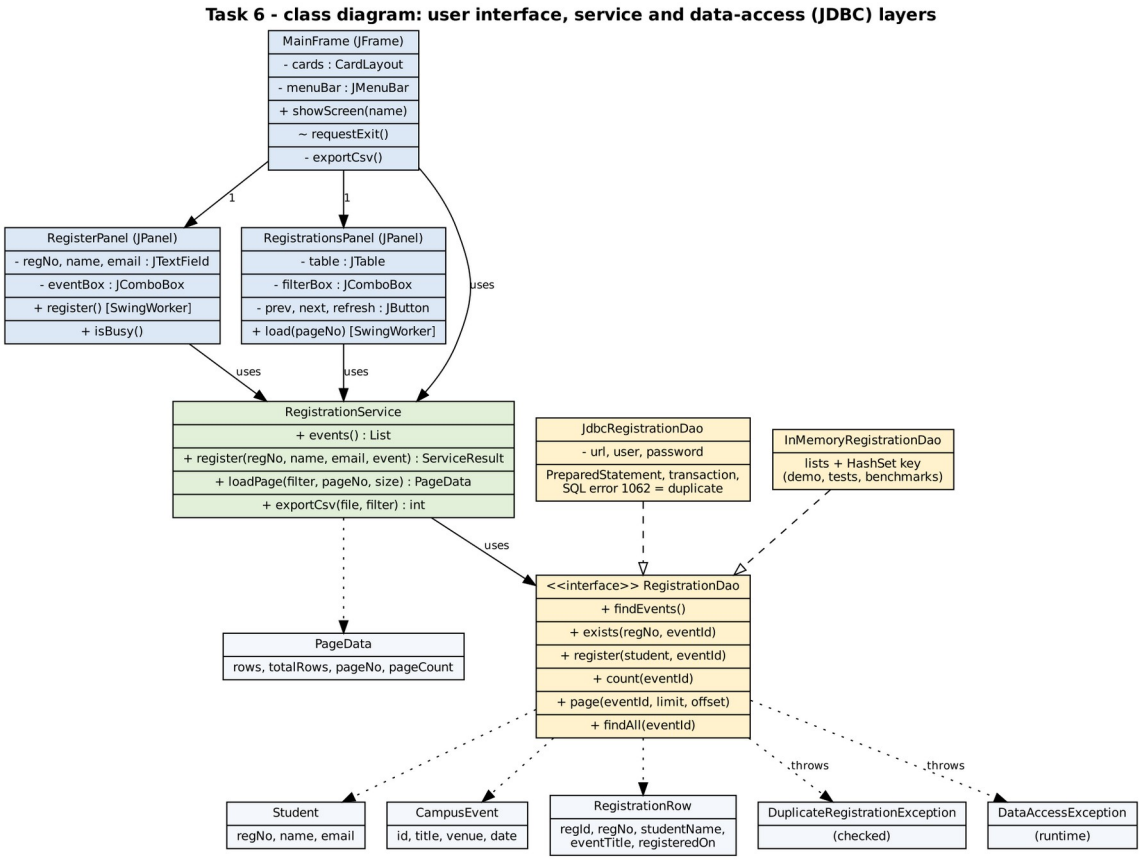


Figure 6.4 – Class diagram (user-interface, service and data-access classes)

```
ALGORITHM RegisterButtonClicked                       // runs on the event-dispatch thread
INPUT   : text of regNo, name, email fields; selected event
OUTPUT  : status message; table refreshed when the registration succeeded
BEGIN
  1.  SET regNo, name, email, event <- values currently shown in the form
  2.  CALL registerButton.setEnabled(false)
  3.  CALL status.setText("Saving ...")
  4.  START a SwingWorker whose background step is:                 // worker thread, not the
event thread
  5.      RETURN service.register(regNo, name, email, event)
  6.  WHEN the worker is done (back on the event-dispatch thread) DO
  7.      SET result <- worker result
  8.      CALL status.setText(result.message) in green if result.success, otherwise in red
  9.      IF result.success THEN  CALL clearFields()  AND  CALL listPanel.load(1)  ENDIF
  10.     CALL registerButton.setEnabled(true)
  11. ENDWHEN
END

ALGORITHM Register (service)                          // worker thread
INPUT   : regNo, name, email, event
OUTPUT  : ServiceResult
BEGIN
  1.  IF any of regNo, name, email is empty THEN RETURN failure "all fields required" ENDIF
  2.  IF regNo is not 9 digits THEN RETURN failure ENDIF
  3.  IF name does not match letters / spaces / dots THEN RETURN failure ENDIF
  4.  IF email does not match the e-mail pattern THEN RETURN failure ENDIF
  5.  IF event is not selected THEN RETURN failure ENDIF
  6.  IF dao.exists(regNo, event) THEN RETURN failure "already registered" ENDIF
  7.  TRY  CALL dao.register(student, event)                        // transaction, UNIQUE key as
last guard
  8.  CATCH DuplicateRegistrationException  RETURN failure "already registered"
```

```
  9.  CATCH DataAccessException            RETURN failure "Database problem: ..."
  10. RETURN success "Registered name for event"
END

ALGORITHM LoadPage                                  // filter changed, Prev, Next or Refresh
INPUT   : filter (event or All), requested page number, page size
OUTPUT  : table shows one page; Prev/Next enabled according to the page
BEGIN
  1.  CALL setControlsEnabled(false)  AND  CALL status.setText("Loading ...")
  2.  START a SwingWorker whose background step is:
  3.      SET total <- dao.count(filter)
  4.      SET pages <- max(1, ceiling(total / pageSize))
  5.      SET page  <- limit the requested page number to the range 1 .. pages
  6.      SET rows  <- dao.page(filter, pageSize, (page - 1) * pageSize)       // SQL LIMIT
pageSize OFFSET (page-1)*pageSize
  7.      RETURN (rows, total, page, pages)
  8.  WHEN done DO
  9.      CALL table model: remove all rows, then add the returned rows
  10.     CALL pageLabel.setText("Page page of pages")  AND  status.setText(total + "
registration(s) found")
  11.     CALL setControlsEnabled(true); Prev is disabled on page 1, Next on the last page
  12. ENDWHEN
END
```

Stage 5 – Implement and Demonstrate (3 marks): the application

The application is a Java package campus.reg with four sub-packages (model, dao, service, ui). It compiles with a plain JDK; MySQL Connector/J is needed only to run the JDBC version.

Task06/sql/campus_events.sql - MySQL script: tables, constraints, sample data

```
-- Task 6 : Campus Event Registration  (MySQL 8)
CREATE DATABASE IF NOT EXISTS campus_events CHARACTER SET utf8mb4;
USE campus_events;

CREATE TABLE student (
    reg_no  VARCHAR(12) PRIMARY KEY,
    name    VARCHAR(60) NOT NULL,
    email   VARCHAR(80) NOT NULL
) ENGINE=InnoDB;

CREATE TABLE event (
    event_id   INT AUTO_INCREMENT PRIMARY KEY,
    title      VARCHAR(80) NOT NULL UNIQUE,
    venue      VARCHAR(60) NOT NULL,
    event_date DATE        NOT NULL
) ENGINE=InnoDB;

CREATE TABLE registration (
    reg_id        INT AUTO_INCREMENT PRIMARY KEY,
    reg_no        VARCHAR(12) NOT NULL,
    event_id      INT         NOT NULL,
    registered_on TIMESTAMP   NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT uq_student_event UNIQUE (reg_no, event_id),          -- RULE: one registration per
student per event
    CONSTRAINT fk_reg_student FOREIGN KEY (reg_no)   REFERENCES student(reg_no),
    CONSTRAINT fk_reg_event   FOREIGN KEY (event_id) REFERENCES event(event_id),
    INDEX idx_reg_event (event_id)                                  -- fast filter by event
) ENGINE=InnoDB;

INSERT INTO event(title, venue, event_date) VALUES
    ('Tech Symposium 2026', 'Main Auditorium',  '2026-12-05'),
    ('Cultural Night',      'Open Air Theatre', '2026-12-12'),
    ('Hackathon 24',        'Computer Lab 3',   '2027-01-10');
```

```
INSERT INTO student(reg_no, name, email) VALUES
    ('192311001', 'Arun Kumar',  'arun@example.com'),
    ('192311002', 'Divya S',     'divya@example.com'),
    ('192311003', 'Kavin Raj',   'kavin@example.com');

INSERT INTO registration(reg_no, event_id) VALUES
    ('192311001', 1), ('192311002', 1), ('192311003', 2);

-- the two queries used by the application
-----------------------------------------------------------
-- page of registrations (? = event id, ? = page size, ? = offset)
-- SELECT r.reg_id, r.reg_no, s.name, e.title, r.registered_on
--   FROM registration r JOIN student s ON s.reg_no = r.reg_no JOIN event e ON e.event_id =
r.event_id
--  WHERE r.event_id = ? ORDER BY r.reg_id LIMIT ? OFFSET ?;
-- duplicate check:  SELECT COUNT(*) FROM registration WHERE reg_no = ? AND event_id = ?;
```

Task06/src/campus/reg/model/CampusEvent.java

```
package campus.reg.model;

/** One event students can register for. toString() is what the combo box shows. */
public record CampusEvent(int id, String title, String venue, String date) {
    public static final CampusEvent ALL = new CampusEvent(0, "All events", "", "");
    @Override public String toString() { return title; }
}
```

Task06/src/campus/reg/model/Student.java

```
package campus.reg.model;

public record Student(String regNo, String name, String email) { }
```

Task06/src/campus/reg/model/RegistrationRow.java

```
package campus.reg.model;

/** One line of the registrations table (already joined with student and event). */
public record RegistrationRow(int regId, String regNo, String studentName, String eventTitle,
String registeredOn) { }
```

Task06/src/campus/reg/model/PageData.java

```
package campus.reg.model;

import java.util.List;

/** One page of rows plus the numbers needed for "Page x of y" and the Prev/Next buttons. */
public record PageData(List<RegistrationRow> rows, int totalRows, int pageNo, int pageCount) {
    public boolean hasPrev() { return pageNo > 1; }
    public boolean hasNext() { return pageNo < pageCount; }
}
```

Task06/src/campus/reg/dao/DataAccessException.java

```
package campus.reg.dao;

/** Unexpected database problem (connection lost, bad SQL ...). The user interface shows a short
message. */
public class DataAccessException extends RuntimeException {
    public DataAccessException(String message, Throwable cause) { super(message, cause); }
}
```

Task06/src/campus/reg/dao/DuplicateRegistrationException.java

```
package campus.reg.dao;

/** The student is already registered for the event (business rule, not a crash). */
public class DuplicateRegistrationException extends Exception {
    public DuplicateRegistrationException(String message) { super(message); }
}
```

Task06/src/campus/reg/dao/RegistrationDao.java

```
package campus.reg.dao;

import campus.reg.model.*;
import java.util.List;

/** Everything the application needs from the database. The service layer only knows this
interface. */
public interface RegistrationDao {
    List<CampusEvent> findEvents();
    boolean exists(String regNo, int eventId);
    void register(Student student, int eventId) throws DuplicateRegistrationException;
    /** eventId null = all events. */
    int count(Integer eventId);
    /** One page: SQL LIMIT limit OFFSET offset. */
    List<RegistrationRow> page(Integer eventId, int limit, int offset);
    /** Every row of the selection (used by the CSV export). */
    List<RegistrationRow> findAll(Integer eventId);
}
```

Task06/src/campus/reg/dao/JdbcRegistrationDao.java - MySQL / JDBC

```
package campus.reg.dao;

import campus.reg.model.*;
import java.sql.*;
import java.util.ArrayList;
import java.util.List;

/** MySQL implementation (JDBC). Needs mysql-connector-j on the class path. Every method opens and
closes its own connection. */
public class JdbcRegistrationDao implements RegistrationDao {
    private static final String SELECT_ROWS =
        "SELECT r.reg_id, r.reg_no, s.name, e.title, r.registered_on "
      + "FROM registration r JOIN student s ON s.reg_no = r.reg_no JOIN event e ON e.event_id =
r.event_id ";

    private final String url, user, password;

    public JdbcRegistrationDao(String url, String user, String password) {
        this.url = url; this.user = user; this.password = password;
    }

    private Connection open() throws SQLException { return DriverManager.getConnection(url, user,
password); }

    @Override
    public List<CampusEvent> findEvents() {
        List<CampusEvent> list = new ArrayList<>();
        try (Connection c = open(); Statement st = c.createStatement();
             ResultSet rs = st.executeQuery("SELECT event_id, title, venue, event_date FROM event
ORDER BY event_date")) {
            while (rs.next()) list.add(new CampusEvent(rs.getInt(1), rs.getString(2),
rs.getString(3), rs.getString(4)));
        } catch (SQLException e) { throw new DataAccessException("Cannot read events: " +
e.getMessage(), e); }
        return list;
    }

    @Override
    public boolean exists(String regNo, int eventId) {
        String sql = "SELECT COUNT(*) FROM registration WHERE reg_no = ? AND event_id = ?";
        try (Connection c = open(); PreparedStatement ps = c.prepareStatement(sql)) {
            ps.setString(1, regNo); ps.setInt(2, eventId);
            try (ResultSet rs = ps.executeQuery()) { rs.next(); return rs.getInt(1) > 0; }
```

```
        } catch (SQLException e) { throw new DataAccessException("Cannot check registration: " +
e.getMessage(), e); }
    }

    @Override
    public void register(Student s, int eventId) throws DuplicateRegistrationException {
        String upsertStudent = "INSERT INTO student(reg_no, name, email) VALUES (?, ?, ?) "
                             + "ON DUPLICATE KEY UPDATE name = VALUES(name), email =
VALUES(email)";
        String insertReg = "INSERT INTO registration(reg_no, event_id) VALUES (?, ?)";
        try (Connection c = open()) {
            c.setAutoCommit(false);                                    // student row +
registration row: all or nothing
            try (PreparedStatement p1 = c.prepareStatement(upsertStudent); PreparedStatement p2 =
c.prepareStatement(insertReg)) {
                p1.setString(1, s.regNo()); p1.setString(2, s.name()); p1.setString(3, s.email());
                p1.executeUpdate();
                p2.setString(1, s.regNo()); p2.setInt(2, eventId);
                p2.executeUpdate();
                c.commit();
            } catch (SQLIntegrityConstraintViolationException e) {
                c.rollback();
                if (e.getErrorCode() == 1062)                          // MySQL "duplicate entry":
the UNIQUE key caught a race
                    throw new DuplicateRegistrationException("Already registered for this
event.");
                throw new DataAccessException("Integrity error: " + e.getMessage(), e);
            } catch (SQLException e) {
                c.rollback();
                throw new DataAccessException("Cannot save registration: " + e.getMessage(), e);
            }
        } catch (SQLException e) { throw new DataAccessException("Database not available: " +
e.getMessage(), e); }
    }

    @Override
    public int count(Integer eventId) {
        String sql = "SELECT COUNT(*) FROM registration" + (eventId == null ? "" : " WHERE
event_id = ?");
        try (Connection c = open(); PreparedStatement ps = c.prepareStatement(sql)) {
            if (eventId != null) ps.setInt(1, eventId);
            try (ResultSet rs = ps.executeQuery()) { rs.next(); return rs.getInt(1); }
        } catch (SQLException e) { throw new DataAccessException("Cannot count registrations: " +
e.getMessage(), e); }
    }

    @Override
    public List<RegistrationRow> page(Integer eventId, int limit, int offset) {
        String sql = SELECT_ROWS + (eventId == null ? "" : "WHERE r.event_id = ? ") + "ORDER BY
r.reg_id LIMIT ? OFFSET ?";
        List<RegistrationRow> rows = new ArrayList<>();
        try (Connection c = open(); PreparedStatement ps = c.prepareStatement(sql)) {
            int i = 1;
            if (eventId != null) ps.setInt(i++, eventId);
            ps.setInt(i++, limit); ps.setInt(i, offset);
            try (ResultSet rs = ps.executeQuery()) { while (rs.next()) rows.add(toRow(rs)); }
        } catch (SQLException e) { throw new DataAccessException("Cannot load registrations: " +
e.getMessage(), e); }
        return rows;
    }

    @Override
    public List<RegistrationRow> findAll(Integer eventId) {
        String sql = SELECT_ROWS + (eventId == null ? "" : "WHERE r.event_id = ? ") + "ORDER BY
r.reg_id";
```

```
        List<RegistrationRow> rows = new ArrayList<>();
        try (Connection c = open(); PreparedStatement ps = c.prepareStatement(sql)) {
            if (eventId != null) ps.setInt(1, eventId);
            try (ResultSet rs = ps.executeQuery()) { while (rs.next()) rows.add(toRow(rs)); }
        } catch (SQLException e) { throw new DataAccessException("Cannot read registrations: " +
e.getMessage(), e); }
        return rows;
    }

    private static RegistrationRow toRow(ResultSet rs) throws SQLException {
        return new RegistrationRow(rs.getInt(1), rs.getString(2), rs.getString(3),
rs.getString(4), rs.getString(5));
    }
}
```

Task06/src/campus/reg/dao/InMemoryRegistrationDao.java - demo, tests, benchmarks

```
package campus.reg.dao;

import campus.reg.model.*;
import java.util.*;

/** A DAO that keeps everything in lists. Used for the demo, the tests and the benchmarks (no
MySQL needed). */
public class InMemoryRegistrationDao implements RegistrationDao {
    private final List<CampusEvent> events = new ArrayList<>();
    private final Map<String, Student> students = new HashMap<>();
    private final List<Integer> eventOfRow = new ArrayList<>();      // event id of each row (same
index as rows)
    private final List<RegistrationRow> rows = new ArrayList<>();
    private final Set<String> keys = new HashSet<>();               // "regNo|eventId" = the
UNIQUE (reg_no, event_id) key
    private volatile long latencyMillis;
    private int nextId = 1;

    public InMemoryRegistrationDao(long latencyMillis) { this.latencyMillis = latencyMillis; }
    /** Imitates the network / query time of a real database: every call waits this long. */
    public void setLatency(long millis) { this.latencyMillis = millis; }

    public static InMemoryRegistrationDao withDemoData() {
        InMemoryRegistrationDao d = new InMemoryRegistrationDao(0);
        d.events.add(new CampusEvent(1, "Tech Symposium 2026", "Main Auditorium", "2026-12-05"));
        d.events.add(new CampusEvent(2, "Cultural Night", "Open Air Theatre", "2026-12-12"));
        d.events.add(new CampusEvent(3, "Hackathon 24", "Computer Lab 3", "2027-01-10"));
        String[] names = {"Arun Kumar", "Divya S", "Kavin Raj", "Meena R", "Rahul V", "Sneha P",
"Imran K", "Priya N", "Balaji T", "Nisha G", "Anand M", "Latha J"};
        for (int i = 0; i < names.length; i++) {
            String reg = "19231" + String.format("%04d", 1001 + i);
            try { d.register(new Student(reg, names[i], names[i].split(" ")[0].toLowerCase() +
"@example.com"), 1 + (i % 3)); }
            catch (DuplicateRegistrationException e) { throw new IllegalStateException(e); }
        }
        return d;
    }

    /** Fast bulk loader for the benchmark (skips the checks). */
    public void bulkAdd(int count) {
        if (events.isEmpty()) events.add(new CampusEvent(1, "Tech Symposium 2026", "Main
Auditorium", "2026-12-05"));
        for (int i = 0; i < count; i++) {
            String reg = String.format("%09d", 100000000 + i);
            rows.add(new RegistrationRow(nextId++, reg, "Student " + i, "Tech Symposium 2026",
"2026-11-08 10:00:00"));
            eventOfRow.add(1);
        }
    }
```

```
    private void pause() { if (latencyMillis > 0) try { Thread.sleep(latencyMillis); } catch
(InterruptedException e) { Thread.currentThread().interrupt(); } }
    private String title(int eventId) { for (CampusEvent e : events) if (e.id() == eventId) return
e.title(); return "?"; }

    @Override public synchronized List<CampusEvent> findEvents() { pause(); return new
ArrayList<>(events); }
    @Override public synchronized boolean exists(String regNo, int eventId) { pause(); return
keys.contains(regNo + "|" + eventId); }

    @Override
    public synchronized void register(Student s, int eventId) throws
DuplicateRegistrationException {
        pause();
        if (!keys.add(s.regNo() + "|" + eventId)) throw new
DuplicateRegistrationException("Already registered for this event.");   // the UNIQUE key
        students.put(s.regNo(), s);
        rows.add(new RegistrationRow(nextId++, s.regNo(), s.name(), title(eventId), "2026-11-08
10:" + String.format("%02d", rows.size() % 60) + ":00"));
        eventOfRow.add(eventId);
    }

    private List<RegistrationRow> select(Integer eventId) {
        if (eventId == null) return rows;
        List<RegistrationRow> out = new ArrayList<>();
        for (int i = 0; i < rows.size(); i++) if (eventOfRow.get(i) == eventId.intValue())
out.add(rows.get(i));
        return out;
    }

    @Override public synchronized int count(Integer eventId) { pause(); return
select(eventId).size(); }

    @Override
    public synchronized List<RegistrationRow> page(Integer eventId, int limit, int offset) {
        pause();
        List<RegistrationRow> all = select(eventId);
        return new ArrayList<>(all.subList(Math.min(offset, all.size()), Math.min(offset + limit,
all.size())));
    }

    @Override public synchronized List<RegistrationRow> findAll(Integer eventId) { pause(); return
new ArrayList<>(select(eventId)); }
}
```

Task06/src/campus/reg/service/ServiceResult.java

```
package campus.reg.service;

public class ServiceResult {
    public final boolean success;
    public final String message;
    private ServiceResult(boolean s, String m) { success = s; message = m; }
    public static ServiceResult ok(String m) { return new ServiceResult(true, m); }
    public static ServiceResult fail(String m) { return new ServiceResult(false, m); }
}
```

Task06/src/campus/reg/service/RegistrationService.java

```
package campus.reg.service;

import campus.reg.dao.*;
import campus.reg.model.*;
import java.io.BufferedWriter;
import java.io.IOException;
import java.nio.charset.StandardCharsets;
```

```
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.List;
import java.util.regex.Pattern;

/** Business rules: input validation, duplicate check, paging, CSV export. No Swing code here, no
SQL here. */
public class RegistrationService {
    private static final Pattern REG_NO = Pattern.compile("\\d{9}");
    private static final Pattern NAME = Pattern.compile("[A-Za-z][A-Za-z .]{1,59}");
    private static final Pattern EMAIL = Pattern.compile("[\\w.+-]+@[\\w-]+(\\.[\\w-]+)+");

    private final RegistrationDao dao;

    public RegistrationService(RegistrationDao dao) { this.dao = dao; }

    public List<CampusEvent> events() { return dao.findEvents(); }

    /** Validates the form, checks for a duplicate, saves. Always returns a result; it never
throws to the window. */
    public ServiceResult register(String regNoText, String nameText, String emailText, CampusEvent
event) {
        String regNo = regNoText == null ? "" : regNoText.trim();
        String name  = nameText == null ? "" : nameText.trim();
        String email = emailText == null ? "" : emailText.trim();

        if (regNo.isEmpty() || name.isEmpty() || email.isEmpty()) return
ServiceResult.fail("Register number, name and e-mail are all required.");
        if (!REG_NO.matcher(regNo).matches())  return ServiceResult.fail("Register number must be
9 digits (example 192311001).");
        if (!NAME.matcher(name).matches())     return ServiceResult.fail("Name may contain
letters, spaces and dots only (2-60 characters).");
        if (!EMAIL.matcher(email).matches())   return ServiceResult.fail("E-mail address is not
valid.");
        if (event == null || event.id() == 0)  return ServiceResult.fail("Select an event.");

        try {
            if (dao.exists(regNo, event.id()))                                  // friendly check
first ...
                return ServiceResult.fail(regNo + " is already registered for " + event.title() +
".");
            dao.register(new Student(regNo, name, email), event.id());           // ... and the
UNIQUE key is the final guard
            return ServiceResult.ok("Registered " + name + " (" + regNo + ") for " + event.title()
+ ".");
        } catch (DuplicateRegistrationException e) {
            return ServiceResult.fail(regNo + " is already registered for " + event.title() +
".");
        } catch (DataAccessException e) {
            return ServiceResult.fail("Database problem: " + e.getMessage());
        }
    }

    /** One page of the table (pageNo starts at 1). */
    public PageData loadPage(CampusEvent filter, int pageNo, int pageSize) {
        Integer eventId = (filter == null || filter.id() == 0) ? null : filter.id();
        int total = dao.count(eventId);
        int pages = Math.max(1, (total + pageSize - 1) / pageSize);
        int page = Math.min(Math.max(1, pageNo), pages);
        return new PageData(dao.page(eventId, pageSize, (page - 1) * pageSize), total, page,
pages);
    }

    /** Writes the (filtered) registrations to a CSV file and returns the number of data rows
written. */
```

```
    public int exportCsv(Path file, CampusEvent filter) throws IOException {
        Integer eventId = (filter == null || filter.id() == 0) ? null : filter.id();
        List<RegistrationRow> rows = dao.findAll(eventId);
        try (BufferedWriter w = Files.newBufferedWriter(file, StandardCharsets.UTF_8)) {
            w.write("reg_id,reg_no,student_name,event_title,registered_on");
            w.newLine();
            for (RegistrationRow r : rows) {
                w.write(r.regId() + "," + csv(r.regNo()) + "," + csv(r.studentName()) + "," +
csv(r.eventTitle()) + "," + csv(r.registeredOn()));
                w.newLine();
            }
        }
        return rows.size();
    }

    private static String csv(String s) { return (s.contains(",") || s.contains("\"")) ? "\"" +
s.replace("\"", "\"\"") + "\"" : s; }
}
```

Task06/src/campus/reg/ui/RegisterPanel.java

```
package campus.reg.ui;

import campus.reg.model.CampusEvent;
import campus.reg.service.RegistrationService;
import campus.reg.service.ServiceResult;
import java.awt.*;
import java.util.List;
import java.util.concurrent.ExecutionException;
import javax.swing.*;

/** Screen 1: the registration form. Collects input and shows the result. Saving runs in a
SwingWorker. */
public class RegisterPanel extends JPanel {
    private final RegistrationService service;
    private final JTextField regNo = new JTextField(14);
    private final JTextField name = new JTextField(14);
    private final JTextField email = new JTextField(14);
    private final JComboBox<CampusEvent> eventBox = new JComboBox<>();
    private final JButton registerButton = new JButton("Register");
    private final JButton clearButton = new JButton("Clear");
    private final JLabel status = new JLabel(" ");
    private volatile boolean busy = false;
    private Runnable onRegistered = () -> { };

    public RegisterPanel(RegistrationService service) {
        this.service = service;
        setLayout(new BorderLayout(8, 8));
        setBorder(BorderFactory.createEmptyBorder(12, 14, 12, 14));

        JLabel heading = new JLabel("Register for an event");
        heading.setFont(heading.getFont().deriveFont(Font.BOLD, 16f));

        JPanel form = new JPanel(new GridLayout(4, 2, 8, 8));
        form.add(new JLabel("Register number:")); form.add(regNo);
        form.add(new JLabel("Student name:"));    form.add(name);
        form.add(new JLabel("E-mail:"));          form.add(email);
        form.add(new JLabel("Event:"));           form.add(eventBox);

        JPanel buttons = new JPanel(new FlowLayout(FlowLayout.CENTER, 10, 4));
        buttons.add(registerButton); buttons.add(clearButton);
        JPanel south = new JPanel(new BorderLayout());
        south.add(buttons, BorderLayout.NORTH);
        south.add(status, BorderLayout.SOUTH);

        add(heading, BorderLayout.NORTH);
```

```
        add(form, BorderLayout.CENTER);
        add(south, BorderLayout.SOUTH);

        registerButton.addActionListener(e -> register());
        clearButton.addActionListener(e -> clear());
    }

    public void setEvents(List<CampusEvent> events) {
        eventBox.removeAllItems();
        eventBox.addItem(CampusEvent.ALL);                    // the "select" prompt (id 0 is
rejected by the service)
        for (CampusEvent ev : events) eventBox.addItem(ev);
        eventBox.getModel().setSelectedItem(CampusEvent.ALL);
    }

    public void setOnRegistered(Runnable r) { onRegistered = r; }

    private void register() {
        final String r = regNo.getText(), n = name.getText(), m = email.getText();
        final CampusEvent ev = (CampusEvent) eventBox.getSelectedItem();
        busy = true;
        registerButton.setEnabled(false);
        showStatus("Saving ...", false);
        new SwingWorker<ServiceResult, Void>() {
            @Override protected ServiceResult doInBackground() { return service.register(r, n, m,
ev); }   // database work: NOT on the EDT
            @Override protected void done() {
                try {
                    ServiceResult result = get();
                    showStatus(result.message, !result.success);
                    if (result.success) { clearFields(); onRegistered.run(); }
                } catch (InterruptedException | ExecutionException ex) {
                    showStatus("Unexpected error: " + ex.getMessage(), true);
                } finally {
                    busy = false;
                    registerButton.setEnabled(true);
                }
            }
        }.execute();
    }

    private void clearFields() { regNo.setText(""); name.setText(""); email.setText("");
eventBox.setSelectedIndex(0); }
    private void clear() { clearFields(); showStatus(" ", false); }
    private void showStatus(String text, boolean error) {
        status.setForeground(error ? new Color(0xB00020) : new Color(0x1B5E20));
        status.setText(text);
    }

    public boolean isBusy() { return busy; }
    public String statusText() { return status.getText(); }
    public JTextField regNoField() { return regNo; }
    public JTextField nameField() { return name; }
    public JTextField emailField() { return email; }
    public JComboBox<CampusEvent> eventBox() { return eventBox; }
    public JButton registerButton() { return registerButton; }
}
```

Task06/src/campus/reg/ui/RegistrationsPanel.java

```
package campus.reg.ui;

import campus.reg.model.*;
import campus.reg.service.RegistrationService;
import java.awt.*;
import java.util.List;
```

```
import java.util.concurrent.ExecutionException;
import javax.swing.*;
import javax.swing.table.DefaultTableModel;

/** Screen 2: all registrations in a JTable, filter by event, one page at a time (SQL LIMIT /
OFFSET inside a SwingWorker). */
public class RegistrationsPanel extends JPanel {
    private static final String[] COLUMNS = {"Reg ID", "Register No", "Student", "Event",
"Registered on"};
    private final RegistrationService service;
    private final int pageSize;
    private final JComboBox<CampusEvent> filterBox = new JComboBox<>();
    private final DefaultTableModel model = new DefaultTableModel(COLUMNS, 0) {
        @Override public boolean isCellEditable(int row, int column) { return false; }
    };
    private final JTable table = new JTable(model);
    private final JButton prevButton = new JButton("Prev");
    private final JButton nextButton = new JButton("Next");
    private final JButton refreshButton = new JButton("Refresh");
    private final JLabel pageLabel = new JLabel("Page 1 of 1");
    private final JLabel status = new JLabel(" ");
    private volatile boolean loading = false;
    private int currentPage = 1;
    private PageData lastPage;

    public RegistrationsPanel(RegistrationService service, int pageSize) {
        this.service = service;
        this.pageSize = pageSize;
        setLayout(new BorderLayout(6, 6));
        setBorder(BorderFactory.createEmptyBorder(10, 12, 10, 12));

        JPanel top = new JPanel(new FlowLayout(FlowLayout.LEFT, 8, 4));
        top.add(new JLabel("Show event:")); top.add(filterBox); top.add(refreshButton);

        table.setFillsViewportHeight(true);
        table.setRowHeight(22);
        JPanel bottom = new JPanel(new BorderLayout());
        JPanel nav = new JPanel(new FlowLayout(FlowLayout.CENTER, 10, 4));
        nav.add(prevButton); nav.add(pageLabel); nav.add(nextButton);
        bottom.add(nav, BorderLayout.CENTER);
        bottom.add(status, BorderLayout.SOUTH);

        int[] widths = {55, 110, 140, 170, 150};
        for (int i = 0; i < widths.length; i++)
table.getColumnModel().getColumn(i).setPreferredWidth(widths[i]);
        JScrollPane scroll = new JScrollPane(table);
        scroll.setColumnHeaderView(table.getTableHeader());       // column titles (normally
installed automatically when the window is shown)

        add(top, BorderLayout.NORTH);
        add(scroll, BorderLayout.CENTER);
        add(bottom, BorderLayout.SOUTH);

        filterBox.addActionListener(e -> { if (!loading) load(1); });
        prevButton.addActionListener(e -> load(currentPage - 1));
        nextButton.addActionListener(e -> load(currentPage + 1));
        refreshButton.addActionListener(e -> load(currentPage));
    }

    public void setEvents(List<CampusEvent> events) {
        filterBox.removeAllItems();
        filterBox.addItem(CampusEvent.ALL);
        for (CampusEvent ev : events) filterBox.addItem(ev);
    }
```

```
    /** Starts loading page pageNo in the background; the table is updated on the event-dispatch
thread when the data arrives. */
    public void load(int pageNo) {
        final CampusEvent filter = (CampusEvent) filterBox.getSelectedItem();
        loading = true;
        setControlsEnabled(false);
        status.setForeground(Color.DARK_GRAY);
        status.setText("Loading ...");
        new SwingWorker<PageData, Void>() {
            @Override protected PageData doInBackground() { return service.loadPage(filter,
pageNo, pageSize); }   // JDBC call, off the EDT
            @Override protected void done() {
                try {
                    show(get());
                } catch (InterruptedException | ExecutionException ex) {
                    status.setForeground(new Color(0xB00020));
                    status.setText("Could not load: " + ex.getCause());
                } finally {
                    loading = false;
                    setControlsEnabled(true);
                    updateNavButtons();
                }
            }
        }.execute();
    }

    private void show(PageData data) {
        lastPage = data;
        currentPage = data.pageNo();
        model.setRowCount(0);
        for (RegistrationRow r : data.rows()) model.addRow(new Object[]{r.regId(), r.regNo(),
r.studentName(), r.eventTitle(), r.registeredOn()});
        pageLabel.setText("Page " + data.pageNo() + " of " + data.pageCount());
        status.setText(data.totalRows() + " registration(s) found");
    }

    private void setControlsEnabled(boolean on) { filterBox.setEnabled(on);
refreshButton.setEnabled(on); prevButton.setEnabled(on); nextButton.setEnabled(on); }
    private void updateNavButtons() { if (lastPage != null)
{ prevButton.setEnabled(lastPage.hasPrev()); nextButton.setEnabled(lastPage.hasNext()); } }

    public CampusEvent selectedFilter() { return (CampusEvent) filterBox.getSelectedItem(); }
    public boolean isLoading() { return loading; }
    public JTable table() { return table; }
    public JComboBox<CampusEvent> filterBox() { return filterBox; }
    public JButton prevButton() { return prevButton; }
    public JButton nextButton() { return nextButton; }
    public JLabel pageLabel() { return pageLabel; }
    public JLabel statusLabel() { return status; }
}
```

Task06/src/campus/reg/ui/MainFrame.java - JMenuBar, CardLayout, export, exit

```
package campus.reg.ui;

import campus.reg.service.RegistrationService;
import java.awt.*;
import java.awt.event.*;
import java.io.File;
import java.nio.file.Path;
import javax.swing.*;

/** The main window: menu bar + two screens (cards). Only wiring and menu actions live here. */
public class MainFrame extends JFrame {
    public static final String REGISTER = "register", LIST = "list";
    private final RegistrationService service;
```

```
    private final RegisterPanel registerPanel;
    private final RegistrationsPanel listPanel;
    private final CardLayout cards = new CardLayout();
    private final JPanel center = new JPanel(cards);
    private final JMenuBar menuBar = new JMenuBar();

    public MainFrame(RegistrationService service, int pageSize) {
        super("Campus Event Registration");
        this.service = service;
        registerPanel = new RegisterPanel(service);
        listPanel = new RegistrationsPanel(service, pageSize);
        registerPanel.setEvents(service.events());
        listPanel.setEvents(service.events());
        registerPanel.setOnRegistered(() -> { if (!listPanel.isLoading()) listPanel.load(1); });
// keep the table up to date

        center.add(registerPanel, REGISTER);
        center.add(listPanel, LIST);
        add(center, BorderLayout.CENTER);
        buildMenu();
        setJMenuBar(menuBar);

        setDefaultCloseOperation(DO_NOTHING_ON_CLOSE);
        addWindowListener(new WindowAdapter() {
            @Override public void windowClosing(WindowEvent e) { requestExit(); }
        });
        setSize(620, 420);
        setLocationRelativeTo(null);
        showScreen(REGISTER);
    }

    /** Shared by the window's close button and File > Exit: asks only when work is still running.
*/
    void requestExit() {
        if (registerPanel.isBusy() || listPanel.isLoading()) {
            int a = JOptionPane.showConfirmDialog(this, "Work is still in progress. Exit anyway?",
"Exit", JOptionPane.YES_NO_OPTION);
            if (a != JOptionPane.YES_OPTION) return;
        }
        dispose();
        System.exit(0);
    }

    private void buildMenu() {
        JMenu file = new JMenu("File");
        JMenuItem export = new JMenuItem("Export CSV...");
        export.setAccelerator(KeyStroke.getKeyStroke(KeyEvent.VK_E, InputEvent.CTRL_DOWN_MASK));
        export.addActionListener(e -> exportCsv());
        JMenuItem exit = new JMenuItem("Exit");
        exit.addActionListener(e -> requestExit());
        file.add(export); file.addSeparator(); file.add(exit);

        JMenu view = new JMenu("View");
        JMenuItem reg = new JMenuItem("Register screen");
        reg.addActionListener(e -> showScreen(REGISTER));
        JMenuItem list = new JMenuItem("Registrations screen");
        list.addActionListener(e -> showScreen(LIST));
        JMenuItem refresh = new JMenuItem("Refresh table");
        refresh.addActionListener(e -> { showScreen(LIST); });
        view.add(reg); view.add(list); view.addSeparator(); view.add(refresh);

        JMenu help = new JMenu("Help");
        JMenuItem about = new JMenuItem("About");
        about.addActionListener(e -> JOptionPane.showMessageDialog(this,
```

```
                "Campus Event Registration\nCultural committee desktop application\nSwing + JDBC
(MySQL)", "About", JOptionPane.INFORMATION_MESSAGE));
        help.add(about);

        menuBar.add(file); menuBar.add(view); menuBar.add(help);
    }

    /** Menu action: ask for a file name, then write the CSV in a worker thread. */
    private void exportCsv() {
        JFileChooser chooser = new JFileChooser();
        chooser.setSelectedFile(new File("registrations.csv"));
        if (chooser.showSaveDialog(this) != JFileChooser.APPROVE_OPTION) return;
        final Path path = chooser.getSelectedFile().toPath();
        new SwingWorker<Integer, Void>() {
            @Override protected Integer doInBackground() throws Exception { return
service.exportCsv(path, listPanel.selectedFilter()); }
            @Override protected void done() {
                try {
                    JOptionPane.showMessageDialog(MainFrame.this, get() + " row(s) exported to " +
path, "Export", JOptionPane.INFORMATION_MESSAGE);
                } catch (Exception ex) {
                    JOptionPane.showMessageDialog(MainFrame.this, "Export failed: " +
ex.getMessage(), "Export", JOptionPane.ERROR_MESSAGE);
                }
            }
        }.execute();
    }

    public void showScreen(String name) {
        cards.show(center, name);
        if (LIST.equals(name) && !listPanel.isLoading()) listPanel.load(1);
    }

    public RegisterPanel registerPanel() { return registerPanel; }
    public RegistrationsPanel listPanel() { return listPanel; }
    @Override public JMenuBar getJMenuBar() { return menuBar; }
}
```

Task06/src/campus/reg/App.java

```
package campus.reg;

import campus.reg.dao.InMemoryRegistrationDao;
import campus.reg.dao.JdbcRegistrationDao;
import campus.reg.dao.RegistrationDao;
import campus.reg.service.RegistrationService;
import campus.reg.ui.MainFrame;
import javax.swing.SwingUtilities;

/**
 * Entry point.
 *   MySQL : java -cp "out;mysql-connector-j-8.4.0.jar" campus.reg.App -Ddb.url=... (see README)
 *   Demo  : java -Ddemo=true -cp out campus.reg.App    (no database, sample data in memory)
 */
public class App {
    public static void main(String[] args) {
        RegistrationDao dao;
        if (Boolean.getBoolean("demo")) {
            dao = InMemoryRegistrationDao.withDemoData();
        } else {
            String url = System.getProperty("db.url",
"jdbc:mysql://localhost:3306/campus_events");
            dao = new JdbcRegistrationDao(url, System.getProperty("db.user", "root"),
System.getProperty("db.pwd", ""));
        }
        RegistrationService service = new RegistrationService(dao);
```

```
        SwingUtilities.invokeLater(() -> new MainFrame(service, 10).setVisible(true));
    }
}
```

Task06/test/ServiceDemo.java - console demonstration of normal, invalid and duplicate cases

```
import campus.reg.dao.InMemoryRegistrationDao;
import campus.reg.model.*;
import campus.reg.service.*;
import java.nio.file.*;
import java.util.List;

/** Console demonstration of the service layer: normal, invalid and duplicate registrations (Stage
5). */
public class ServiceDemo {
    static int pass = 0, fail = 0;
    static void show(String title, ServiceResult r, boolean expectOk) {
        boolean ok = r.success == expectOk;
        if (ok) pass++; else fail++;
        System.out.printf("%-4s %-40s -> %s%n", ok ? "PASS" : "FAIL", title, r.message);
    }

    public static void main(String[] args) throws Exception {
        InMemoryRegistrationDao dao = InMemoryRegistrationDao.withDemoData();
        RegistrationService service = new RegistrationService(dao);
        List<CampusEvent> events = service.events();
        System.out.println("Events: " + events);
        CampusEvent tech = events.get(0), cultural = events.get(1), hack = events.get(2);
        System.out.println("Registrations at start: " + service.loadPage(CampusEvent.ALL, 1,
100).totalRows());
        System.out.println();

        System.out.println("--- NORMAL registrations ---");
        show("new student, Hackathon",              service.register("192319001", "Ravi Teja",
"ravi@example.com", hack), true);
        show("same student, another event",         service.register("192319001", "Ravi Teja",
"ravi@example.com", cultural), true);
        show("name with dot and spaces",            service.register("192319002", "A. P. Mohan",
"mohan@college.edu.in", tech), true);

        System.out.println();
        System.out.println("--- INVALID input (nothing reaches the database) ---");
        show("all fields empty",                    service.register("", "", "", tech), false);
        show("register number too short",           service.register("1923", "Ravi Teja",
"ravi@example.com", tech), false);
        show("register number with letters",        service.register("19231900A", "Ravi Teja",
"ravi@example.com", tech), false);
        show("name with digits",                    service.register("192319003", "Ravi123",
"ravi@example.com", tech), false);
        show("e-mail without @",                    service.register("192319003", "Ravi Teja",
"ravi.example.com", tech), false);
        show("no event selected (prompt)",          service.register("192319003", "Ravi Teja",
"ravi@example.com", CampusEvent.ALL), false);
        show("SQL text in the name",                service.register("192319003", "x'); DROP TABLE
student;--", "a@b.com", tech), false);
        show("null values",                         service.register(null, null, null, null),
false);

        System.out.println();
        System.out.println("--- DUPLICATE registrations ---");
        show("same student, same event again",      service.register("192319001", "Ravi Teja",
"ravi@example.com", hack), false);
        show("same again (different spelling)",     service.register(" 192319001 ", "Ravi T",
"ravi@example.com", hack), false);
        show("duplicate of seeded data",            service.register("192311001", "Arun Kumar",
"arun@example.com", tech), false);
```

```
        // the second line of defence: the UNIQUE key itself (what MySQL does when two clients
register at the same moment)
        try {
            dao.register(new Student("192319001", "Ravi Teja", "ravi@example.com"), hack.id());
            System.out.println("FAIL  UNIQUE key did not fire");
            fail++;
        } catch (campus.reg.dao.DuplicateRegistrationException e) {
            System.out.println("PASS direct insert behind the service's back          -> blocked
by the UNIQUE key: " + e.getMessage());
            pass++;
        }

        System.out.println();
        System.out.println("--- Paging (page size 5) and filtering ---");
        PageData p = service.loadPage(CampusEvent.ALL, 1, 5);
        System.out.println("All events: " + p.totalRows() + " rows, " + p.pageCount() + " pages;
page 1:");
        for (RegistrationRow r : p.rows()) System.out.println("   " + r);
        p = service.loadPage(hack, 1, 5);
        System.out.println("Filter = " + hack + ": " + p.totalRows() + " rows");
        p = service.loadPage(CampusEvent.ALL, 99, 5);
        System.out.println("Page 99 requested -> clamped to page " + p.pageNo() + " of " +
p.pageCount());

        System.out.println();
        System.out.println("--- Export ---");
        Path out = Paths.get("registrations_export.csv");
        int n = service.exportCsv(out, CampusEvent.ALL);
        System.out.println(n + " rows written to " + out + ". First lines:");
        List<String> lines = Files.readAllLines(out);
        for (int i = 0; i < Math.min(4, lines.size()); i++) System.out.println("   " +
lines.get(i));

        System.out.println();
        System.out.println("Result: " + pass + " passed, " + fail + " failed");
    }
}
```

Database design (ER diagram)

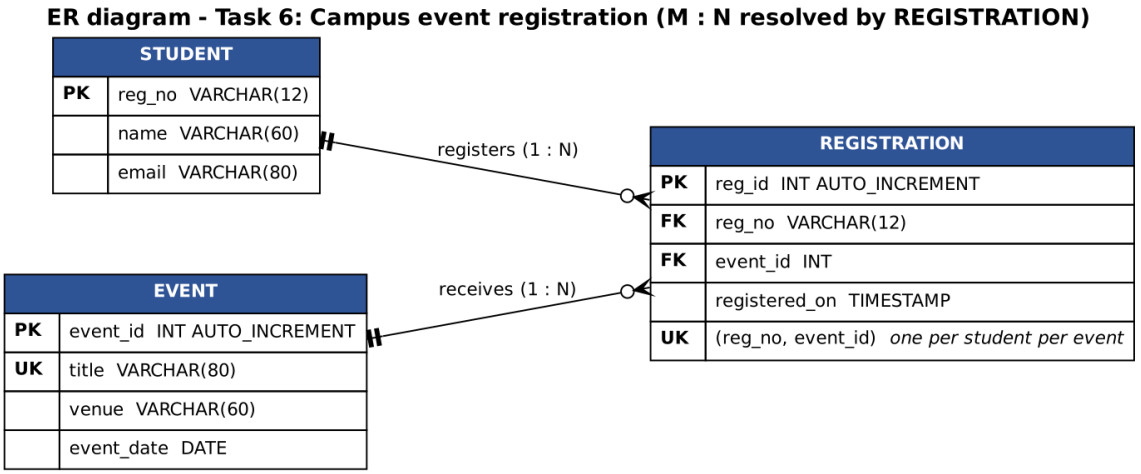


Figure 6.5 – ER diagram: a student can register for many events and an event has many students; REGISTRATION resolves the many-to-many relationship and carries the UNIQUE key

Stage 5 (continued) – Demonstration of normal, invalid and duplicate registrations

Normal, invalid and duplicate cases at service level (output of java ServiceDemo):

Output of ServiceDemo (tests T1–T9)

```
Events: [Tech Symposium 2026, Cultural Night, Hackathon 24]
Registrations at start: 12

--- NORMAL registrations ---
PASS new student, Hackathon                  -> Registered Ravi Teja (192319001) for Hackathon 24.
PASS same student, another event             -> Registered Ravi Teja (192319001) for Cultural Night.
PASS name with dot and spaces                -> Registered A. P. Mohan (192319002) for Tech Symposium
2026.

--- INVALID input (nothing reaches the database) ---
PASS all fields empty                        -> Register number, name and e-mail are all required.
PASS register number too short               -> Register number must be 9 digits (example 192311001).
PASS register number with letters            -> Register number must be 9 digits (example 192311001).
PASS name with digits                        -> Name may contain letters, spaces and dots only (2-60
characters).
PASS e-mail without @                        -> E-mail address is not valid.
PASS no event selected (prompt)              -> Select an event.
PASS SQL text in the name                    -> Name may contain letters, spaces and dots only (2-60
characters).
PASS null values                             -> Register number, name and e-mail are all required.

--- DUPLICATE registrations ---
PASS same student, same event again          -> 192319001 is already registered for Hackathon 24.
PASS same again (different spelling)         -> 192319001 is already registered for Hackathon 24.
PASS duplicate of seeded data                -> 192311001 is already registered for Tech Symposium 2026.
PASS direct insert behind the service's back          -> blocked by the UNIQUE key: Already registered
for this event.

--- Paging (page size 5) and filtering ---
All events: 15 rows, 3 pages; page 1:
   RegistrationRow[regId=1, regNo=192311001, studentName=Arun Kumar, eventTitle=Tech Symposium 2026,
registeredOn=2026-11-08 10:00:00]
   RegistrationRow[regId=2, regNo=192311002, studentName=Divya S, eventTitle=Cultural Night,
registeredOn=2026-11-08 10:01:00]
   RegistrationRow[regId=3, regNo=192311003, studentName=Kavin Raj, eventTitle=Hackathon 24,
registeredOn=2026-11-08 10:02:00]
   RegistrationRow[regId=4, regNo=192311004, studentName=Meena R, eventTitle=Tech Symposium 2026,
registeredOn=2026-11-08 10:03:00]
   RegistrationRow[regId=5, regNo=192311005, studentName=Rahul V, eventTitle=Cultural Night,
registeredOn=2026-11-08 10:04:00]
Filter = Hackathon 24: 5 rows
Page 99 requested -> clamped to page 3 of 3

--- Export ---
15 rows written to registrations_export.csv. First lines:
   reg_id,reg_no,student_name,event_title,registered_on
   1,192311001,Arun Kumar,Tech Symposium 2026,2026-11-08 10:00:00
   2,192311002,Divya S,Cultural Night,2026-11-08 10:01:00
   3,192311003,Kavin Raj,Hackathon 24,2026-11-08 10:02:00

Result: 15 passed, 0 failed
```

The same cases through the real window (the real MainFrame, RegisterPanel and RegistrationsPanel with their SwingWorkers, driven by button clicks; off-screen render). The output lists the menu structure, each step and the responsiveness measurement:

Output of RegistrationHarness

```
=== Menu bar ===
  File : Export CSV...  | Exit
  View : Register screen  Registrations screen  | Refresh table
  Help : About

=== Stage 5 demonstration through the real window ===
[invalid]   empty form, click Register  -> Register number, name and e-mail are all required.
```

```
[invalid]   bad e-mail                 -> E-mail address is not valid.
[normal]    valid form                 -> Registered Ravi Teja (192319010) for Hackathon 24.
[duplicate] same student, same event   -> 192319010 is already registered for Hackathon 24.
[table]     Registrations screen: Page 1 of 3, 13 registration(s) found
      1                    192311001             Arun Kumar            Tech Symposium 2026   2026-11-08
10:00:00
      2                    192311002             Divya S               Cultural Night        2026-11-08
10:01:00
      3                    192311003             Kavin Raj             Hackathon 24          2026-11-08
10:02:00
      4                    192311004             Meena R               Tech Symposium 2026   2026-11-08
10:03:00
      5                    192311005             Rahul V               Cultural Night        2026-11-08
10:04:00
[paging]    Next -> Page 2 of 3
      6                    192311006             Sneha P               Hackathon 24          2026-11-08
10:05:00
      7                    192311007             Imran K               Tech Symposium 2026   2026-11-08
10:06:00
      8                    192311008             Priya N               Cultural Night        2026-11-08
10:07:00
      9                    192311009             Balaji T              Hackathon 24          2026-11-08
10:08:00
      10                   192311010             Nisha G               Tech Symposium 2026   2026-11-08
10:09:00
[filter]    event = Cultural Night -> Page 1 of 1, 4 registration(s) found
      2                    192311002             Divya S               Cultural Night        2026-11-08
10:01:00
      5                    192311005             Rahul V               Cultural Night        2026-11-08
10:04:00
      8                    192311008             Priya N               Cultural Night        2026-11-08
10:07:00
      11                   192311011             Anand M               Cultural Night        2026-11-08
10:10:00

=== Responsiveness with a slow database (400 ms per call, count + page = 2 calls) ===
page loaded after 855 ms; while loading the status showed "Loading ..."; 21 pings to the event-dispatch
thread, slowest answer 2.00 ms
final status: 12 registration(s) found / Page 1 of 3
```

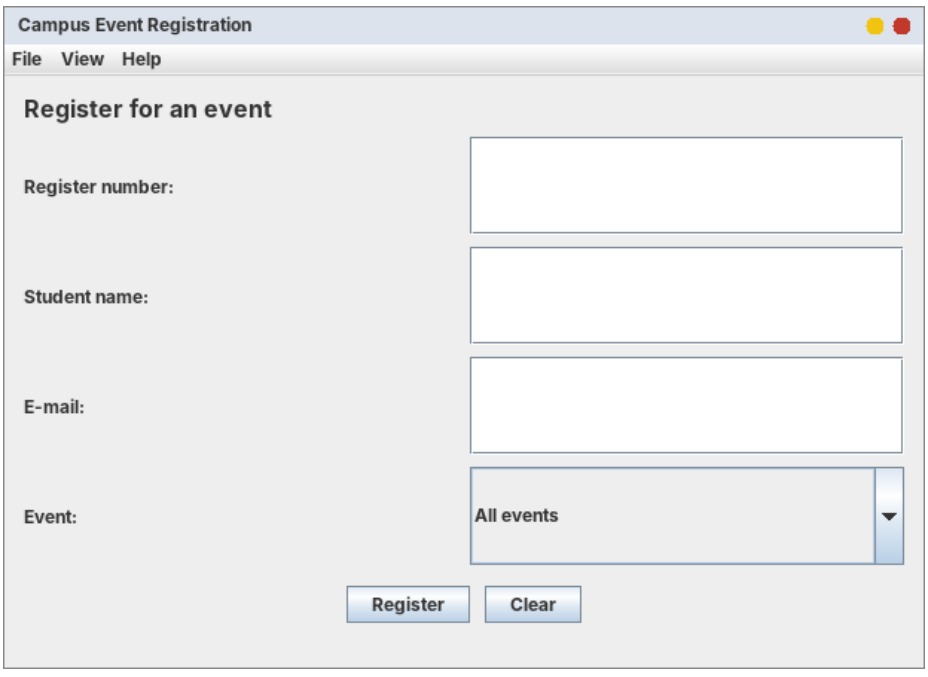


Figure 6.6 – Demonstrates REQ-10: the Register screen with the menu bar (File, View, Help)

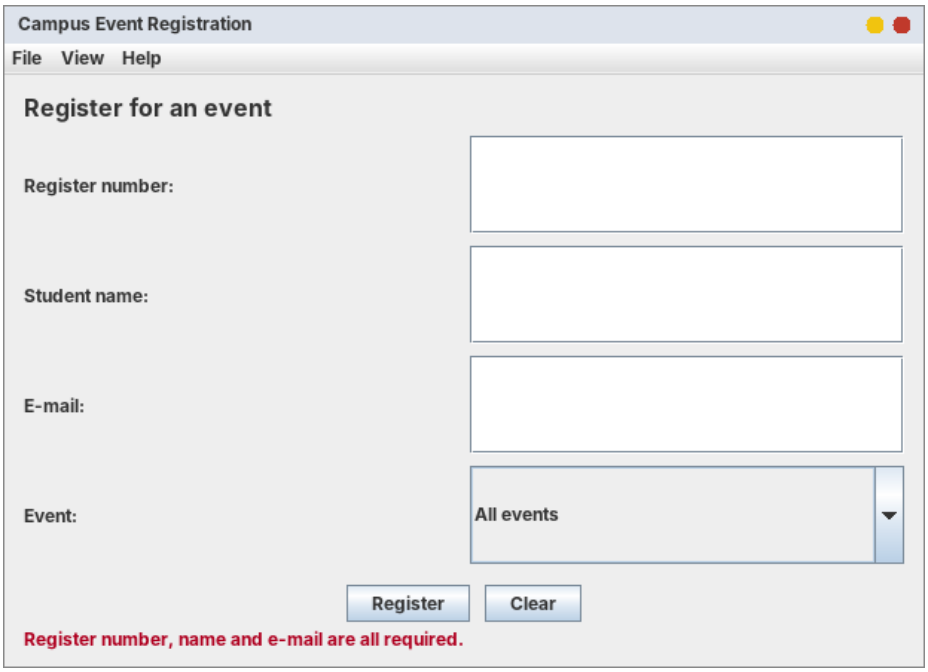


Figure 6.7 – Demonstrates REQ-04 (T4): empty form

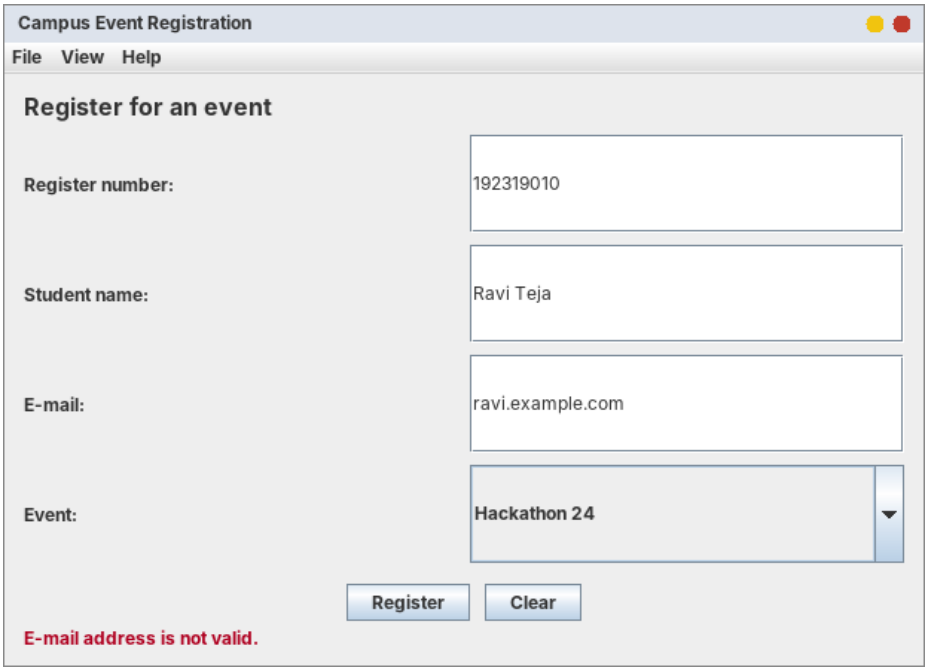


Figure 6.8 – Demonstrates REQ-03 (T3): invalid e-mail address

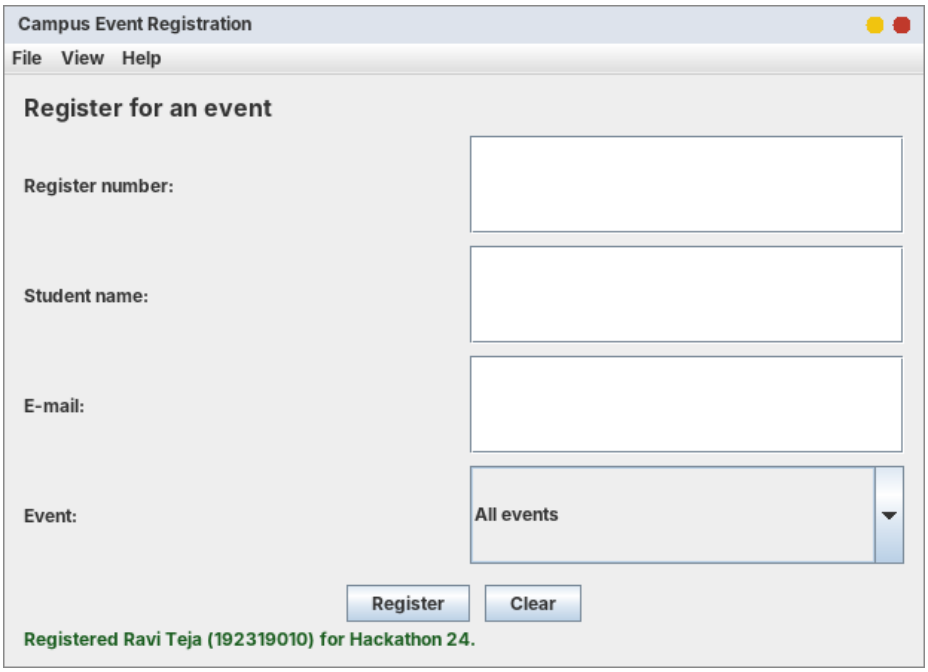


Figure 6.9 – Demonstrates the normal case (REQ-07): registration saved

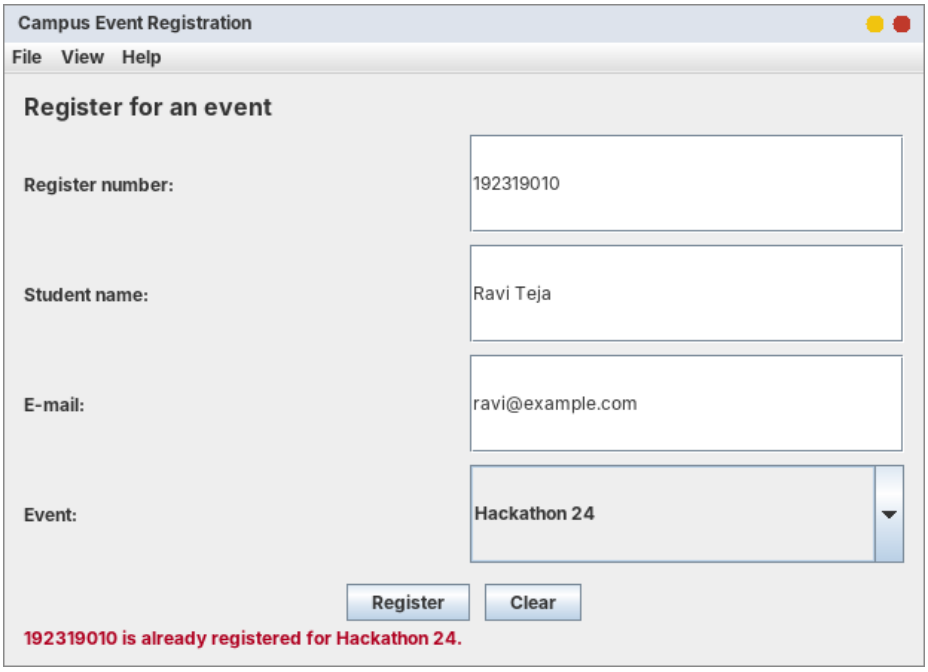


Figure 6.10 – Demonstrates REQ-05 (T5): the same student and event again is rejected

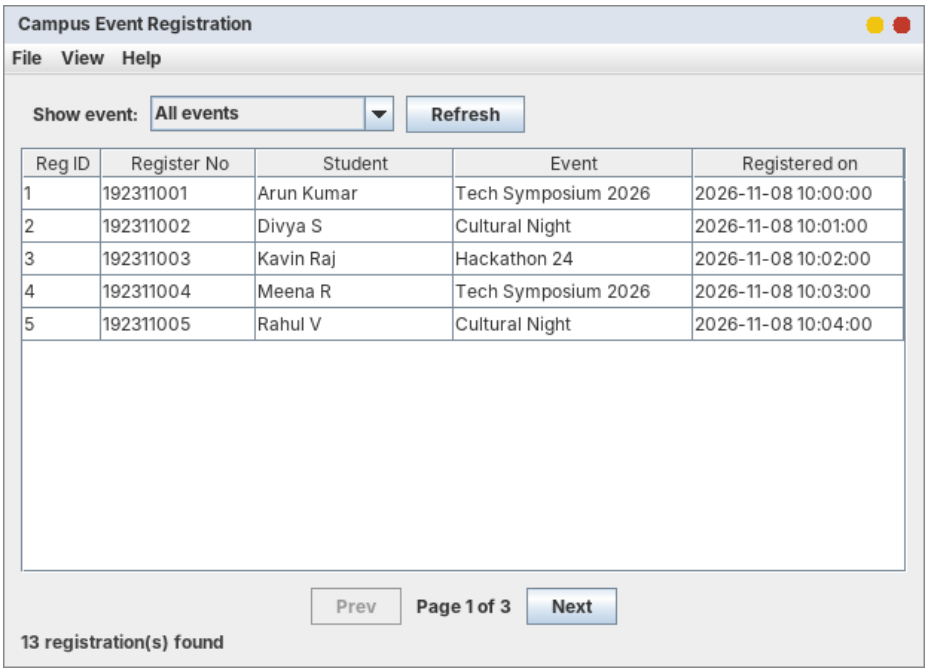


Figure 6.11 – Demonstrates REQ-08 (T8): Registrations screen, page 1 of 3 (13 registrations, 5 per page in the demo)

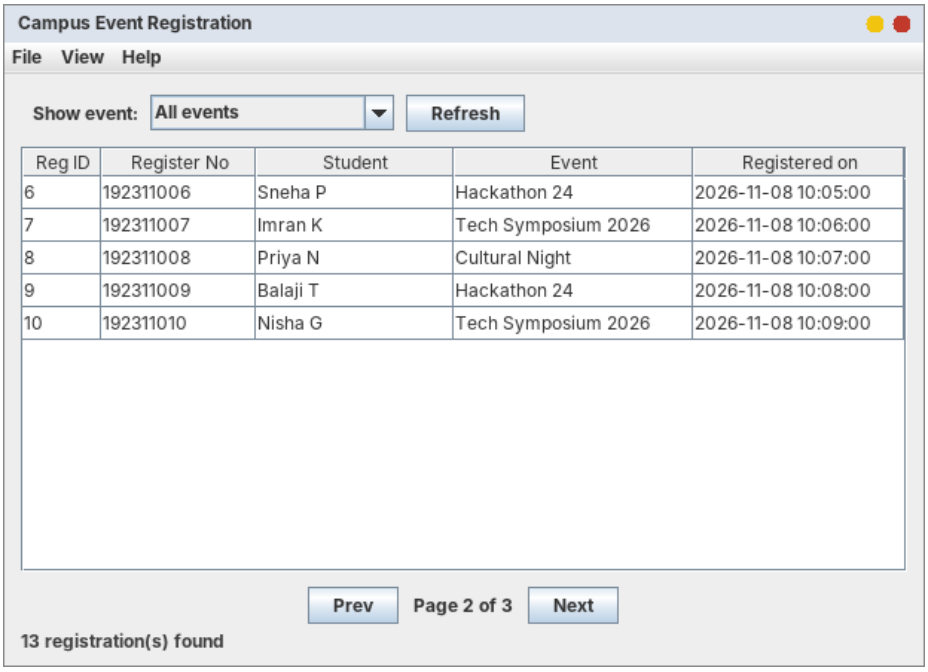


Figure 6.12 – Demonstrates REQ-08: the Next button shows page 2

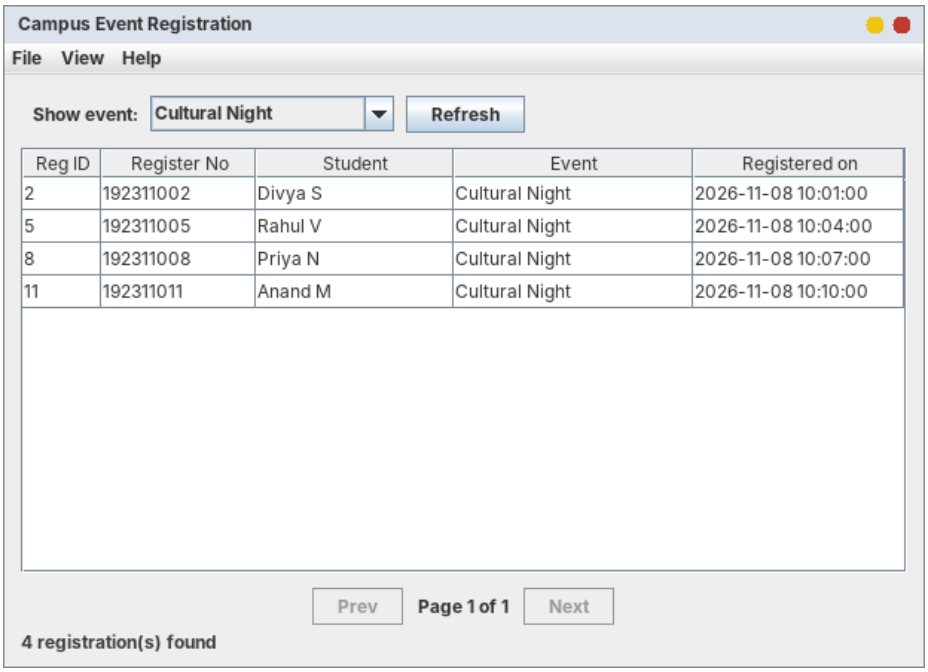


Figure 6.13 – Demonstrates REQ-08: filtered by the event "Cultural Night" (4 registrations)

REQ-11 (the window never freezes). With a database that needs 400 ms per call (count + page = 2 calls, 855 ms in total) the Registrations screen showed "Loading …" while the event-dispatch thread answered 21 test requests, the slowest in 2.00 ms. The window therefore stayed responsive for the whole load.

SQL script check (SQLite stand-in)

No MySQL server was available while preparing this report, so the logic of the MySQL script (constraints, joins, paging) was checked with SQLite after a mechanical conversion (sql_logic_check.py). SQLite is not MySQL: the error texts and the upsert syntax differ, so this verifies the table design and the queries, not the MySQL driver. Run campus_events.sql in MySQL and the JDBC version of the application on your computer for the final submission.

Output of python3 sql_logic_check.py

```
TABLES        : ['student', 'event', 'registration']
seed rows     : students=3 events=3 registrations=3

-- RULE 1: one registration per student per event (UNIQUE (reg_no, event_id))
   duplicate rejected  : UNIQUE constraint failed: registration.reg_no, registration.event_id
   same student, other event accepted: OK

-- RULE 2: registration must point to an existing student and event (FOREIGN KEY)
   (999999999, 1) rejected  : FOREIGN KEY constraint failed
   (192311001, 99) rejected  : FOREIGN KEY constraint failed

-- Upsert of the student row (MySQL: ON DUPLICATE KEY UPDATE; SQLite: ON CONFLICT DO UPDATE)
   ('192311001', 'Arun K', 'arun.new@example.com')

-- The paging query used by JdbcRegistrationDao.page()  (LIMIT 2 OFFSET 0, then OFFSET 2)
   (1, '192311001', 'Arun K', 'Tech Symposium 2026')
   (2, '192311002', 'Divya S', 'Tech Symposium 2026')
   ---
   (3, '192311003', 'Kavin Raj', 'Cultural Night')
   (4, '192311001', 'Arun K', 'Cultural Night')
   ---
-- Filter by event 1
   (1, '192311001', 'Arun K', 'Tech Symposium 2026')
   (2, '192311002', 'Divya S', 'Tech Symposium 2026')
```

```
   query plan: ['SEARCH e USING INTEGER PRIMARY KEY (rowid=?)', 'SEARCH r USING INDEX idx_reg_event
(event_id=?)', 'SEARCH s USING INDEX sqlite_autoindex_student_1 (reg_no=?)']

-- Cost of paging with 100,000 registrations (SQLite stand-in, one run each, milliseconds)
   all rows (no LIMIT)                                100004 rows    144.90 ms
   page of 50, OFFSET 0                                 50 rows      0.06 ms
   page of 50, OFFSET 50,000                            50 rows     15.39 ms
   page of 50, OFFSET 99,950 (last page)                50 rows     30.12 ms
```

Alternative Concept and Complexity Analysis

The alternative solution. Instead of one page at a time (SQL LIMIT/OFFSET), load **all** registrations at once and let JTable scroll:

```
// used: one page of 50 rows                                    // alternative: everything at once
rows = dao.page(eventId, 50, (page - 1) * 50);                  rows = dao.findAll(eventId);
// SELECT ... without LIMIT
model.setRowCount(0);                                           model.setRowCount(0);
for (RegistrationRow r : rows) model.addRow(...);               for (RegistrationRow r : rows)
model.addRow(...);   // N rows
```

N registrations, page size P	One page (used)	All rows at once
Fetch time	O(P) rows transferred, plus the cost of skipping OFFSET rows in the database	O(N) rows transferred
Work on the event-dispatch thread (fill the table model)	O(P)	O(N): the window is blocked while the model is filled, even if the fetch ran in a worker
Memory in the application	O(P)	O(N)
Search / sort across all rows in the window	needs a new query	free, but only while the data fit in memory

Measurements (PaginationBenchmark.java; the "database" is an in-memory DAO without network time, so the fetch times are lower than MySQL would give; the table-model work on the event-dispatch thread and its memory are real; average of 5 runs):

Rows N	Approach	Fetch (ms)	Event thread blocked: fill model (ms)	Event-thread allocation (MB)
1,000	all at once	0.01	1.09	0.16
1,000	page of 50	0.01	0.06	0.01
10,000	all at once	0.03	7.01	1.64
10,000	page of 50	0.01	0.02	0.01
100,000	all at once	0.10	36.97	13.21
100,000	page of 50	0.01	0.02	0.01

Table 6.3 – Everything at once against one page of 50 (from Task06/pagination_results.csv)

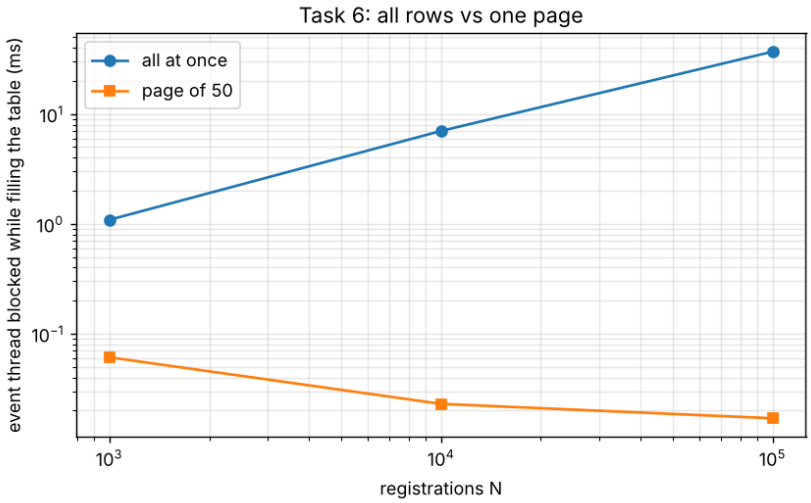


Figure 6.14 – Time the event-dispatch thread is blocked while the table is filled (logarithmic axes)

Analysis. Filling the table with 100,000 rows blocks the event-dispatch thread for 37.0 ms and allocates 13.2 MB, against 0.02 ms and 0.01 MB for one page: the all-at-once cost grows linearly with N (1,000 → 10,000 → 100,000 rows: 1.09 → 7.01 → 36.97 ms), the page cost does not depend on N. With a real MySQL server the fetch of 100,000 rows would add network and driver time (not measured here), which only strengthens the conclusion. **Cost of the paging itself:** LIMIT/OFFSET makes the database skip the first OFFSET rows, so deep pages are slower. In the SQLite check, a page of 50 took 0.06 ms at OFFSET 0, 15.39 ms at OFFSET 50,000 and 30.12 ms at OFFSET 99,950, while loading all 100,000 rows took 144.90 ms (O(offset + P) for a page against O(N) for everything). If the committee ever has millions of registrations, the refinement is keyset paging (WHERE reg_id > ? ORDER BY reg_id LIMIT 50), which is O(P) at any depth. **Effect on the responsiveness of the window.** *One page in a SwingWorker (used):* the fetch runs on the worker thread and the event thread only fills 50 rows (about 0.02 ms), so the window answered every request within 2 ms while a slow database was being read (REQ-11). *All rows at once:* even if the fetch is moved to a worker, the table model of 100,000 rows must be filled on the event thread, which blocks it for the measured time above (about two frames of animation at 60 frames per second for this in-memory stand-in; with a real JTable rendering and a real database the pause is longer); and if the fetch is done directly in the button's listener, the window freezes for the whole fetch plus the fill (hundreds of milliseconds up to seconds with MySQL; not measured here). **Evaluation:** paging is the right design for this application; loading everything is acceptable only for a few hundred rows.

Refinements made after tracing

- A status message was wiped by a listener (found in Task 2 by tracing events); the same “reset first, message second” order is used in RegisterPanel.
- The Exit menu item first built a synthetic WindowEvent; it was replaced by one method requestExit() shared by the menu item and the window-closing handler (shorter, testable).
- The table header did not appear in the off-screen render because JTable installs it only when the window is shown; the header is now set explicitly (scroll.setColumnHeaderView) and the columns have sensible widths.

Known limitations (honest list)

- The JDBC classes and the MySQL script were not executed against a MySQL server in the preparation of this report (see the SQLite check); they must be run once on your computer.
- Student rows are updated when the same register number is used with a new name or e-mail (ON DUPLICATE KEY UPDATE); a real system would also authenticate the committee desk.
- The export writes the whole selection in the worker thread but has no progress bar.

Appendix A – Repository Layout, How to Run, GitHub Steps

```
CSA09-<RegisterNumber>-Assignment4/
  README.md   GITHUB_STEPS.md
  Task01/  FeeService, FeePanel, FeeWindow, repositories, tests, schema.sql, outputs,
dispatch_results.csv
  Task02/  AppointmentService, AppointmentPanel, AppointmentWindow, WindowTrace, schema.sql,
outputs, cache_results.csv
  Task03/  LibraryScreen, LibraryNested, LibraryGridBag, LibraryAbsolute, LibraryFrame, outputs,
layout_results.csv
  Task04/  DrawPadBuggy, DrawPadStep2, DrawPad, DrawPadBuffered, errors/, compile outputs,
repaint_results.csv
  Task05/  ConverterA..D, ConverterA2..D2, ConverterE, ConverterF, Converter (Complete the Code),
outputs, change_effort.txt
  Task06/  src/campus/reg/{model,dao,service,ui}, sql/campus_events.sql, sql/sql_logic_check.py,
test/, outputs
  diagrams/    ER diagrams, class / container / flow diagrams (Graphviz .dot + .png)
  screenshots/ one folder per task (off-screen renders; replace by real screenshots)
  graphs/      figures of the measurements
  tools/       headless shims, harness, run_headless.sh
  report/      this report
```

README.md of the repository

```
# CSA09 – Programming in Java – Assignment 4 (CO4): Event-Driven GUI Design

| | |
|---|---|
| Name | <YOUR NAME> |
| Register number | <REGISTER NUMBER> |
| Java version | OpenJDK 21 (check with `java -version`) |
| Repository | CSA09-<RegisterNumber>-Assignment4 |

One folder per task: Java source, SQL scripts, captured console output (`*_output.txt`), CSV measurements
and screenshots
(`screenshots/taskNN/`). The report is in `report/`. Diagrams (ER, class, container, flow) are in
`diagrams/`.

## How to compile and run (on a computer with a display)
Open a terminal in the task folder.

| Task | Commands | Result (short note) |
|---|---|---|
| 1 | `cd Task01` · `javac *.java` (exclude `FeeHarness.java`) · `java FeeServiceTest` · `java -
Dfee.demo=true FeeWindow` | Service tests 12/12 pass. Window works without a database in demo mode;
MySQL: `-Dfee.url=... -Dfee.user=... -Dfee.pwd=...` and put `mysql-connector-j-8.x.jar` on the class
path. |
| 2 | `cd Task02` · `javac *.java` · `java AppointmentServiceTest` · `java AppointmentWindow` · `java
WindowTrace` | Doctors cached in a Map; window-event trace printed by `WindowTrace`. |
| 3 | `cd Task03` · `javac *.java` · `java LibraryFrame nested` (or `gridbag`, `absolute`) | Resize the
window: results grow, search row and buttons keep their size (nested design). |
| 4 | `cd Task04` · `javac DrawPad*.java` · `java DrawPad` · `javac errors/DrawPadWithOverride.java`
(shows the compiler error) | Dots survive minimise/restore. |
| 5 | `cd Task05` · `javac Converter*.java` · `java Converter` (and `ConverterA` … `ConverterF`) |
Convert button and Enter key use one lambda listener. |
| 6 | `cd Task06` · create the database: `mysql -u root -p < sql/campus_events.sql` · `javac -d out $
(find src -name "*.java")` · `java -Ddemo=true -cp out campus.reg.App` (no database) or `java -
Ddb.url=jdbc:mysql://localhost:3306/campus_events -Ddb.user=root -Ddb.pwd=... -cp "out;mysql-connector-j-
8.x.jar" campus.reg.App` | Register, view, filter, page and export registrations. |

(On Linux / macOS use `:` instead of `;` in the class path.)

## Notes about the evidence in this repository
```

```
* `tools/headless-shims`, `tools/harness`, `tools/run_headless.sh`: a test harness that runs the real GUI
classes **without a display**
  and renders them to PNG. The screenshots in `screenshots/` were produced this way (off-screen renders).
Retake real screenshots on your own
  computer for the final submission.
* `Task06/sql/sql_logic_check.py` checks the SQL logic with SQLite (no MySQL server was available); run
`campus_events.sql` in MySQL as well.
```

GitHub steps

```
# GitHub submission steps (commit progressively – at least one commit per task)

```
git init
git branch -M main
git remote add origin https://github.com/<username>/CSA09-<RegisterNumber>-Assignment4.git

git add Task01 diagrams/er_task1.png diagrams/class_task1.png && git commit -m "Task 1: diagnosis and
repaired fee window"
git add Task02 && git commit -m "Task 2: appointment desk design and implementation"
git add Task03 && git commit -m "Task 3: library search screen layouts"
git add Task04 && git commit -m "Task 4: DrawPad forensics and Complete the Code"
git add Task05 && git commit -m "Task 5: converter event-handling styles"
git add Task06 && git commit -m "Task 6: campus event registration application"
git add README.md diagrams screenshots tools report && git commit -m "Add README, diagrams, screenshots
and report"
git push -u origin main
```
Do the commits when you finish each task on your own computer (the commit history is checked). Put the
repository link on the cover page.
```

Appendix B – Test Harness, Shims and Benchmarks

These files are **not part of the applications**. They exist so that the real GUI classes could be executed on a server without a display. The application sources use only the standard Java API and were also compiled with the plain JDK (javac) without the shims.

B.1 Headless shims and how the harness runs

tools/run_headless.sh

```
#!/bin/sh
# Usage: tools/run_headless.sh <task-folder> <MainClass> [args...]
# Compiles the task's .java files (plus the harness helpers) against the headless shims and runs
MainClass.
# This is ONLY for machines without a display. On a normal PC just use javac / java.
ROOT=$(cd "$(dirname "$0")/.." && pwd)
SH=$ROOT/tools/headless-shims
SO=$ROOT/tools/.shimout
JC="java -m jdk.compiler/com.sun.tools.javac.Main"
if [ ! -d "$SO" ]; then
  mkdir -p "$SO"
  $JC --patch-module java.desktop=$SH -d "$SO" $(find $SH -name '*.java') || exit 1
fi
TASK=$1; MAIN=$2; shift 2
OUT=$ROOT/$TASK/.out_headless
rm -rf "$OUT"; mkdir -p "$OUT"
$JC --patch-module java.desktop=$SO -d "$OUT" $(find $ROOT/$TASK -name '*.java' -not -path
'*/errors/*') $ROOT/tools/harness/*.java 2>&1 | grep -v "^Note:"
cd $ROOT/$TASK
java --patch-module java.desktop=$SO -Djava.awt.headless=true -cp "$OUT" "$MAIN" "$@"
```

tools/headless-shims/java/awt/Frame.java - test-only stand-in for the native Frame

```
package java.awt;
import java.awt.event.*;
import java.awt.image.BufferedImage;
import java.util.ArrayList;
import java.util.List;
/** TEST-ONLY stand-in for java.awt.Frame so GUI programs can run in a headless JVM (no display,
no native peers). */
public class Frame extends Panel {
    public static final int NORMAL = 0, ICONIFIED = 1;
    private String title = "";
    private final List<WindowListener> wl = new ArrayList<>();
    public Frame() { this(""); }
    public Frame(String title) { super(new BorderLayout()); this.title = title; }
    public Frame(String title, GraphicsConfiguration gc) { this(title); }
    public String getTitle() { return title; }
    public void setTitle(String t) { title = t; }
    public synchronized void addWindowListener(WindowListener l) { wl.add(l); }
    public void addWindowFocusListener(WindowFocusListener l) { }
    public void addWindowStateListener(WindowStateListener l) { }
    public void setExtendedState(int s) { }
    public void setIconImage(Image i) { }
    public void setResizable(boolean b) { }
    public void setLocationRelativeTo(Component c) { }
    public void setDefaultLookAndFeelDecorated(boolean b) { }
    public void pack() { setSize(getPreferredSize()); layoutAll(this); }
    public void dispose() { }
    public static void layoutAll(Container c) { c.doLayout(); for (Component k :
c.getComponents()) if (k instanceof Container) layoutAll((Container) k); }
    /** Paint the whole component tree into an image (what the screen would show). */
    public BufferedImage snapshot() {
        layoutAll(this);
```

```
        BufferedImage im = new BufferedImage(Math.max(1, getWidth()), Math.max(1, getHeight()),
BufferedImage.TYPE_INT_RGB);
        Graphics2D g = im.createGraphics(); g.setColor(Color.WHITE); g.fillRect(0, 0, getWidth(),
getHeight());
        g.setColor(Color.BLACK);                    // a real window sets its foreground colour
before paint()
        g.setRenderingHint(RenderingHints.KEY_TEXT_ANTIALIASING,
RenderingHints.VALUE_TEXT_ANTIALIAS_ON);
        paint(g);                                   // the frame's own paint() (user code may draw
here)
        for (int i = getComponentCount() - 1; i >= 0; i--) {   // then the children, as native
peers would draw themselves
            Component k = getComponent(i);
            if (!k.isVisible()) continue;
            Graphics cg = g.create(k.getX(), k.getY(), k.getWidth(), k.getHeight());
            k.paint(cg); cg.dispose();
        }
        g.dispose(); return im;
    }
}
```

tools/headless-shims/java/awt/Label.java

```
package java.awt;
/** TEST-ONLY stand-in for java.awt.Label (no native peer): paints its text itself. */
public class Label extends Component {
    public static final int LEFT = 0, CENTER = 1, RIGHT = 2;
    private String text; private int alignment;
    public Label() { this(""); }
    public Label(String t) { this(t, LEFT); }
    public Label(String t, int a) { text = t == null ? "" : t; alignment = a; }
    public String getText() { return text; }
    public void setText(String t) { text = t; }
    public int getAlignment() { return alignment; }
    public void setAlignment(int a) { alignment = a; }
    public Dimension getPreferredSize() { FontMetrics fm = getFontMetrics(getFont() == null ? new
Font("Dialog", Font.PLAIN, 12) : getFont()); return new Dimension(fm.stringWidth(text) + 8,
fm.getHeight() + 4); }
    public Dimension getMinimumSize() { return getPreferredSize(); }
    public void paint(Graphics g) { g.setColor(Color.BLACK); g.setFont(getFont() == null ? new
Font("Dialog", Font.PLAIN, 12) : getFont()); g.drawString(text, 4, g.getFontMetrics().getAscent()
+ 2); }
}
```

tools/headless-shims/javax/swing/JOptionPane.java - records dialogs instead of showing them

```
package javax.swing;
import java.awt.Component;
import java.util.ArrayList;
import java.util.List;
/** TEST-ONLY stand-in for JOptionPane: no dialog is shown, the call is recorded and a scripted
answer is returned. */
public class JOptionPane {
    public static final int ERROR_MESSAGE = 0, INFORMATION_MESSAGE = 1, WARNING_MESSAGE = 2,
QUESTION_MESSAGE = 3, PLAIN_MESSAGE = -1;
    public static final int DEFAULT_OPTION = -1, YES_NO_OPTION = 0, YES_NO_CANCEL_OPTION = 1,
OK_CANCEL_OPTION = 2;
    public static final int YES_OPTION = 0, NO_OPTION = 1, CANCEL_OPTION = 2, OK_OPTION = 0,
CLOSED_OPTION = -1;
    public static final List<String> log = new ArrayList<>();
    public static int nextConfirm = YES_OPTION;
    public static String lastMessage = null, lastTitle = null;
    public static void showMessageDialog(Component p, Object m) { showMessageDialog(p, m,
"Message", INFORMATION_MESSAGE); }
    public static void showMessageDialog(Component p, Object m, String title, int type)
{ lastMessage = String.valueOf(m); lastTitle = title; log.add("[dialog:" + title + "] " + m); }
```

```
    public static int showConfirmDialog(Component p, Object m) { return showConfirmDialog(p, m,
"Confirm", YES_NO_OPTION); }
    public static int showConfirmDialog(Component p, Object m, String title, int option)
{ lastMessage = String.valueOf(m); lastTitle = title; log.add("[confirm:" + title + "] " + m + " -
> " + nextConfirm); return nextConfirm; }
}
```

tools/harness/HeadlessRender.java

```
import java.awt.*;
import java.awt.image.BufferedImage;
import javax.swing.JComponent;
import javax.imageio.ImageIO;
import java.io.File;

/** Test harness helper: paints a Swing component tree off-screen into a PNG (no display needed).
*/
public class HeadlessRender {
    public static void layoutAll(Container c) {
        c.doLayout();
        for (Component k : c.getComponents()) if (k instanceof Container) layoutAll((Container)
k);
    }
    /** Lays out comp at w x h, paints it and adds a simple title bar so the picture looks like a
window. */
    public static BufferedImage render(JComponent comp, int w, int h, String title) {
        comp.setSize(w, h);
        layoutAll(comp);
        return withTitle(paint(comp, w, h), title);
    }
    public static BufferedImage paint(Component comp, int w, int h) {
        BufferedImage im = new BufferedImage(Math.max(1, w), Math.max(1, h),
BufferedImage.TYPE_INT_RGB);
        Graphics2D g = im.createGraphics();
        g.setColor(Color.WHITE); g.fillRect(0, 0, w, h);
        g.setRenderingHint(RenderingHints.KEY_TEXT_ANTIALIASING,
RenderingHints.VALUE_TEXT_ANTIALIAS_ON);
        comp.paint(g);
        g.dispose();
        return im;
    }
    public static BufferedImage withTitle(BufferedImage body, String title) {
        int bar = 26;
        BufferedImage out = new BufferedImage(body.getWidth(), body.getHeight() + bar,
BufferedImage.TYPE_INT_RGB);
        Graphics2D g = out.createGraphics();
        g.setRenderingHint(RenderingHints.KEY_TEXT_ANTIALIASING,
RenderingHints.VALUE_TEXT_ANTIALIAS_ON);
        g.setColor(new Color(0xDDE3EC)); g.fillRect(0, 0, out.getWidth(), bar);
        g.setColor(new Color(0x333333)); g.setFont(new Font("Dialog", Font.BOLD, 12));
g.drawString(title, 10, 17);
        g.setColor(new Color(0xC0392B)); g.fillOval(out.getWidth() - 22, 7, 12, 12);
        g.setColor(new Color(0xF1C40F)); g.fillOval(out.getWidth() - 40, 7, 12, 12);
        g.drawImage(body, 0, bar, null);
        g.setColor(new Color(0x888888)); g.drawRect(0, 0, out.getWidth() - 1, out.getHeight() -
1);
        g.dispose();
        return out;
    }
    public static void save(BufferedImage im, String path) throws Exception {
        new File(path).getParentFile().mkdirs();
        ImageIO.write(im, "png", new File(path));
    }
    public static void edtIdle() throws Exception { javax.swing.SwingUtilities.invokeAndWait(() ->
{ }); }
}
```

tools/harness/FrameRender.java

```
import java.awt.Frame;
import java.awt.image.BufferedImage;
/** Works only with the headless Frame shim (uses Frame.snapshot()). */
public class FrameRender {
    public static BufferedImage of(Frame f) { return HeadlessRender.withTitle(f.snapshot(),
f.getTitle()); }
}
```

B.2 Task harnesses (drive the real classes with real events)

Task01/FeeHarness.java

```
import java.awt.image.BufferedImage;
import javax.swing.SwingUtilities;

/** Headless demonstration: drives the real FeePanel with button clicks and renders it. Not part
of the application. */
public class FeeHarness {
    static void waitWhileBusy(FeePanel p) throws Exception {
        long t0 = System.currentTimeMillis();
        while (p.isBusy() && System.currentTimeMillis() - t0 < 10000) Thread.sleep(20);
        HeadlessRender.edtIdle();
    }

    public static void main(String[] args) throws Exception {
        String dir = "../screenshots/task01/";
        InMemoryPaymentRepository repo = new InMemoryPaymentRepository();
        FeeService service = new FeeService(repo, new SimulatedBank(1500));      // 1.5 s bank
delay
        final FeePanel[] holder = new FeePanel[1];
        SwingUtilities.invokeAndWait(() -> holder[0] = new FeePanel(service));
        FeePanel panel = holder[0];
        panel.setSize(430, 300);

        HeadlessRender.save(HeadlessRender.render(panel, 430, 300, "Student Fee Payment"), dir +
"t1_01_initial.png");
        System.out.println("[1] window opened: status=\"" + panel.statusText() + "\"");

        panel.payAction().actionPerformed(new java.awt.event.ActionEvent(panel, 0, "Pay"));   //
like clicking Pay with empty fields
        waitWhileBusy(panel);
        System.out.println("[2] Pay with empty fields -> status: " + panel.statusText());
        HeadlessRender.save(HeadlessRender.render(panel, 430, 300, "Student Fee Payment"), dir +
"t1_02_empty_error.png");

        panel.regNoField().setText("192311001");
        panel.amountField().setText("abc");
        panel.payAction().actionPerformed(new java.awt.event.ActionEvent(panel, 0, "Pay"));
        waitWhileBusy(panel);
        System.out.println("[3] Pay with amount 'abc'   -> status: " + panel.statusText());
        HeadlessRender.save(HeadlessRender.render(panel, 430, 300, "Student Fee Payment"), dir +
"t1_03_invalid_amount.png");

        panel.amountField().setText("15000");
        long start = System.currentTimeMillis();
        panel.payAction().actionPerformed(new java.awt.event.ActionEvent(panel, 0, "Pay"));
        // while the bank call is running, ask the event-dispatch thread to do something small and
time the answer
        long worst = 0; int pings = 0;
        while (panel.isBusy()) {
            long t = System.nanoTime();
            SwingUtilities.invokeAndWait(() -> { });
            worst = Math.max(worst, System.nanoTime() - t); pings++;
            Thread.sleep(50);
```

```
        }
        HeadlessRender.edtIdle();
        long total = System.currentTimeMillis() - start;
        System.out.println("[4] Pay 15000 (bank takes 1.5 s): finished after " + total + " ms;
status: " + panel.statusText());
        System.out.printf("    event-dispatch thread stayed responsive: %d pings during the
payment, slowest answer %.2f ms%n", pings, worst / 1e6);
        HeadlessRender.save(HeadlessRender.render(panel, 430, 300, "Student Fee Payment"), dir +
"t1_04_success.png");

        panel.clearAction().actionPerformed(new java.awt.event.ActionEvent(panel, 0, "Clear"));
        panel.regNoField().setText("192311001");
        panel.receiptAction().actionPerformed(new java.awt.event.ActionEvent(panel, 0,
"Receipt"));
        waitWhileBusy(panel);
        System.out.println("[5] Clear, then Receipt for 192311001 -> receipt area:");
        System.out.println("    " + panel.receiptText().replace("\n", "\n    "));
        HeadlessRender.save(HeadlessRender.render(panel, 430, 300, "Student Fee Payment"), dir +
"t1_05_receipt.png");
    }
}
```

Task02/AppointmentHarness.java

```
import java.awt.event.ActionEvent;
import javax.swing.*;

/** Headless demonstration of Task 2: drives the real AppointmentPanel / AppointmentWindow. Not
part of the application. */
public class AppointmentHarness {
    static void waitIdle(AppointmentPanel p) throws Exception {
        long t0 = System.currentTimeMillis();
        while (p.isBusy() && System.currentTimeMillis() - t0 < 5000) Thread.sleep(10);
        HeadlessRender.edtIdle();
    }
    static void shot(AppointmentPanel p, String name) throws Exception {
        HeadlessRender.save(HeadlessRender.render(p, 430, 250, "Outpatient Appointment Desk"),
"../screenshots/task02/" + name);
    }
    static String doctors(AppointmentPanel p) {
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < p.doctorBox().getItemCount(); i++) sb.append(i == 0 ? "" : ",
").append(p.doctorBox().getItemAt(i));
        return "[" + sb + "]";
    }
    public static void main(String[] args) throws Exception {
        AppointmentService service = new AppointmentService(new InMemoryAppointmentRepository());
        final AppointmentWindow[] w = new AppointmentWindow[1];
        SwingUtilities.invokeAndWait(() -> w[0] = new AppointmentWindow(service));      // JFrame
(headless shim)
        AppointmentPanel p = w[0].panel();
        p.setSize(430, 250);
        shot(p, "t2_01_initial.png");
        System.out.println("[1] window created. Doctor list: " + doctors(p));

        SwingUtilities.invokeAndWait(() -> p.departmentBox().setSelectedItem("Cardiology"));
        System.out.println("[2] department = Cardiology  -> doctor list: " + doctors(p));
        shot(p, "t2_02_cardiology.png");
        SwingUtilities.invokeAndWait(() -> p.departmentBox().setSelectedItem("Paediatrics"));
        System.out.println("[3] department = Paediatrics -> doctor list: " + doctors(p));
        shot(p, "t2_03_paediatrics.png");

        SwingUtilities.invokeAndWait(() -> p.bookButton().doClick());
        waitIdle(p);
        System.out.println("[4] Book with empty patient ID -> " + p.statusText());
        shot(p, "t2_04_error.png");
```

```
        SwingUtilities.invokeAndWait(() -> { p.patientIdField().setText("P1001");
p.doctorBox().setSelectedIndex(1); p.slotBox().setSelectedItem("10:00"); });
        System.out.println("[5] in-progress booking? " + p.isBookingInProgress() + "  -> closing
asks for confirmation");
        JOptionPane.nextConfirm = JOptionPane.NO_OPTION;
        System.out.println("    receptionist answers NO  -> window closes? " +
w[0].confirmClose());
        JOptionPane.nextConfirm = JOptionPane.YES_OPTION;
        System.out.println("    receptionist answers YES -> window closes? " +
w[0].confirmClose());
        for (String line : JOptionPane.log) System.out.println("    " + line);
        JOptionPane.log.clear();

        SwingUtilities.invokeAndWait(() -> p.bookButton().doClick());
        waitIdle(p);
        System.out.println("[6] Book -> " + p.statusText());
        shot(p, "t2_05_success.png");
        System.out.println("[7] form cleared after success; in-progress now? " +
p.isBookingInProgress() + "  -> closing needs no question: " + w[0].confirmClose());

        SwingUtilities.invokeAndWait(() -> { p.patientIdField().setText("P1002");
p.departmentBox().setSelectedItem("Paediatrics"); p.doctorBox().setSelectedIndex(1);
p.slotBox().setSelectedItem("10:00"); });
        SwingUtilities.invokeAndWait(() -> p.bookButton().doClick());
        waitIdle(p);
        System.out.println("[8] Same doctor and slot, other patient -> " + p.statusText());
        shot(p, "t2_06_slot_taken.png");

        System.out.println();
        System.out.println("Events handled by the panel, in order:");
        int i = 1;
        for (String line : p.trace()) System.out.println("  " + (i++) + ". " + line);
    }
}
```

Task03/ResizeHarness.java

```
import java.awt.*;
import java.awt.image.BufferedImage;
import javax.swing.*;

/** Lays out each design at three window sizes, prints the region sizes and saves a picture. Not
part of the application. */
public class ResizeHarness {
    static String dim(Component c) { return c.getWidth() + "x" + c.getHeight(); }

    public static void main(String[] args) throws Exception {
        int[][] sizes = {{520, 330}, {800, 500}, {1100, 650}};
        String[] names = {"nested", "gridbag", "absolute"};
        System.out.println("Region sizes (width x height in pixels) after layout at three window
sizes");
        for (String n : names) {
            System.out.println();
            System.out.println("DESIGN: " + n);
            System.out.printf("%-12s %-10s %-10s %-10s %-10s %-10s %-10s%n", "window", "results",
"search fld", "Search btn", "Prev btn", "status", "title");
            int firstResultsH = 0, firstResultsW = 0, firstBtnW = 0, firstBtnH = 0;
            for (int i = 0; i < sizes.length; i++) {
                final int w = sizes[i][0], h = sizes[i][1];
                final LibraryScreen[] s = new LibraryScreen[1];
                SwingUtilities.invokeAndWait(() -> s[0] = n.equals("nested") ? new LibraryNested()
: n.equals("gridbag") ? new LibraryGridBag() : new LibraryAbsolute());
                LibraryScreen screen = s[0];
                screen.setSize(w, h);
                HeadlessRender.layoutAll(screen);
```

```
                System.out.printf("%-12s %-10s %-10s %-10s %-10s %-10s %-10s%n", w + "x" + h,
dim(screen.resultsPane()), dim(screen.queryField()),
                        dim(screen.searchButton()), dim(screen.prevButton()),
dim(screen.statusLabel()), dim(screen.titleLabel()));
                if (i == 0) { firstResultsW = screen.resultsPane().getWidth(); firstResultsH =
screen.resultsPane().getHeight(); firstBtnW = screen.searchButton().getWidth(); firstBtnH =
screen.searchButton().getHeight(); }
                else {
                    boolean grew = screen.resultsPane().getWidth() > firstResultsW &&
screen.resultsPane().getHeight() > firstResultsH;
                    boolean btnSame = screen.searchButton().getWidth() == firstBtnW &&
screen.searchButton().getHeight() == firstBtnH;
                    System.out.println("             results grew: " + grew + "   Search button
kept its size: " + btnSame);
                }
                screen.searchButton().doClick();                      // fill the results area so
the picture shows data
                HeadlessRender.save(HeadlessRender.render(screen, w, h, "Campus Library Search - "
+ n + " - " + w + "x" + h),
                        "../screenshots/task03/t3_" + n + "_" + w + "x" + h + ".png");
            }
        }
    }
}
```

Task04/DrawPadHarness.java

```
import java.awt.*;
import java.awt.event.MouseEvent;
import java.awt.image.BufferedImage;
import java.util.Arrays;

/**
 * Headless demonstration of Task 4 (not part of the application). The window events are real
MouseEvents sent to the real classes.
 * "Minimise and restore" is imitated by clearing what is on the screen and asking the frame to
paint itself again.
 */
public class DrawPadHarness {
    static final int[][] CLICKS = {{60, 70}, {150, 120}, {240, 90}, {320, 200}, {100, 220}};
    static final String DIR = "../screenshots/task04/";

    static void click(Frame f, int x, int y) {
        f.dispatchEvent(new MouseEvent(f, MouseEvent.MOUSE_CLICKED, System.currentTimeMillis(), 0,
x, y, 1, false));
    }
    static BufferedImage overlay(BufferedImage base, BufferedImage dots) {
        BufferedImage out = new BufferedImage(base.getWidth(), base.getHeight(),
BufferedImage.TYPE_INT_RGB);
        Graphics2D g = out.createGraphics(); g.drawImage(base, 0, 0, null); g.drawImage(dots, 0,
0, null); g.dispose(); return out;
    }

    public static void main(String[] args) throws Exception {
        // ---------- 1. the original program ----------
        DrawPadBuggy buggy = new DrawPadBuggy();
        for (int[] c : CLICKS) click(buggy, c[0], c[1]);
        System.out.println("=== 1. Original DrawPadBuggy: 5 mouse clicks sent to the window ===");
        System.out.println("count = " + buggy.count + "      label = \"" + buggy.status.getText()
+ "\"");
        Object listener = buggy.getMouseListeners()[0];
        System.out.println("methods declared by the listener class : " +
Arrays.toString(Arrays.stream(listener.getClass().getDeclaredMethods()).map(m ->
m.getName()).toArray()));
        boolean overrides = true;
```

```
        try { listener.getClass().getDeclaredMethod("mouseClicked", MouseEvent.class); } catch
(NoSuchMethodException e) { overrides = false; }
        System.out.println("listener overrides mouseClicked(MouseEvent) : " + overrides + "  ->
the inherited empty method of MouseAdapter runs");
        HeadlessRender.save(FrameRender.of(buggy), DIR + "t4_01_original_after_5_clicks.png");

        // ---------- 2. spelling fixed, but dots drawn with getGraphics() ----------
        final BufferedImage dotLayer = new BufferedImage(400, 300, BufferedImage.TYPE_INT_ARGB);
// what getGraphics() paints on
        DrawPadStep2 step2 = new DrawPadStep2() {
            @Override public Graphics getGraphics() { Graphics2D g = dotLayer.createGraphics();
g.setColor(Color.BLACK); return g; }
        };
        for (int[] c : CLICKS) click(step2, c[0], c[1]);
        System.out.println();
        System.out.println("=== 2. Spelling fixed, dots drawn with getGraphics() ===");
        System.out.println("after 5 clicks              : count = " + step2.count + ", label = \""
+ step2.status.getText() + "\", dots on screen = 5");
        HeadlessRender.save(HeadlessRender.withTitle(overlay(step2.snapshot(), dotLayer), "Draw
Pad"), DIR + "t4_02_step2_before_minimise.png");
        Graphics2D clear = dotLayer.createGraphics(); clear.setComposite(AlphaComposite.Clear);
clear.fillRect(0, 0, 400, 300); clear.dispose();   // the window system throws the pixels away
        System.out.println("after minimise + restore    : paint() runs again; Frame.paint knows
nothing about the dots -> dots on screen = 0, label still \"" + step2.status.getText() + "\"");
        HeadlessRender.save(HeadlessRender.withTitle(overlay(step2.snapshot(), dotLayer), "Draw
Pad"), DIR + "t4_03_step2_after_restore.png");

        // ---------- 3. completed DrawPad (Complete the Code) ----------
        DrawPad pad = new DrawPad();
        pad.setVisible(true);
        for (int[] c : CLICKS) click(pad, c[0], c[1]);
        System.out.println();
        System.out.println("=== 3. Completed DrawPad (TODO 1 and TODO 2) ===");
        System.out.println("after 5 clicks              : clickCount() = " + pad.clickCount() + ",
dots painted from the stored points");
        HeadlessRender.save(FrameRender.of(pad), DIR + "t4_04_drawpad_before_minimise.png");
        System.out.println("after minimise + restore    : paint() draws the stored points again ->
still 5 dots, same picture");
        HeadlessRender.save(FrameRender.of(pad), DIR + "t4_05_drawpad_after_restore.png");
        click(pad, 20, 20);
        System.out.println("one more click              : clickCount() = " + pad.clickCount());

        // ---------- 4. BufferedImage version ----------
        DrawPadBuffered buf = new DrawPadBuffered();
        buf.setVisible(true);
        for (int[] c : CLICKS) click(buf, c[0], c[1]);
        System.out.println();
        System.out.println("=== 4. DrawPadBuffered (off-screen image) ===");
        System.out.println("after 5 clicks              : clickCount() = " + buf.clickCount());
        HeadlessRender.save(FrameRender.of(buf), DIR + "t4_06_buffered_after_restore.png");
    }
}
```

Task05/ConverterHarness.java

```
import java.awt.Component;
import java.awt.Container;
import java.awt.Frame;
import java.awt.event.ActionListener;
import java.lang.reflect.Field;
import java.util.*;
import javax.swing.*;

/** Headless demonstration of Task 5: runs every converter version through real button clicks and
Enter key events. */
public class ConverterHarness {
```

```
    @SuppressWarnings("unchecked")
    static <T> T field(Object o, String name) throws Exception { Field f =
o.getClass().getDeclaredField(name); f.setAccessible(true); return (T) f.get(o); }

    static String run(Object win, String text, int currencyIndex, boolean useEnter) throws
Exception {
        JTextField amount = field(win, "amount");
        JComboBox<?> target = field(win, "target");
        JLabel result = field(win, "result");
        JButton button = field(win, "convertButton");
        amount.setText(text); target.setSelectedIndex(currencyIndex); result.setText(" ");
        JOptionPane.lastMessage = null;
        if (useEnter) amount.postActionEvent(); else button.doClick();
        return result.getText();
    }

    static JButton buttonOf(Object win) throws Exception {
        try { return field(win, "convertButton"); }
        catch (NoSuchFieldException e) {                                  // Converter.java keeps
the button in a local variable
            return findButton((Container) ((JFrame) win).getContentPane());
        }
    }
    static JButton findButton(Container c) {              // direct children only (a JComboBox
contains its own arrow button)
        for (Component k : c.getComponents()) if (k instanceof JButton) return (JButton) k;
        return null;
    }

    public static void main(String[] args) throws Exception {
        String[] names = {"ConverterA", "ConverterB", "ConverterC", "ConverterD", "ConverterA2",
"ConverterB2", "ConverterC2", "ConverterD2", "ConverterE", "ConverterF"};
        System.out.println("=== Task 5: every version, amount 5000 INR -> EUR (index 1) and
invalid text 'abc' ===");
        System.out.printf("%-12s %-12s %-14s %-14s %-14s %-22s%n", "class", "button click", "Enter
key", "invalid 'abc'", "dialog shown?", "dialog text");
        for (String n : names) {
            Object win = Class.forName(n).getDeclaredConstructor().newInstance();
            ((Frame) win).setVisible(true);
            String viaButton = run(win, "5000", 1, false);
            String dialog1 = JOptionPane.lastMessage;
            String viaEnter = run(win, "5000", 1, true);
            String dialog2 = JOptionPane.lastMessage;
            String bad = run(win, "abc", 1, false);
            boolean enterWorks = viaEnter.equals("55.00 EUR");
            System.out.printf("%-12s %-12s %-14s %-14s %-14s %-22s%n", n, viaButton, enterWorks ?
viaEnter : "(nothing)", bad,
                    dialog1 != null ? "yes" : "no", dialog1 != null ? dialog1 : "-");
        }

        // the "Complete the Code" version
        System.out.println();
        System.out.println("=== Complete the Code: Converter.java (lambda, one listener for button
and Enter) ===");
        Converter conv = new Converter();
        conv.setVisible(true);
        JTextField amount = field(conv, "amount"); JLabel result = field(conv, "result");
JComboBox<?> target = field(conv, "target");
        JButton button = buttonOf(conv);
        amount.setText("2500"); target.setSelectedIndex(0); button.doClick();
        System.out.println("2500 INR -> USD, button click : " + result.getText());
        HeadlessRender.save(FrameRender.of(conv),
"../screenshots/task05/t5_01_valid_conversion.png");
        amount.setText("5000"); target.setSelectedIndex(1); amount.postActionEvent();
        System.out.println("5000 INR -> EUR, Enter key    : " + result.getText());
```

```
        amount.setText("12abc"); button.doClick();
        System.out.println("'12abc'                       : " + result.getText());
        HeadlessRender.save(FrameRender.of(conv),
"../screenshots/task05/t5_02_invalid_amount.png");
        amount.setText(""); button.doClick();
        System.out.println("empty field                   : " + result.getText());
        System.out.println("listeners on the button / on the text field : " +
button.getActionListeners().length + " / " + amount.getActionListeners().length
                + "   same object? " + (button.getActionListeners()[0] ==
amount.getActionListeners()[0]));

        // listener objects per version (space) and number of triggers
        System.out.println();
        System.out.println("=== Listener objects registered per version (distinct objects) ===");
        for (String n : new String[]{"ConverterA2", "ConverterB2", "ConverterC2", "ConverterD2",
"ConverterE", "ConverterF"}) {
            Object win = Class.forName(n).getDeclaredConstructor().newInstance();
            JTextField a = field(win, "amount"); JButton b = field(win, "convertButton");
            Set<Object> distinct = Collections.newSetFromMap(new IdentityHashMap<>());
            int registrations = 0;
            for (ActionListener l : b.getActionListeners()) { distinct.add(l); registrations++; }
            for (ActionListener l : a.getActionListeners()) { distinct.add(l); registrations++; }
            if (n.equals("ConverterF")) {
                JMenuItem mi = field(win, "convertMenuItem");
                for (ActionListener l : mi.getActionListeners()) { distinct.add(l); registrations+
+; }
            }
            System.out.printf("%-12s registrations: %d   distinct listener objects: %d%n", n,
registrations, distinct.size());
        }
        // ConverterF shared Action
        Object f = Class.forName("ConverterF").getDeclaredConstructor().newInstance();
        JButton fb = field(f, "convertButton"); JMenuItem fm = field(f, "convertMenuItem");
JTextField fa = field(f, "amount"); JLabel fr = field(f, "result");
        fa.setText("100"); fm.doClick();
        System.out.println("ConverterF menu item click (same Action) -> result label: " +
fr.getText() + "; Action shared by button and menu item: " + (fb.getAction() == fm.getAction()));
    }
}
```

Task06/test/RegistrationHarness.java

```
import campus.reg.dao.InMemoryRegistrationDao;
import campus.reg.model.CampusEvent;
import campus.reg.service.RegistrationService;
import campus.reg.ui.*;
import java.awt.Component;
import javax.swing.*;

/** Headless demonstration of the whole Swing application (real MainFrame, RegisterPanel,
RegistrationsPanel, SwingWorkers). */
public class RegistrationHarness {
    static MainFrame frame;
    static RegisterPanel reg;
    static RegistrationsPanel list;
    static final String DIR = "../screenshots/task06/";

    static void waitIdle() throws Exception {
        long t0 = System.currentTimeMillis();
        while ((reg.isBusy() || list.isLoading()) && System.currentTimeMillis() - t0 < 5000)
Thread.sleep(10);
        HeadlessRender.edtIdle();
    }
    static void shot(String name) throws Exception { HeadlessRender.save(FrameRender.of(frame),
DIR + name); }
```

```
    static void click(JButton b) throws Exception { SwingUtilities.invokeAndWait(b::doClick);
waitIdle(); }
    static void fill(String r, String n, String m, int eventIndex) throws Exception {
        SwingUtilities.invokeAndWait(() -> { reg.regNoField().setText(r);
reg.nameField().setText(n); reg.emailField().setText(m);
reg.eventBox().setSelectedIndex(eventIndex); });
    }
    static void printTable() {
        for (int i = 0; i < list.table().getRowCount(); i++) {
            StringBuilder sb = new StringBuilder("      ");
            for (int c = 0; c < list.table().getColumnCount(); c++) sb.append(String.format("%-
22s", list.table().getValueAt(i, c)));
            System.out.println(sb);
        }
    }

    public static void main(String[] args) throws Exception {
        RegistrationService service = new
RegistrationService(InMemoryRegistrationDao.withDemoData());
        SwingUtilities.invokeAndWait(() -> { frame = new MainFrame(service, 5);
frame.setVisible(true); });
        reg = frame.registerPanel(); list = frame.listPanel();

        System.out.println("=== Menu bar ===");
        JMenuBar bar = frame.getJMenuBar();
        for (int i = 0; i < bar.getMenuCount(); i++) {
            JMenu m = bar.getMenu(i);
            StringBuilder sb = new StringBuilder(m.getText() + " : ");
            for (int k = 0; k < m.getItemCount(); k++) sb.append(m.getItem(k) == null ? "| " :
m.getItem(k).getText() + "  ");
            System.out.println("  " + sb);
        }
        System.out.println();
        shot("t6_01_register_screen.png");

        System.out.println("=== Stage 5 demonstration through the real window ===");
        click(reg.registerButton());
        System.out.println("[invalid]   empty form, click Register  -> " + reg.statusText());
        shot("t6_02_invalid_empty.png");

        fill("192319010", "Ravi Teja", "ravi.example.com", 3);
        click(reg.registerButton());
        System.out.println("[invalid]   bad e-mail                 -> " + reg.statusText());
        shot("t6_03_invalid_email.png");

        fill("192319010", "Ravi Teja", "ravi@example.com", 3);
        click(reg.registerButton());
        System.out.println("[normal]    valid form                 -> " + reg.statusText());
        shot("t6_04_normal_registration.png");

        fill("192319010", "Ravi Teja", "ravi@example.com", 3);
        click(reg.registerButton());
        System.out.println("[duplicate] same student, same event   -> " + reg.statusText());
        shot("t6_05_duplicate.png");

        SwingUtilities.invokeAndWait(() -> frame.showScreen(MainFrame.LIST));
        waitIdle();
        System.out.println("[table]     Registrations screen: " + list.pageLabel().getText() + ",
" + list.statusLabel().getText());
        printTable();
        shot("t6_06_registrations_page1.png");

        click(list.nextButton());
        System.out.println("[paging]    Next -> " + list.pageLabel().getText());
        printTable();
```

```
        shot("t6_07_registrations_page2.png");

        SwingUtilities.invokeAndWait(() ->
list.filterBox().setSelectedItem(service.events().get(1)));   // Cultural Night
        waitIdle();
        System.out.println("[filter]    event = Cultural Night -> " + list.pageLabel().getText() +
", " + list.statusLabel().getText());
        printTable();
        shot("t6_08_filter_cultural_night.png");

        // responsiveness: the same screen with a slow database (every call waits 400 ms)
        System.out.println();
        System.out.println("=== Responsiveness with a slow database (400 ms per call, count + page
= 2 calls) ===");
        InMemoryRegistrationDao slowDao = InMemoryRegistrationDao.withDemoData();
        slowDao.setLatency(400);
        RegistrationService slow = new RegistrationService(slowDao);
        final RegistrationsPanel[] holder = new RegistrationsPanel[1];
        SwingUtilities.invokeAndWait(() -> { holder[0] = new RegistrationsPanel(slow, 5);
holder[0].setEvents(java.util.List.of()); });
        RegistrationsPanel slowPanel = holder[0];
        long start = System.nanoTime();
        SwingUtilities.invokeAndWait(() -> slowPanel.load(1));
        int pings = 0; long worst = 0; String seen = "";
        while (slowPanel.isLoading()) {
            long t = System.nanoTime();
            SwingUtilities.invokeAndWait(() -> { });
            worst = Math.max(worst, System.nanoTime() - t); pings++;
            seen = slowPanel.statusLabel().getText();
            Thread.sleep(40);
        }
        HeadlessRender.edtIdle();
        System.out.printf("page loaded after %d ms; while loading the status showed \"%s\"; %d
pings to the event-dispatch thread, slowest answer %.2f ms%n",
                (System.nanoTime() - start) / 1000000, seen, pings, worst / 1e6);
        System.out.println("final status: " + slowPanel.statusLabel().getText() + " / " +
slowPanel.pageLabel().getText());
    }
}
```

B.3 Benchmarks used in the alternative-concept analyses

Task01/DispatchBenchmark.java

```
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.io.PrintWriter;
import java.lang.management.ManagementFactory;
import java.util.HashMap;
import java.util.Map;

/**
 * Alternative concept: one listener with an if-else chain on the action command  versus
 * one listener object per button.  Measures dispatch time (ns per event) for k buttons and the
memory for the listeners.
 */
public class DispatchBenchmark {
    static long sink = 0;

    static long allocated() {
        com.sun.management.ThreadMXBean b = (com.sun.management.ThreadMXBean)
ManagementFactory.getThreadMXBean();
        return b.getThreadAllocatedBytes(Thread.currentThread().getId());
    }
```

```
    // one shared listener with an if-else chain (written as a loop over the commands: same k
comparisons)
    static class ChainListener implements ActionListener {
        final String[] commands;
        ChainListener(String[] commands) { this.commands = commands; }
        public void actionPerformed(ActionEvent e) {
            String cmd = e.getActionCommand();
            for (int i = 0; i < commands.length; i++) {
                if (commands[i].equals(cmd)) { sink += i; return; }        // "if
(cmd.equals(...)) { ... }"
            }
        }
    }

    // each button has its own listener object: the event goes straight to the right one
    static class OwnListener implements ActionListener {
        final int id;
        OwnListener(int id) { this.id = id; }
        public void actionPerformed(ActionEvent e) { sink += id; }
    }

    public static void main(String[] args) throws Exception {
        int[] ks = {3, 10, 100, 1000, 10000};
        final int EVENTS = 200000;
        PrintWriter csv = new PrintWriter("dispatch_results.csv");

csv.println("k,chain_ns_per_event,map_ns_per_event,own_listener_ns_per_event,chain_listener_bytes,
own_listener_bytes");
        System.out.println("=== Dispatch time for the LAST button (worst case), ns per event,
average of 5 runs ===");
        System.out.printf("%8s %14s %14s %16s %16s %16s%n", "k", "if-else chain", "HashMap", "own
listener", "bytes: 1 listener", "bytes: k listeners");

        Object src = new Object();
        for (int w = 0; w < 3; w++) run(100, EVENTS / 4, src);        // warm-up
        for (int k : ks) {
            double chain = 0, map = 0, own = 0;
            for (int r = 0; r < 5; r++) { double[] t = run(k, EVENTS, src); chain += t[0]; map +=
t[1]; own += t[2]; }
            chain /= 5; map /= 5; own /= 5;

            // memory: one chain listener (+ the command array) against k separate listener
objects
            String[] cmds = new String[k];
            for (int i = 0; i < k; i++) cmds[i] = "cmd" + i;
            long a0 = allocated();
            ActionListener one = new ChainListener(cmds);
            long oneBytes = allocated() - a0;
            a0 = allocated();
            ActionListener[] many = new ActionListener[k];
            for (int i = 0; i < k; i++) many[i] = new OwnListener(i);
            long manyBytes = allocated() - a0;
            sink += one.hashCode() + many.length;

            System.out.printf("%8d %14.1f %14.1f %16.1f %16d %16d%n", k, chain, map, own,
oneBytes, manyBytes);
            csv.printf("%d,%.1f,%.1f,%.1f,%d,%d%n", k, chain, map, own, oneBytes, manyBytes);
        }
        csv.close();
        System.out.println("(checksum " + sink + ")");
    }

    // returns {chain, map, own} nanoseconds per event
    static double[] run(int k, int events, Object src) {
        String[] cmds = new String[k];
```

```
        ActionListener[] own = new ActionListener[k];
        Map<String, ActionListener> byName = new HashMap<>();
        for (int i = 0; i < k; i++) {
            cmds[i] = "cmd" + i;
            own[i] = new OwnListener(i);
            byName.put(cmds[i], own[i]);
        }
        ActionListener chain = new ChainListener(cmds);
        String last = "cmd" + (k - 1);
        ActionEvent ev = new ActionEvent(src, ActionEvent.ACTION_PERFORMED, last);

        long s = System.nanoTime();
        for (int i = 0; i < events; i++) chain.actionPerformed(ev);
        double t1 = (System.nanoTime() - s) / (double) events;

        s = System.nanoTime();
        for (int i = 0; i < events; i++) byName.get(ev.getActionCommand()).actionPerformed(ev);
        double t2 = (System.nanoTime() - s) / (double) events;

        ActionListener direct = own[k - 1];                   // a button already holds its own
listener
        s = System.nanoTime();
        for (int i = 0; i < events; i++) direct.actionPerformed(ev);
        double t3 = (System.nanoTime() - s) / (double) events;
        return new double[]{t1, t2, t3};
    }
}
```

Task02/CacheBenchmark.java

```
import java.io.PrintWriter;
import java.util.*;

/**
 * Alternative concept: doctors cached once in a Map<String, List<Doctor>>  versus  fetched from
the database on every
 * department change.  The "database" is InMemoryAppointmentRepository (a full scan per query,
plus an assumed round-trip time).
 */
public class CacheBenchmark {
    static long sink = 0;

    static long usedHeap() {
        for (int i = 0; i < 3; i++) System.gc();
        Runtime r = Runtime.getRuntime();
        return r.totalMemory() - r.freeMemory();
    }

    static InMemoryAppointmentRepository build(int d, int m, long latency) {
        InMemoryAppointmentRepository repo = new InMemoryAppointmentRepository(latency);
        int perDept = m / d;
        for (int i = 0; i < d; i++)
            for (int j = 0; j < perDept; j++) repo.addDoctor(new Doctor("D" + i + "_" + j, "Dr.
Name" + j, "Dept" + i));
        return repo;
    }

    public static void main(String[] args) throws Exception {
        int[][] cases = {{10, 1000}, {10, 100000}, {100, 100000}, {1000, 200000}};
        PrintWriter csv = new PrintWriter("cache_results.csv");

csv.println("departments,doctors,cache_ns_per_change,db_scan_ns_per_change,db_with_2ms_roundtrip_m
s,cache_extra_MB");
        System.out.println("=== Department change: cached Map  vs  query on every change ===");
        System.out.printf("%6s %9s %18s %20s %24s %16s%n", "d", "m", "Map lookup (ns)", "DB query
scan (ns)", "DB with 2 ms trip (ms)", "Map memory (MB)");
```

```
        for (int[] c : cases) {
            int d = c[0], m = c[1];
            long before = usedHeap();
            InMemoryAppointmentRepository repo = build(d, m, 0);
            long afterRepo = usedHeap();
            AppointmentService service = new AppointmentService(repo);          // builds the Map
(the cache)
            long afterCache = usedHeap();
            double cacheMB = (afterCache - afterRepo) / 1048576.0;

            String[] names = new String[d];
            for (int i = 0; i < d; i++) names[i] = "Dept" + i;
            int changes = 20000;
            for (int w = 0; w < 5000; w++) sink += service.doctorsFor(names[w % d]).size();   //
warm-up
            long s = System.nanoTime();
            for (int i = 0; i < changes; i++) sink += service.doctorsFor(names[i % d]).size();
            double mapNs = (System.nanoTime() - s) / (double) changes;

            int dbChanges = Math.max(20, 2000000 / m);                           // keep total run
time small
            s = System.nanoTime();
            for (int i = 0; i < dbChanges; i++) sink += repo.findDoctors(names[i % d]).size();
            double dbNs = (System.nanoTime() - s) / (double) dbChanges;
            double withTripMs = 2.0 + dbNs / 1e6;
            System.out.printf("%6d %9d %18.1f %20.1f %24.3f %16.2f%n", d, m, mapNs, dbNs,
withTripMs, cacheMB);
            csv.printf("%d,%d,%.1f,%.1f,%.3f,%.2f%n", d, m, mapNs, dbNs, withTripMs, cacheMB);
            sink += repo.slots().size();
        }
        csv.close();
        System.out.println("(checksum " + sink + ")");
    }
}
```

Task03/LayoutBenchmark.java

```
import java.awt.*;
import java.io.PrintWriter;
import javax.swing.*;

/** Alternative concept: layout computation time and number of container objects of the three
designs. */
public class LayoutBenchmark {
    static int containers(Component c) {
        int n = 0;
        if (c instanceof Container) { n = 1; for (Component k : ((Container) c).getComponents()) n
+= containers(k); }
        return n;
    }
    static int panels(Component c) {            // only the helper panels we created (JPanel), not
buttons / scroll panes
        int n = (c instanceof JPanel) ? 1 : 0;
        if (c instanceof Container) for (Component k : ((Container) c).getComponents()) n +=
panels(k);
        return n;
    }
    static void layoutAll(Container c) { c.doLayout(); for (Component k : c.getComponents()) if (k
instanceof Container) layoutAll((Container) k); }

    public static void main(String[] args) throws Exception {
        String[] names = {"nested", "gridbag", "absolute"};
        int resizes = 20000;
        PrintWriter csv = new PrintWriter("layout_results.csv");

csv.println("design,helper_panels,all_containers,components_total,microseconds_per_resize");
```

```
        System.out.println("=== Time for one resize (layout of the whole tree) and number of
container objects ===");
        System.out.printf("%-10s %14s %16s %14s %22s%n", "design", "helper JPanels", "all
containers", "components", "layout time (us/resize)");
        for (String n : names) {
            LibraryScreen s = n.equals("nested") ? new LibraryNested() : n.equals("gridbag") ? new
LibraryGridBag() : new LibraryAbsolute();
            for (int w = 0; w < 3000; w++) { s.setSize(500 + (w % 400), 300 + (w % 200));
layoutAll(s); }      // warm-up
            double total = 0;
            for (int r = 0; r < 5; r++) {
                long t = System.nanoTime();
                for (int i = 0; i < resizes; i++) { s.setSize(500 + (i % 400), 300 + (i % 200));
layoutAll(s); }
                total += (System.nanoTime() - t) / (double) resizes / 1000.0;
            }
            double us = total / 5;
            int hp = panels(s) - 1;                                  // minus the screen itself
            int all = containers(s);
            int comps = countAll(s);
            System.out.printf("%-10s %14d %16d %14d %22.2f%n", n, hp, all, comps, us);
            csv.printf("%s,%d,%d,%d,%.2f%n", n, hp, all, comps, us);
        }
        csv.close();
    }
    static int countAll(Component c) { int n = 1; if (c instanceof Container) for (Component k :
((Container) c).getComponents()) n += countAll(k); return n; }
}
```

Task04/RepaintBenchmark.java

```
import java.awt.*;
import java.awt.image.BufferedImage;
import java.io.PrintWriter;
import java.lang.management.ManagementFactory;
import java.util.ArrayList;
import java.util.List;
import java.util.Random;

/** Alternative concept: time per repaint and memory for n clicks - redraw n points  versus  copy
one off-screen image. */
public class RepaintBenchmark {
    static long sink = 0;
    static long allocated() {
        com.sun.management.ThreadMXBean b = (com.sun.management.ThreadMXBean)
ManagementFactory.getThreadMXBean();
        return b.getThreadAllocatedBytes(Thread.currentThread().getId());
    }

    public static void main(String[] args) throws Exception {
        int[] ns = {100, 1000, 10000, 100000, 1000000};
        int w = 400, h = 300;
        BufferedImage screen = new BufferedImage(w, h, BufferedImage.TYPE_INT_RGB);       //
stands for the window surface
        Graphics2D sg = screen.createGraphics();
        Random rnd = new Random(1);
        PrintWriter csv = new PrintWriter("repaint_results.csv");

csv.println("n,points_repaint_us,image_repaint_us,points_memory_bytes,image_memory_bytes");
        System.out.println("=== Time of ONE repaint (microseconds) and memory, window 400 x 300
===");
        System.out.printf("%10s %20s %20s %20s %20s%n", "n clicks", "redraw n points", "copy 1
image", "points memory (B)", "image memory (B)");

        for (int n : ns) {
            long a0 = allocated();
```

```
            List<Point> points = new ArrayList<>();
            for (int i = 0; i < n; i++) points.add(new Point(rnd.nextInt(w), rnd.nextInt(h)));
            long pointBytes = allocated() - a0;

            BufferedImage canvas = new BufferedImage(w, h, BufferedImage.TYPE_INT_ARGB);
            Graphics2D cg = canvas.createGraphics();
            for (Point p : points) cg.fillOval(p.x - 3, p.y - 3, 6, 6);                     //
the dots are drawn once
            cg.dispose();
            long imageBytes = (long) w * h * 4;

            int reps = n >= 100000 ? 5 : 50;
            for (int i = 0; i < 3; i++) { redraw(sg, points); sg.drawImage(canvas, 0, 0, null); }
// warm-up
            long t = System.nanoTime();
            for (int i = 0; i < reps; i++) redraw(sg, points);
            double pointsUs = (System.nanoTime() - t) / (double) reps / 1000.0;
            int reps2 = 2000;
            t = System.nanoTime();
            for (int i = 0; i < reps2; i++) sg.drawImage(canvas, 0, 0, null);
            double imageUs = (System.nanoTime() - t) / (double) reps2 / 1000.0;

            System.out.printf("%10d %20.1f %20.1f %20d %20d%n", n, pointsUs, imageUs, pointBytes,
imageBytes);
            csv.printf("%d,%.1f,%.1f,%d,%d%n", n, pointsUs, imageUs, pointBytes, imageBytes);
        }
        csv.close();
        System.out.println("Image memory for a full-HD window (1920 x 1080 x 4 bytes) = " + (1920L
* 1080 * 4) + " bytes");
        System.out.println("(checksum " + sink + ")");
    }

    static void redraw(Graphics2D g, List<Point> points) {
        g.setColor(Color.WHITE); g.fillRect(0, 0, 400, 300); g.setColor(Color.BLACK);
        for (Point p : points) g.fillOval(p.x - 3, p.y - 3, 6, 6);
        sink += points.size();
    }
}
```

Task06/test/PaginationBenchmark.java

```
import campus.reg.dao.InMemoryRegistrationDao;
import campus.reg.model.RegistrationRow;
import java.io.PrintWriter;
import java.util.List;
import javax.swing.SwingUtilities;
import javax.swing.table.DefaultTableModel;

/**
 * Alternative concept: load ALL 100,000 registrations into the JTable model at once  versus  one
page of 50 rows.
 * The "database" is an in-memory DAO (no network), so the fetch times are lower than MySQL would
give; the TABLE-MODEL work
 * (which must run on the event-dispatch thread) and the memory are real measurements.
 */
public class PaginationBenchmark {
    static final String[] COLS = {"Reg ID", "Register No", "Student", "Event", "Registered on"};
    static long sink = 0;

    static long allocated() {
        com.sun.management.ThreadMXBean b = (com.sun.management.ThreadMXBean)
java.lang.management.ManagementFactory.getThreadMXBean();
        return b.getThreadAllocatedBytes(Thread.currentThread().getId());
    }

    static DefaultTableModel fill(List<RegistrationRow> rows) {
```

```
        DefaultTableModel m = new DefaultTableModel(COLS, 0);
        for (RegistrationRow r : rows) m.addRow(new Object[]{r.regId(), r.regNo(),
r.studentName(), r.eventTitle(), r.registeredOn()});
        return m;
    }

    public static void main(String[] args) throws Exception {
        SwingUtilities.invokeAndWait(() -> { });                // start the event-dispatch
thread before measuring
        PrintWriter csv = new PrintWriter("pagination_results.csv");
        csv.println("rows,approach,fetch_ms,edt_model_fill_ms,edt_allocated_MB");
        System.out.println("=== Loading registrations into a JTable model: everything at once vs
one page of 50 (average of 5 runs) ===");
        System.out.printf("%9s %-14s %12s %24s %22s%n", "rows", "approach", "fetch (ms)", "EDT
blocked: fill (ms)", "EDT allocation (MB)");
        for (int n : new int[]{1000, 10000, 100000}) {
            InMemoryRegistrationDao dao = new InMemoryRegistrationDao(0);
            dao.bulkAdd(n);
            for (int w = 0; w < 5; w++) { timedFill(dao.page(null, 50, 0));
timedFill(dao.findAll(null)); }      // warm-up
            for (int mode = 0; mode < 2; mode++) {
                double fetch = 0, fillMs = 0, mb = 0;
                for (int r = 0; r < 5; r++) {
                    long t = System.nanoTime();
                    List<RegistrationRow> rows = (mode == 0) ?
dao.findAll(null)                // SQL without LIMIT
                                                              : dao.page(null, 50, n /
2);        // SQL ... LIMIT 50 OFFSET n/2
                    fetch += (System.nanoTime() - t) / 1e6;
                    double[] f = timedFill(rows);
                    fillMs += f[0]; mb += f[1];
                }
                String name = mode == 0 ? "all at once" : "page of 50";
                System.out.printf("%9d %-14s %12.2f %24.2f %22.2f%n", n, name, fetch / 5, fillMs /
5, mb / 5);
                csv.printf("%d,%s,%.3f,%.3f,%.3f%n", n, name, fetch / 5, fillMs / 5, mb / 5);
            }
        }
        csv.close();
        System.out.println("(checksum " + sink + ")");
    }

    /** Fills a table model ON the event-dispatch thread and returns {milliseconds the EDT was
busy, MB allocated by the EDT}. */
    static double[] timedFill(List<RegistrationRow> rows) throws Exception {
        final double[] out = new double[2];
        SwingUtilities.invokeAndWait(() -> {
            long a0 = allocated(), t = System.nanoTime();
            DefaultTableModel m = fill(rows);
            out[0] = (System.nanoTime() - t) / 1e6;
            out[1] = (allocated() - a0) / 1048576.0;
            sink += m.getRowCount();
        });
        return out;
    }
}
```

Task06/sql/sql_logic_check.py

```
"""
Checks the LOGIC of campus_events.sql (constraints, joins, paging) in SQLite, because no MySQL
server was available.
The MySQL script is converted mechanically (AUTO_INCREMENT, ENGINE, INDEX, CREATE DATABASE/USE).
SQLite is NOT MySQL:
error texts and some syntax differ, so this proves the table design and the queries, not the MySQL
driver or server.
```

```
Run:  python3 sql_logic_check.py
"""
import re, sqlite3, time

script = open("campus_events.sql").read()
script = re.sub(r"CREATE DATABASE[^;]*;", "", script)
script = re.sub(r"USE [^;]*;", "", script)
script = script.replace("INT AUTO_INCREMENT PRIMARY KEY", "INTEGER PRIMARY KEY AUTOINCREMENT")
script = re.sub(r"\)\s*ENGINE=InnoDB;", ");", script)
script = re.sub(r",\s*INDEX idx_reg_event \(event_id\)[^\n]*", "", script)

db = sqlite3.connect(":memory:")
db.execute("PRAGMA foreign_keys = ON")
db.executescript(script)
db.execute("CREATE INDEX idx_reg_event ON registration(event_id)")       # the INDEX line of the
MySQL script

print("TABLES        :", [r[0] for r in db.execute("SELECT name FROM sqlite_master WHERE
type='table' AND name NOT LIKE 'sqlite_%'")])
print("seed rows     : students=%d events=%d registrations=%d" % tuple(db.execute(f"SELECT
COUNT(*) FROM {t}").fetchone()[0] for t in ("student", "event", "registration")))

print("\n-- RULE 1: one registration per student per event (UNIQUE (reg_no, event_id))")
try:
    db.execute("INSERT INTO registration(reg_no, event_id) VALUES ('192311001', 1)")
    print("   duplicate accepted  <-- WRONG")
except sqlite3.IntegrityError as e:
    print("   duplicate rejected  :", e)
db.execute("INSERT INTO registration(reg_no, event_id) VALUES ('192311001', 2)")
print("   same student, other event accepted: OK")

print("\n-- RULE 2: registration must point to an existing student and event (FOREIGN KEY)")
for reg, ev in (("999999999", 1), ("192311001", 99)):
    try:
        db.execute("INSERT INTO registration(reg_no, event_id) VALUES (?, ?)", (reg, ev))
        print("   accepted <-- WRONG", reg, ev)
    except sqlite3.IntegrityError as e:
        print("   (%s, %d) rejected  :" % (reg, ev), e)

print("\n-- Upsert of the student row (MySQL: ON DUPLICATE KEY UPDATE; SQLite: ON CONFLICT DO
UPDATE)")
db.execute("INSERT INTO student(reg_no,name,email) VALUES ('192311001','Arun
K','arun.new@example.com') "
           "ON CONFLICT(reg_no) DO UPDATE SET name=excluded.name, email=excluded.email")
print("  ", db.execute("SELECT * FROM student WHERE reg_no='192311001'").fetchone())

PAGE_SQL = ("SELECT r.reg_id, r.reg_no, s.name, e.title, r.registered_on FROM registration r "
            "JOIN student s ON s.reg_no = r.reg_no JOIN event e ON e.event_id = r.event_id "
            "{where} ORDER BY r.reg_id LIMIT ? OFFSET ?")
print("\n-- The paging query used by JdbcRegistrationDao.page()  (LIMIT 2 OFFSET 0, then OFFSET
2)")
for off in (0, 2):
    for row in db.execute(PAGE_SQL.format(where=""), (2, off)): print("  ", row[:4])
    print("   ---")
print("-- Filter by event 1")
for row in db.execute(PAGE_SQL.format(where="WHERE r.event_id = ?"), (1, 10, 0)): print("  ",
row[:4])
print("   query plan:", [r[3] for r in db.execute("EXPLAIN QUERY PLAN " +
PAGE_SQL.format(where="WHERE r.event_id = ?"), (1, 10, 0))][:3])

print("\n-- Cost of paging with 100,000 registrations (SQLite stand-in, one run each,
milliseconds)")
db.execute("PRAGMA foreign_keys = OFF")
db.executemany("INSERT INTO student(reg_no,name,email) VALUES (?,?,?)", [("%09d" % (100000000 +
i), "Student %d" % i, "s%d@x.com" % i) for i in range(100000)])
```

```
db.executemany("INSERT INTO registration(reg_no,event_id) VALUES (?,1)", [("%09d" % (100000000 +
i),) for i in range(100000)])
def timed(label, sql, params):
    db.execute(sql, params).fetchall()                      # warm
    t = time.perf_counter(); rows = db.execute(sql, params).fetchall(); ms = (time.perf_counter()
- t) * 1000
    print("   %-52s %4d rows  %8.2f ms" % (label, len(rows), ms))
timed("all rows (no LIMIT)", PAGE_SQL.format(where="").replace("LIMIT ? OFFSET ?", "LIMIT -1
OFFSET 0"), ())
timed("page of 50, OFFSET 0", PAGE_SQL.format(where=""), (50, 0))
timed("page of 50, OFFSET 50,000", PAGE_SQL.format(where=""), (50, 50000))
timed("page of 50, OFFSET 99,950 (last page)", PAGE_SQL.format(where=""), (50, 99950))
```

Appendix C – Traceability and Viva Preparation

C.1 Requirement-to-evidence table

Task	Requirement / defect	Evidence in the report
1	D1–D10 diagnosed; repaired design	Table 1.1, Figure 1.1, test output, Figures 1.3–1.7
1	Window stays responsive (D5)	Output [4] of FeeHarness: 32 pings, slowest 1.28 ms; Figure 1.6
1	Alternative: listener per button vs chain	Table 1.2, Figure 1.8
2	Event model, hierarchy, pseudocode, trace	Tables 2.1–2.2, Figures 2.1–2.2, pseudocode, harness output
2	Alternative: cache vs query	Table 2.3, Figure 2.10
3	Results grow, search row and buttons keep size	resize_output.txt, Table 3.1, Figures 3.2–3.4
3	Alternative: GridBag / absolute	Figures 3.5–3.7, Table 3.2
4	Wrong method name; failing interaction	Output section 1, Figures 4.1–4.3
4	Complete the Code (TODO 1 and 2); @Override experiment	Figures 4.5–4.6; compiler outputs (1) and (2)
4	Alternative: BufferedImage	Table 4.1, Figure 4.7
5	Four styles; changed requirement; comparison	Converter output, Table 5.1, Table 5.2
5	Complete the Code; alternatives E and F	Figures 5.1–5.2; Table 5.3
6	REQ-01 … REQ-12	Table 6.1 with test identifiers; Figures 6.6–6.13; ServiceDemo output
6	Alternative: all rows at once	Table 6.3, Figure 6.14, SQL check output

C.2 Viva preparation – short answers

Question	Answer
Why is e.getActionCommand() == "Pay" wrong?	== compares references; two equal strings can be different objects. Use equals() – or better, give every button its own listener or Action.
Why must JDBC not run in the listener?	The listener runs on the event-dispatch thread; a slow call freezes painting and events. Use SwingWorker: doInBackground() off the EDT, done() on the EDT.
What is the difference between ItemEvent and ActionEvent on a JComboBox?	ItemEvent fires twice (DESELECTED, SELECTED) and tells the state change; ActionEvent fires once per user selection.
Why BorderLayout CENTER for the results area?	CENTER gets all the extra space; NORTH/SOUTH keep their preferred height. FlowLayout never stretches its components.
Why did the dots disappear after minimising?	They were drawn with getGraphics() and not stored. On repaint, paint() draws only what the program remembers. Store the points and draw them in paint().
What does @Override give?	The compiler checks that the method really overrides: mouseclicked (small c) gives “method does not override or implement a method from a supertype”.
Which listener style for a large application?	Lambdas/method references that call controller methods, and one shared Action for commands with several triggers; not “frame implements ActionListener”.
How is a duplicate registration prevented?	Service pre-check for a friendly message, and the UNIQUE (reg_no, event_id) key as the final guard (it catches two clients registering at the same time).
Why paging with LIMIT / OFFSET?	Fetch time, memory and the time the EDT is blocked depend on the page size, not on the table size; deep OFFSET still costs O(offset) – keyset paging fixes that.

C.3 Final checklist before submission

- Replace every <placeholder> (name, register number, college, GitHub link, date).
- Run every program on your own computer; retake the screenshots from the real windows; run WindowTrace and paste its output in Task 2.4.
- Create the MySQL database with campus_events.sql, run the JDBC version of Task 6 and add one screenshot of the data in MySQL.
- Create the GitHub repository CSA09-<RegisterNumber>-Assignment4 and commit task by task.